

# Chapter 1

## Chapter-1 Introduction to Data Science

### What is Data Science?

#### 1.1 Definition and Scope

Data Science is a multidisciplinary field that uses scientific methods, algorithms, processes, and systems to extract knowledge and insights from structured and unstructured data. It sits at the intersection of three core areas:

- **Statistics and Mathematics** – Provides the theoretical foundation for data analysis, probability, distributions, hypothesis testing, and regression.
- **Computer Science and Programming** – Enables data manipulation, algorithm implementation, scalable computing, and automation using languages like Python and R.
- **Domain Expertise** – Allows meaningful interpretation of results within a specific context (finance, healthcare, engineering, etc.).

#### 1.2 Why Data Science Has Become Important

- Explosive growth of data – Every day, 2.5 quintillion bytes of data are created (social media, sensors, transactions, logs).
- Increased computing power – Cloud platforms, GPUs, and distributed computing (Hadoop, Spark) make large-scale analysis feasible.
- Affordable storage – Data storage costs have dropped dramatically.
- Competitive advantage – Organizations use data-driven decisions to outperform competitors.

#### 1.3 Core Activities in Data Science

| Activity                        | Description                                                   | Example                                     |
|---------------------------------|---------------------------------------------------------------|---------------------------------------------|
| Data Collection                 | Gather data from various sources                              | Web scraping, APIs, database queries        |
| Data Cleaning                   | Handle missing values, remove duplicates, fix inconsistencies | Replacing null ages with median             |
| Exploratory Data Analysis (EDA) | Visualize and summarize data characteristics                  | Histograms, box plots, correlation matrices |
| Feature Engineering             | Create new features from existing data                        | Extracting day-of-week from timestamp       |
| Modeling                        | Apply statistical or machine learning algorithms              | Linear regression, random forest            |
| Evaluation                      | Assess model performance                                      | Accuracy, precision, recall, RMSE           |
| Deployment                      | Put model into production                                     | REST API, batch prediction system           |
| Monitoring                      | Track model performance over time                             | Drift detection, retraining triggers        |

#### 1.0.1 1.4 Types of Data in Data Science

##### A. By Structure

- Structured – Tabular format (rows and columns). Examples: Excel files, SQL tables, CSV files.
- Semi-structured – Has tags or markers but not rigid tables. Examples: JSON, XML, HTML.
- Unstructured – No predefined format. Examples: text documents, images, videos, audio.

##### B. By Measurement Scale

- Nominal – Categories without order (colors, yes/no, zip codes).
- Ordinal – Categories with order (ratings: 1st, 2nd, 3rd; education level).
- Interval – Numeric with equal intervals but no true zero (temperature in Celsius).
- Ratio – Numeric with true zero (height, weight, age, income).

##### C. By Source

- Primary data – Collected directly for a specific purpose (surveys, experiments).
- Secondary data – Already existing (government databases, company logs, public datasets).

### 1.0.2 1.5 Common Terms in Data Science

| Term            | Definition                                          |
|-----------------|-----------------------------------------------------|
| Dataset         | A collection of related data records                |
| Observation/Row | A single entity or data point                       |
| Feature/Column  | An attribute or variable describing the observation |
| Label/Target    | The value to predict (in supervised learning)       |
| Training set    | Data used to learn model parameters                 |
| Validation set  | Data used to tune hyperparameters                   |
| Test set        | Data used for final evaluation                      |
| Overfitting     | Model learns noise instead of signal                |
| Underfitting    | Model is too simple to capture patterns             |

## Roles of a Data Scientist

### 2.1 Complete Data Science Team Structure

In a mature organization, multiple roles work together:

| Role             | Experience Level | Primary Tasks                                                                         | Typical Tools                               |
|------------------|------------------|---------------------------------------------------------------------------------------|---------------------------------------------|
| Data Analyst     | Entry to Mid     | Query data, build dashboards, create reports, answer business questions               | SQL, Excel, Tableau, Power BI               |
| Data Scientist   | Mid to Senior    | Build predictive models, experiment design, feature engineering, statistical analysis | Python, R, scikit-learn, TensorFlow, pandas |
| ML Engineer      | Mid to Senior    | Deploy models, build pipelines, MLOps, versioning, monitoring                         | Docker, Kubernetes, MLflow, Airflow         |
| Data Engineer    | Mid to Senior    | Build data pipelines, ETL processes, data warehousing                                 | Spark, Hadoop, Airflow, AWS Glue            |
| Business Analyst | Mid              | Translate business problems into data questions, gather requirements                  | Jira, Confluence, Excel                     |
| Domain Expert    | Varies           | Provide context, validate results, ensure actionable insights                         | Domain-specific tools                       |

### 2.2 Detailed Responsibilities of a Data Scientist

Daily and weekly tasks include:

- **Data Acquisition** – Write SQL queries to extract data from databases; use APIs to fetch real-time data; scrape websites when necessary.
- **Data Cleaning and Wrangling** – Handle missing values (drop or impute); detect and treat outliers; standardize formats; merge multiple data sources.
- **Exploratory Data Analysis (EDA)** – Generate summary statistics (mean, median, standard deviation); create visualizations (scatter plots, box plots, heatmaps); identify patterns, correlations, and anomalies.
- **Feature Engineering** – Create interaction terms; bin continuous variables; encode categorical variables; extract date/time components; create rolling statistics for time series.
- **Model Selection and Training** – Choose appropriate algorithms based on problem type (regression, classification, clustering); split data into train/validation/test sets; train models; tune hyperparameters using cross-validation.
- **Model Evaluation** – Calculate performance metrics (accuracy, precision, recall, F1, ROC-AUC, RMSE, MAE); compare multiple models; perform error analysis.
- **Interpretation and Communication** – Explain model predictions using SHAP or LIME; present findings to non-technical stakeholders; create visual summaries; document methodology.
- **Collaboration** – Work with data engineers to productionize models; consult with domain experts to validate assumptions; mentor junior team members.

### 2.3 Skills Required for a Data Scientist

Technical Skills (Hard Skills):

- **Programming:** Python (pandas, numpy, scikit-learn, matplotlib, seaborn) or R (tidyverse, caret)
- **SQL:** Joins, subqueries, window functions, aggregations
- **Statistics:** Descriptive statistics, probability, hypothesis testing, regression
- **Machine Learning:** Supervised (regression, classification) and unsupervised (clustering, PCA)
- **Data Visualization:** Matplotlib, seaborn, Plotly, Tableau
- **Big Data Tools:** Spark, Hadoop (for large-scale data)
- **Cloud Platforms:** AWS (S3, SageMaker), GCP (BigQuery), Azure

**Soft Skills:**

- **Problem-solving** – Breaking down complex business problems
- **Communication** – Explaining technical results to non-technical audiences
- **Curiosity** – Asking the right questions
- **Business acumen** – Understanding what drives value
- **Collaboration** – Working in cross-functional teams

**2.4 Typical Data Science Project Effort Distribution**

|                            |     |
|----------------------------|-----|
| Data Cleaning & Wrangling  | 60% |
| Exploratory Data Analysis  | 20% |
| Modeling & Validation      | 15% |
| Deployment & Communication | 5%  |

This distribution is often cited in industry surveys and reflects why data scientists spend most of their time on data preparation rather than modeling.

**Real-World Applications**

**3.1 Engineering Applications**

**Predictive Maintenance:**

- Sensors collect vibration, temperature, and pressure data from machinery.
- Data science models (anomaly detection, survival analysis, random forests) predict when equipment is likely to fail.
- Maintenance can be scheduled proactively, reducing downtime by 30–50%.

Example: Manufacturing plant with 1000 motors. Historical failure data + real-time sensor readings → model predicts failure probability for each motor. Alerts sent 7 days before predicted failure.

**Quality Control:**

- Computer vision models inspect products on assembly lines.
- Defects are detected in real-time (scratches, missing components, incorrect dimensions).
- Reduces waste and improves customer satisfaction.

**Structural Health Monitoring:**

- Strain gauges and accelerometers on bridges, buildings, and dams.
- Time-series analysis detects unusual patterns indicating structural weakness.
- Prevents catastrophic failures.

**3.2 Healthcare Applications**

**Disease Prediction and Diagnosis:**

- Electronic Health Records (EHR) contain patient demographics, lab results, medications, and diagnoses.
- Models predict risk of diseases like diabetes, heart disease, or sepsis.
- Early intervention improves outcomes and reduces costs.

Example: Hospital uses logistic regression on patient data (age, BMI, blood pressure, family history) → predicts 5-year heart attack risk with 85% accuracy. High-risk patients receive preventive counseling.

**Medical Imaging:**

- Deep learning (CNNs) analyzes X-rays, MRIs, CT scans, and pathology slides.
- Detects tumors, fractures, pneumonia, and retinal diseases.
- Can match or exceed radiologist accuracy for specific conditions.

**Personalized Medicine:**

- Genomic data + treatment outcomes → models predict which drug works best for which patient.
- Reduces trial-and-error prescribing.
- Improves treatment efficacy and reduces side effects.

**Hospital Operations:**

- Predict patient admission rates to manage staffing.
- Forecast length of stay for bed management.
- Identify readmission risk for follow-up planning.

### 3.3 Internet of Things (IoT) Applications

#### Smart Homes:

- Thermostats learn occupancy patterns and adjust temperature automatically.
- Energy consumption is optimized, saving 10–20% on bills.
- Anomaly detection identifies unusual appliance behavior (possible failure or energy waste).

#### Smart Cities:

- Traffic sensors + cameras + GPS data → real-time traffic flow optimization.
- Adaptive traffic lights reduce congestion by 25%.
- Parking sensors guide drivers to available spots, reducing circling time.

#### Wearable Devices:

- Fitness trackers and smartwatches collect heart rate, steps, sleep quality, and activity.
- Personalized health insights: “Your recovery score is low. Rest recommended today.”
- Fall detection automatically calls emergency services.

#### Industrial IoT (IIoT):

- Thousands of sensors on factory floors, oil rigs, and power plants.
- Real-time monitoring prevents accidents and optimizes production.
- Edge computing processes data locally for immediate alerts.

#### Agriculture (Precision Farming):

- Soil moisture sensors + weather data + crop health imagery → optimize irrigation.
- Reduce water usage by 30% while maintaining yield.
- Drone imagery identifies pest infestations early.

### 3.4 Finance Applications

#### Credit Scoring and Risk Assessment:

- Traditional credit scores (FICO) use limited features (payment history, debt).
- Data science models incorporate thousands of features (transaction history, spending patterns, employment stability).
- Better risk prediction reduces default rates and expands credit access.

#### Fraud Detection:

- Real-time analysis of transactions identifies suspicious patterns.
- Anomaly detection flags unusual purchase locations, amounts, or frequencies.
- Example: Credit card used in two different countries within 1 hour → flagged as potential fraud.

#### Algorithmic Trading:

- Models analyze market data, news sentiment, and social media.
- High-frequency trading algorithms execute thousands of trades per second.
- Requires careful risk management to avoid flash crashes.

#### Customer Segmentation and Targeting:

- Clustering algorithms group customers by behavior.
- Personalized product recommendations and interest rate offers.
- Improves customer retention and lifetime value.

#### Regulatory Compliance (Anti-Money Laundering):

- Network analysis identifies suspicious transaction chains.
- Models flag unusual patterns that may indicate money laundering or terrorist financing.

### 3.5 Other Important Applications

#### Retail and E-commerce:

- Recommendation systems ("Customers who bought this also bought...").
- Demand forecasting for inventory management.
- Dynamic pricing based on demand and competitor prices.
- Customer churn prediction with retention offers.

#### Transportation and Logistics:

- Route optimization for delivery trucks (saving fuel and time).
- Ride-hailing demand prediction (Uber, Lyft).
- Autonomous vehicles (computer vision + sensor fusion + decision making).

#### Marketing and Advertising:

- Customer lifetime value (CLV) prediction.
- Ad click-through rate (CTR) prediction for ad auctions.
- Sentiment analysis of social media mentions.
- A/B testing for campaign optimization.

#### Education:

- Student performance prediction for early intervention.
- Personalized learning paths based on mastery.
- Dropout risk identification.

#### Cybersecurity:

- Network intrusion detection (identifying unusual traffic patterns).
- User behavior analytics (detecting compromised accounts).
- Malware classification using file attributes.

## 4. Data Science Workflows

### 4.1 CRISP-DM (Cross-Industry Standard Process for Data Mining)

CRISP-DM was developed in 1996 and remains the most widely used process model for data science projects. It is iterative and non-linear – you can go back to earlier phases as you learn more.

#### Phase 1: Business Understanding

**Goal:** Understand project objectives and requirements from a business perspective, then convert this knowledge into a data science problem definition.

##### Activities:

- Determine business objectives (increase sales? reduce churn? detect fraud?)
- Assess the current situation (available resources, constraints, assumptions)
- Define success criteria in business terms (not just model accuracy)
- Produce a project plan with milestones and deliverables

**Example:** A bank wants to reduce credit card fraud. Business objective: Reduce fraud losses by 20% within 6 months. Success criteria: Fraud detection rate > 85% with false positive rate < 5%.

**Deliverable:** Project charter, business success criteria, data science goals (e.g., achieve 0.90 AUC on test set).

#### Phase 2: Data Understanding

**Goal:** Collect initial data, describe it, explore it, and verify its quality.

##### Activities:

- Collect raw data from available sources (databases, files, APIs)
- Load data into analysis environment (pandas DataFrame, R data frame)
- Describe data structure (number of rows, columns, data types)
- Calculate basic statistics (mean, median, min, max, standard deviation for numeric columns; value counts for categorical)
- Explore data visually (histograms, box plots, scatter plots)
- Verify data quality (missing values, duplicates, inconsistencies)

##### Example (Credit Card Fraud):

- Collect 6 months of transaction data (5 million rows, 30 columns)
- Data types: transaction\_id (string), amount (float), timestamp (datetime), merchant\_code (string), card\_present (boolean), etc.
- Summary statistics: avg amount \$85, median amount \$42, max \$9,999
- Missing values: 2% missing merchant\_code, 0.1% missing amount
- Visualizations: Distribution of transaction amounts (highly skewed), fraud rate by hour of day

**Deliverable:** Data understanding report, initial data quality assessment.

### Phase 3: Data Preparation

**Goal:** Transform raw data into a clean, feature-rich dataset suitable for modeling. This typically consumes 60–80% of project time.

#### Activities:

##### A. Data Cleaning:

- Handle missing values:
  - Drop rows with missing target values
  - For numeric features: impute with mean, median, or mode
  - For categorical features: impute with mode or create “missing” category
  - Advanced: KNN imputation or model-based imputation
- Remove duplicates (identical rows or near-duplicates)
- Fix data type errors (string numbers to float, date strings to datetime)
- Detect and treat outliers:
  - Z-score method (values  $> 3$  standard deviations)
  - IQR method (values outside  $1.5 \times \text{IQR}$ )
  - Domain knowledge (age  $> 150$  is impossible)

##### B. Data Transformation:

- Scaling/normalization (important for distance-based algorithms):
  - Min-Max scaling: values between 0 and 1
  - Standardization (Z-score): mean 0, standard deviation 1
  - Robust scaling (using median and IQR) – handles outliers better
- Handling skewed distributions: log transformation, Box-Cox transformation
- Encoding categorical variables:
  - Label encoding (assign 0,1,2... for ordinal categories)
  - One-hot encoding (binary columns for each category)
  - Target encoding (replace category with mean of target)

##### C. Feature Engineering (Creating new features):

- Date/time features: hour of day, day of week, month, day of year, is\_weekend
- Aggregations: customer lifetime total spend, average transaction amount, frequency of purchases
- Interaction features: product of two features (e.g., age  $\times$  income)
- Ratio features: debt-to-income ratio, body mass index (weight/height<sup>2</sup>)
- Text features: word count, sentiment score, TF-IDF vectors

##### D. Data Splitting:

- Training set (60–80%) – used to learn model parameters
- Validation set (10–20%) – used to tune hyperparameters
- Test set (10–20%) – used for final evaluation (untouched until end)

##### E. Handling Class Imbalance (for classification):

**Problem:** fraud is rare (0.1% of transactions)

##### Solutions:

- Undersampling majority class (randomly remove non-fraud)
- Oversampling minority class (duplicate fraud examples)
- SMOTE (Synthetic Minority Oversampling) – creates synthetic examples
- Use class weights in model training

**Deliverable:** Cleaned, transformed, feature-engineered dataset ready for modeling.

Phase 4: Modeling

**Goal:** Apply various modeling techniques, tune parameters, and select the best model.

**Activities:**

**A. Algorithm Selection (based on problem type):**

| Problem Type                                         | Algorithms                                                                      |
|------------------------------------------------------|---------------------------------------------------------------------------------|
| Binary Classification (fraud/churn/spam)             | Logistic Regression, Decision Tree, Random Forest, XGBoost, SVM, Neural Network |
| Multi-class Classification (digit/image recognition) | Random Forest, XGBoost, Neural Network, KNN                                     |
| Regression (price, demand, temperature)              | Linear Regression, Ridge/Lasso, Decision Tree, Random Forest, XGBoost           |
| Clustering (segmentation)                            | K-Means, Hierarchical, DBSCAN                                                   |
| Dimensionality Reduction                             | PCA, t-SNE, UMAP                                                                |

**B. Training and Validation:**

- Train initial baseline model (simple algorithm with default parameters)
- Use cross-validation (typically 5-fold or 10-fold) for stable performance estimates
- Iterate: adjust features, try different algorithms, tune hyperparameters

**C. Hyperparameter Tuning:**

- Grid Search: exhaustive search over parameter grid
- Random Search: random sampling of parameter space (more efficient)
- Bayesian Optimization: intelligent probabilistic search

**Common hyperparameters to tune:**

- Decision Tree: max\_depth, min\_samples\_split, min\_samples\_leaf
- Random Forest: n\_estimators, max\_features, max\_depth
- XGBoost: learning\_rate, n\_estimators, max\_depth, subsample
- Neural Network: number of layers, neurons per layer, learning rate, dropout rate

**D. Model Evaluation Metrics:**

| Problem Type                | Metrics                                                |
|-----------------------------|--------------------------------------------------------|
| Regression                  | MAE, RMSE, R-squared (R <sup>2</sup> ), MAPE           |
| Classification (balanced)   | Accuracy, Log Loss                                     |
| Classification (imbalanced) | Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix |
| Clustering                  | Inertia, Silhouette Score, Davies-Bouldin Index        |

**Confusion Matrix (Binary Classification):**

| Actual \ Predicted | Positive                           | Negative                            |
|--------------------|------------------------------------|-------------------------------------|
| Positive           | True Positive (TP)                 | False Negative (FN) – Type II Error |
| Negative           | False Positive (FP) – Type I Error | True Negative (TN)                  |

**Precision:**

$$\text{Precision} = \frac{TP}{TP + FP}$$

”Of those predicted positive, how many were actually positive?”

**Recall:**

$$\text{Recall} = \frac{TP}{TP + FN}$$

”Of actual positives, how many were correctly identified?”

**F1 Score:**

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(Harmonic mean of precision and recall)

**ROC-AUC:** ROC curve plots True Positive Rate vs False Positive Rate.

- AUC = 0.5 → random model
- AUC = 1.0 → perfect model
- Good models: AUC > 0.8

**Deliverable:** Best performing model with tuned hyperparameters and evaluated performance metrics.

**Phase 5: Evaluation****Goal:** Assess the model against business objectives and determine if it is good enough for deployment.**Activities:**

- Run final model on the unseen test set (one-time evaluation)
- Ensure test performance matches validation performance (check for overfitting)
- Translate technical metrics into business terms:
  - “95% accuracy” → “9,500 out of 10,000 transactions correctly classified”
  - “0.92 ROC-AUC” → “Model ranks fraud cases higher than 92% of non-fraud cases”
- Perform cost-benefit analysis:
  - Cost of false positive (wasted investigation time, customer inconvenience)
  - Cost of false negative (fraud loss, customer liability)
- Find optimal threshold to minimize total cost
- Review against original business success criteria
- Identify limitations or assumptions that could affect production performance

**Example (Fraud Detection Business Evaluation):**

- Test set: 100,000 transactions (100 fraud, 99,900 non-fraud)
- Model results: 85 fraud correctly detected (Recall 85%), 1,000 flagged as fraud (Precision 8.5%)

**Business impact:**

- Cost of false positives:  $915 \times \$5$  (manual review) = \$4,575
- Savings from detected fraud:  $85 \times \$200 = \$17,000$
- Net benefit: \$12,425 per 100,000 transactions

**Decision:** Proceed to deployment (ROI positive).**Deliverable:** Evaluation report, deployment recommendation, list of remaining risks.**Phase 6: Deployment****Goal:** Put the model into production environment where it can provide business value.**Activities:****A. Deployment Strategies:**

- Batch prediction: Model runs on schedule (nightly/hourly) – used for recommendations, reporting
- Real-time API: Model responds instantly to requests – used for fraud detection, personalization
- Embedded model: Model integrated into application code
- Dashboard: Descriptive analytics using tools like Tableau or Power BI

**B. Production Requirements:**

- Model serialization (pickle, joblib, ONNX)
- Build prediction service (REST API using Flask/FastAPI)
- Containerization using Docker
- Orchestration using Kubernetes for scaling
- Monitoring:
  - Prediction volume
  - Latency
  - Error rates
  - Data drift
  - Concept drift

**C. Documentation:**

- Data dictionary (definition of all features)
- Model card (purpose, limitations, performance)
- API documentation (request/response format, endpoints)
- Maintenance plan (retraining schedule, ownership)

**D. Retraining Strategy:**

- Fixed schedule: monthly or quarterly retraining
- Trigger-based: retrain when performance drops below threshold
- Online learning: incremental updates with new data

**Deliverable:** Deployed production model, monitoring system, and full documentation.

4.2 OSEMN Model

OSEMN (pronounced "awesome") is a simpler, more data-science-focused workflow. It is ideal for individual projects, Kaggle competitions, and exploratory analysis in notebooks.

Step O: Obtain

**Goal:** Acquire data from its source into your analysis environment.

**Data sources and methods:**

| Source Type   | Method                      | Example                                      |
|---------------|-----------------------------|----------------------------------------------|
| Local file    | read_csv(), read_excel()    | pd.read_csv('data.csv')                      |
| Database      | SQL query via connection    | SELECT * FROM transactions                   |
| API           | HTTP requests (GET, POST)   | requests.get('https://api.example.com/data') |
| Web scraping  | BeautifulSoup, Scrapy       | Extract table from HTML page                 |
| Cloud storage | boto3, google-cloud-storage | Download from S3 bucket                      |
| Kaggle        | Kaggle API                  | kaggle competitions download -c titanic      |

**Important considerations:**

- Check license and terms of use
- Document data source and retrieval date
- Store raw data unchanged (preserve original)

**Output:** Raw dataset (may be messy, unprocessed).

Step S: Scrub

**Goal:** Clean and prepare data for analysis. This is the most time-consuming step.

**Detailed scrubbing activities:**

**1. Initial Assessment:**

```
1 df.shape           # (rows, columns)
2 df.info()          # data types and non-null counts
3 df.describe()      # summary statistics
4 df.isnull().sum()  # missing values per column
```

**2. Handle Missing Values:**

**Identify patterns:** Are missing values random or systematic?

**Numeric columns options:**

- Delete rows (if few missing, <5%)
- Impute with mean (symmetric distribution)
- Impute with median (skewed distribution or outliers)
- Impute with mode (most frequent value)
- Forward/backward fill (time series)
- Interpolation (time series with trend)

**Categorical columns options:**

- Delete rows
- Impute with mode
- Create "Unknown" or "Missing" category
- Advanced: KNN or regression-based imputation

**3. Handle Duplicates:**

- Exact duplicates: df.drop\_duplicates()
- Near duplicates: fuzzy matching or tolerance thresholds

**4. Fix Data Types:**

```
1 df['amount'] = df['amount'].astype(float)
2 df['date'] = pd.to_datetime(df['date'])
3 df['category'] = df['category'].astype('category')
```

**5. Handle Outliers:**

**Detection methods:**

- Z-score > 3 or < -3
- IQR method (outside 1.5 × IQR)
- Domain rules (age > 120, negative price)
- Visualization (box plots, scatter plots)

**Treatment options:**

- Remove erroneous data
- Winsorization (cap/floor values)
- Log transformation
- Use robust models (tree-based methods)

**6. Standardize Formats:**

- Text normalization (lowercase, strip whitespace, spelling fixes)
- Date formatting (YYYY-MM-DD)
- Consistent categories (e.g., NY, New York, new york → NY)

**Output:** Clean, consistent, analysis-ready dataset.

**Step E: Explore**

**Goal:** Understand the data through visualization and statistics. Generate hypotheses about patterns and relationships.

**Exploratory activities:**

**1. Univariate Analysis (single variable):**

| Variable Type | Visualizations                    | Statistics                                                           |
|---------------|-----------------------------------|----------------------------------------------------------------------|
| Numeric       | Histogram, Box plot, Density plot | Mean, Median, Mode, Std Dev, Skewness, Kurtosis, Min, Max, Quartiles |
| Categorical   | Bar chart, Pie chart              | Frequency count, Percentage, Mode                                    |

**Questions to answer:**

- What is the central tendency of the variable?
- How spread out are the values?
- Is the distribution symmetric or skewed?
- Are there any clusters or gaps?

**2. Bivariate Analysis (two variables):**

| Combination                | Visualizations                           | Statistics                                |
|----------------------------|------------------------------------------|-------------------------------------------|
| Numeric vs Numeric         | Scatter plot, Hexbin plot, Line plot     | Pearson correlation, Spearman correlation |
| Numeric vs Categorical     | Box plot, Violin plot, Bar chart (means) | ANOVA, t-test, Grouped stats              |
| Categorical vs Categorical | Stacked bar chart, Mosaic plot           | Chi-square test, Cramér's V               |

**Questions to answer:**

- Are variables correlated (positive, negative, none)?
- Is relationship linear or non-linear?
- Does category affect numeric variable?
- Are variables independent?

**3. Multivariate Analysis (3+ variables):**

- Correlation matrix (heatmap)
- Pairplots (scatter matrix)
- Parallel coordinates
- 3D scatter plots (limited usefulness)
- Facet grids / small multiples

**4. Missing Data Patterns:**

- Visualize missingness (e.g., missingno library)
- Check correlation of missing values across columns

**5. Hypothesis Generation:**

- Higher income customers have higher purchase amounts
- Fraud is more common for weekend transactions
- Churn risk increases after 3 months of inactivity

**Output:** Visualizations, statistical summaries, and hypotheses for modeling.

Step M: Model

**Goal:** Apply statistical or machine learning models to make predictions, find patterns, or test hypotheses.

**Modeling approaches by goal:**

| Goal                     | Approach              | Example Algorithms                                  |
|--------------------------|-----------------------|-----------------------------------------------------|
| Prediction (continuous)  | Regression            | Linear, Ridge, Lasso, Random Forest, XGBoost        |
| Prediction (category)    | Classification        | Logistic Regression, SVM, Random Forest, Neural Net |
| Pattern discovery        | Clustering            | K-Means, Hierarchical, DBSCAN                       |
| Dimensionality reduction | Feature extraction    | PCA, t-SNE                                          |
| Relationship testing     | Statistical inference | Hypothesis tests, Confidence intervals              |

**Basic modeling workflow:**

1. Prepare features and target

- $X$  = input features
- $y$  = target variable

2. Train-test split

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y, test_size=0.2, random_state=42
5 )
```

3. Train model

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier()
4 model.fit(X_train, y_train)
```

4. Predict

```
1 y_pred = model.predict(X_test)
2 y_pred_proba = model.predict_proba(X_test)[:, 1]
```

5. Evaluate

```
1 from sklearn.metrics import accuracy_score, roc_auc_score
2
3 print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
4 print(f"AUC: {roc_auc_score(y_test, y_pred_proba)}")
```

**Output:** Trained model with performance metrics.

Step I: Interpret

**Goal:** Translate model outputs into actionable insights for stakeholders.

**Interpretation activities:**

1. Model Explanation (non-technical audience):

- What does the model do? (one simple sentence)
- How well does it perform? (simple metrics, avoid jargon)
- What features are most important? (top 5 feature chart)
- When does the model fail? (limitations and edge cases)

2. Feature Importance:

- Identify most influential features
- Understand direction of impact (positive/negative)

3. Individual Prediction Explanation:

- SHAP (SHapley Additive exPlanations)
- LIME (Local Interpretable Model-agnostic Explanations)

Example explanation:

”This transaction was flagged as fraud because: amount is unusually high (\$950 vs typical \$45), location is new (first time in country X), and time is unusual (3 AM).”

4. Business Recommendations:

- Translate findings into actions
- Example: Customers inactive for 30 days have 60% churn risk. Send retention offer on day 25.

5. Visual Summary:

- Confusion matrix visualization
- ROC curve
- Feature importance bar chart
- Example predictions with explanations

**Output:** Clear report or presentation with actionable recommendations.

4.3 CRISP-DM vs OSEMN: Detailed Comparison

| Aspect              | CRISP-DM                               | OSEMN                           |
|---------------------|----------------------------------------|---------------------------------|
| Number of phases    | 6                                      | 5                               |
| Focus               | Full project lifecycle                 | Analytical workflow             |
| Business focus      | Strong (Phase 1 explicitly business)   | Weak (starts at obtaining data) |
| Deployment included | Yes (Phase 6)                          | No (stops at interpretation)    |
| Evaluation          | Separate phase                         | Part of interpretation          |
| Iteration           | Explicitly iterative                   | Implied iteration               |
| Best for            | Enterprise team projects               | Individual/Kaggle/notebook work |
| Documentation       | Heavy (requires project documentation) | Light                           |
| Time perspective    | Weeks to months                        | Hours to days                   |
| Origin              | Industry consortium (1996)             | Data science community (2000s)  |
| Learning curve      | Steeper                                | Gentler                         |

When to use CRISP-DM:

- Working in a team on a production project
- Need formal approval at each stage
- Business stakeholders require regular updates
- Long-term project with maintenance phase

When to use OSEMN:

- Exploring a new dataset independently
- Kaggle competition or personal project
- Quick proof-of-concept
- Teaching or learning data science

## Practice Questions

### SECTION A: 2-Mark Questions

1. Define Data Science.
2. What are the three core components of Data Science?
3. List the six phases of the CRISP-DM model.
4. List the five steps of the OSEMN model.
5. What is the difference between structured and unstructured data? Give one example of each.
6. Define overfitting and underfitting.
7. What is the typical percentage of time a Data Scientist spends on data cleaning?
8. What is Exploratory Data Analysis (EDA)?
9. What is the difference between a training set and a test set?
10. Define precision and recall.

### SECTION B: 4-Mark Questions

1. Explain the three core components of Data Science with suitable examples.
2. Describe any four real-world applications of Data Science in healthcare.
3. Describe any four real-world applications of Data Science in IoT.
4. Describe any four real-world applications of Data Science in finance.
5. Differentiate between a Data Analyst, Data Scientist, and ML Engineer (any four points).
6. Compare CRISP-DM and OSEMN models on any four parameters.
7. Explain the difference between overfitting and underfitting. How can each be detected and prevented?
8. Explain any four activities performed during the Data Preparation phase of CRISP-DM.
9. What is feature engineering? Explain with two examples.
10. Explain the confusion matrix for binary classification with a diagram. Define TP, FP, TN, FN.

### SECTION C: 8-Mark Questions

1. Explain the CRISP-DM methodology in detail. Discuss all six phases with suitable examples from a real-world data science project.
2. Explain the OSEMN model in detail. Discuss all five steps with suitable examples from a real-world data science project.
3. Compare and contrast the CRISP-DM and OSEMN workflows in detail. Under what circumstances would you prefer one over the other?
4. Discuss the various roles in a data science team (Data Analyst, Data Scientist, ML Engineer, Data Engineer, Domain Expert). Explain their responsibilities, required skills, and how they collaborate.
5. With suitable examples from at least three different domains (healthcare, finance, IoT, engineering, retail), explain how data science is transforming real-world decision-making.
6. Explain the complete modeling workflow in data science including: algorithm selection, train-validation-test split, training, hyperparameter tuning, and evaluation metrics (accuracy, precision, recall, F1, ROC-AUC).
7. Explain the Data Preparation phase of CRISP-DM in detail. Include data cleaning (missing values, outliers, duplicates), data transformation (scaling, encoding), and feature engineering with examples.
8. Explain the Evaluation and Deployment phases of CRISP-DM in detail. Include evaluation metrics, cost-benefit analysis, deployment strategies (batch, real-time API), monitoring, and retraining.
9. You are a Data Scientist working on a credit card fraud detection project. Walk through the entire CRISP-DM workflow from Business Understanding to Deployment with specific examples at each phase.
10. Compare and contrast the four types of data by measurement scale: nominal, ordinal, interval, and ratio. Give two examples of each and explain why the distinction matters in data science.



# Chapter 2

## Chapter-2 Data Collection and Preprocessing

### What is Data

Data refers to raw facts, figures, or observations that are collected for analysis or reference.

It can exist in many forms such as numbers, text, images, audio, video, or symbols. On its own, data has no meaning until it is processed and interpreted.

For example, the numbers 85, 90, 78 are data. When organized as “student marks in a test,” they become meaningful information.

**In simple terms:**

- Data = raw input
- Information = processed, meaningful data
- Knowledge = insights derived from information

#### Data Types (Classification by Structure)

| Data Type            | Description                                                | Examples                                                  | Advantages / Challenges                                                                                                           |
|----------------------|------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Structured Data      | Organized in predefined rows and columns with fixed schema | CSV files, Excel spreadsheets, SQL tables                 | Easy to query, store, and analyze; efficient storage / Limited flexibility; schema changes are difficult                          |
| Semi-structured Data | Has tags/markers to separate elements but no rigid schema  | JSON, XML, HTML, YAML                                     | Flexible schema; self-describing; hierarchical representation / More complex to parse than structured; increased storage overhead |
| Unstructured Data    | No predefined format or organization                       | Text documents, images, videos, audio, social media posts | Rich information content; human-readable formats / Difficult to analyze; requires NLP/CV techniques; large storage needed         |

#### Data Types (Classification by Measurement Scale)

| Scale Type | Description                                            | Mathematical Operations                               | Examples                                                               |
|------------|--------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------|
| Nominal    | Categories with no order or ranking                    | Mode, frequency count                                 | Gender (Male/Female), colors, yes/no, zip codes                        |
| Ordinal    | Categories with meaningful order but unequal intervals | Mode, median, percentile                              | Education (High School < Bachelor < Master < PhD), ratings (1–5 stars) |
| Interval   | Numeric with equal intervals but no true zero          | +, -, mean, standard deviation                        | Temperature (Celsius/Fahrenheit), dates, IQ scores                     |
| Ratio      | Numeric with equal intervals AND a true zero           | +, -, ×, $\nabla\bullet$ , all statistical operations | Height, weight, age, income, price, counts                             |

Data Sources and Collection Methods

| Source Type   | Method                                                                                        | Use Case                                  | Example                                                  |
|---------------|-----------------------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------------------|
| Local Files   | <code>pd.read_csv()</code> ,<br><code>pd.read_excel()</code> ,<br><code>pd.read_json()</code> | Small to medium datasets on your computer | <code>sales_data.csv</code> , <code>customer.xlsx</code> |
| Databases     | SQL queries (SELECT, JOIN, WHERE)                                                             | Large structured data requiring querying  | PostgreSQL, MySQL, Oracle, SQL Server                    |
| APIs (REST)   | HTTP requests (GET, POST, PUT, DELETE)                                                        | Real-time or third-party data             | Twitter API, Weather API, Google Maps API                |
| Web Scraping  | BeautifulSoup, Selenium, Scrapy                                                               | Data not available through APIs           | Extracting product prices from e-commerce sites          |
| Cloud Storage | boto3 (AWS S3), google-cloud-storage                                                          | Scalable storage of large datasets        | Data lakes, backup storage                               |
| IoT Sensors   | MQTT, CoAP protocols                                                                          | Real-time streaming data                  | Temperature sensors, fitness trackers                    |

Working with Different Data Types in Data Science

1. What is a CSV File?

CSV (Comma-Separated Values) is a simple file format used to store tabular data, such as a spreadsheet or database table.

1.1 Structure of a CSV File

- Each line in the file represents a row of data
- Values within a row are separated by delimiters (usually commas)
- The first row often contains column headers

1.2 Example of a CSV File

File: `customers.csv` (raw format)

```
1 customer_id,name,age,city,amount
2 1,Alice,25,New York,250.50
3 2,Bob,30,Los Angeles,180.00
4 3,Charlie,35,Chicago,320.75
5 4,Diana,28,Houston,95.25
6 5,Eve,42,Phoenix,510.00
```

Same data displayed as a table:

| customer_id | name    | age | city        | amount |
|-------------|---------|-----|-------------|--------|
| 1           | Alice   | 25  | New York    | 250.50 |
| 2           | Bob     | 30  | Los Angeles | 180.00 |
| 3           | Charlie | 35  | Chicago     | 320.75 |
| 4           | Diana   | 28  | Houston     | 95.25  |
| 5           | Eve     | 42  | Phoenix     | 510.00 |

1.3 Features of CSV Files

| Feature                 | Description                                                         |
|-------------------------|---------------------------------------------------------------------|
| Human-readable          | Can be opened in any text editor                                    |
| Lightweight             | Smaller file size compared to Excel                                 |
| Universal compatibility | Supported by Excel, Google Sheets, databases, programming languages |
| No formatting           | Stores only data, not formulas or styling                           |
| Simple structure        | Easy to parse and generate                                          |

1.4 CSV Delimiters (Alternatives to Comma)

| Delimiter     | File Extension | Region / Use                            |
|---------------|----------------|-----------------------------------------|
| Comma (,)     | .csv           | Most common, English regions            |
| Semicolon (;) | .csv           | European regions (Excel uses semicolon) |
| Tab (\t)      | .tsv or .txt   | When data contains commas               |
| Pipe (—)      | .csv           | When data contains commas and tabs      |
| Space ( )     | .txt           | Simple data without spaces              |

2. Method 1: Using Python’s Built-in CSV Module

The `csv` module is Python’s built-in library for reading and writing CSV files. It does not require any installation.

2.1 Importing the CSV Module

```
1 import csv
```

## 2.2 CSV Reader – Reading CSV Files

### 2.2.1 Basic CSV Reader

```
1 import csv
2
3 # Open and read CSV file
4 with open('customers.csv', 'r') as file:
5     csv_reader = csv.reader(file)
6
7     # Read header row
8     header = next(csv_reader)
9     print(f"Headers: {header}")
10
11    # Read all data rows
12    print("\nData rows:")
13    for row in csv_reader:
14        print(row)
```

Output:

```
1 Headers: ['customer_id', 'name', 'age', 'city', 'amount']
2
3 Data rows:
4 ['1', 'Alice', '25', 'New York', '250.50']
5 ['2', 'Bob', '30', 'Los Angeles', '180.00']
6 ['3', 'Charlie', '35', 'Chicago', '320.75']
7 ['4', 'Diana', '28', 'Houston', '95.25']
8 ['5', 'Eve', '42', 'Phoenix', '510.00']
```

### 2.2.2 CSV Reader with Different Delimiter

```
1 import csv
2
3 # Reading a semicolon-separated file
4 with open('customers_semicolon.csv', 'r') as file:
5     csv_reader = csv.reader(file, delimiter=';')
6
7     for row in csv_reader:
8         print(row)
```

### 2.2.3 CSV DictReader (Read as Dictionaries)

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.DictReader(file)
5
6     for row in csv_reader:
7         print(f"Name: {row['name']}, City: {row['city']}, Amount: {row['amount']}")
```

Output:

```
1 Name: Alice, City: New York, Amount: 250.50
2 Name: Bob, City: Los Angeles, Amount: 180.00
3 Name: Charlie, City: Chicago, Amount: 320.75
4 Name: Diana, City: Houston, Amount: 95.25
5 Name: Eve, City: Phoenix, Amount: 510.00
```

### 2.2.4 Reading CSV into a List

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.reader(file)
5     data = list(csv_reader)
6
7 print(f"Total rows: {len(data)}")
8 print(f"First row (headers): {data[0]}")
9 print(f"Second row (first data row): {data[1]}")
```

### 2.2.5 Reading Specific Columns

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.DictReader(file)
5
6     for row in csv_reader:
7         name = row['name']
8         amount = row['amount']
9         print(f"{name}: ${amount}")
```

## 2.3 CSV Writer – Writing CSV Files

### 2.3.1 Basic CSV Writer

```
1 import csv
2
3 # Data to write
```

```

4 data = [
5     ['customer_id', 'name', 'age', 'city', 'amount'],
6     [1, 'Alice', 25, 'New York', 250.50],
7     [2, 'Bob', 30, 'Los Angeles', 180.00],
8     [3, 'Charlie', 35, 'Chicago', 320.75]
9 ]
10
11 # Write to CSV file
12 with open('output_csv_writer.csv', 'w', newline='') as file:
13     csv_writer = csv.writer(file)
14
15     csv_writer.writerows(data)
16
17 print("CSV file created successfully!")

```

### 2.3.2 Writing Row by Row

```

1 import csv
2
3 with open('output_row_by_row.csv', 'w', newline='') as file:
4     csv_writer = csv.writer(file)
5
6     csv_writer.writerow(['customer_id', 'name', 'age', 'city', 'amount'])
7
8     csv_writer.writerow([1, 'Alice', 25, 'New York', 250.50])
9     csv_writer.writerow([2, 'Bob', 30, 'Los Angeles', 180.00])
10    csv_writer.writerow([3, 'Charlie', 35, 'Chicago', 320.75])
11
12 print("Rows written successfully!")

```

### 2.3.3 CSV DictWriter (Write from Dictionaries)

```

1 import csv
2
3 data = [
4     {'customer_id': 1, 'name': 'Alice', 'age': 25, 'city': 'New York', 'amount': 250.50},
5     {'customer_id': 2, 'name': 'Bob', 'age': 30, 'city': 'Los Angeles', 'amount': 180.00},
6     {'customer_id': 3, 'name': 'Charlie', 'age': 35, 'city': 'Chicago', 'amount': 320.75}
7 ]
8
9 fieldnames = ['customer_id', 'name', 'age', 'city', 'amount']
10
11 with open('output_dictwriter.csv', 'w', newline='') as file:
12     writer = csv.DictWriter(file, fieldnames=fieldnames)
13
14     writer.writeheader()
15     writer.writerows(data)
16
17 print("DictWriter CSV created successfully!")

```

### 2.3.4 Writing with Different Delimiter

```

1 import csv
2
3 data = [
4     ['Name', 'City', 'Amount'],
5     ['Alice', 'New York', 250.50],
6     ['Bob', 'Los Angeles', 180.00]
7 ]
8
9 # Semicolon delimiter
10 with open('output_semicolon.csv', 'w', newline='') as file:
11     csv_writer = csv.writer(file, delimiter=';')
12     csv_writer.writerows(data)
13
14 # Tab delimiter
15 with open('output_tab.tsv', 'w', newline='') as file:
16     csv_writer = csv.writer(file, delimiter='\t')
17     csv_writer.writerows(data)
18
19 print("Files with different delimiters created!")

```

### 2.3.5 Appending to Existing CSV File

```

1 import csv
2
3 new_rows = [
4     [6, 'Frank', 38, 'San Antonio', 200.00],
5     [7, 'Grace', 27, 'San Diego', 450.00]
6 ]
7
8 with open('customers.csv', 'a', newline='') as file:
9     csv_writer = csv.writer(file)
10    csv_writer.writerows(new_rows)
11
12 print("Data appended successfully!")

```

### 3. Method 2: Using Pandas for CSV Operations

Pandas is more powerful and recommended for data analysis. It requires installation (`pip install pandas`).

#### 3.1 Reading CSV Files with Pandas

```
1 import pandas as pd
2
3 # Basic read
4 df = pd.read_csv('customers.csv')
5 print(df)
```

Output:

|   | customer_id | name    | age | city        | amount |
|---|-------------|---------|-----|-------------|--------|
| 0 | 1           | Alice   | 25  | New York    | 250.50 |
| 1 | 2           | Bob     | 30  | Los Angeles | 180.00 |
| 2 | 3           | Charlie | 35  | Chicago     | 320.75 |
| 3 | 4           | Diana   | 28  | Houston     | 95.25  |
| 4 | 5           | Eve     | 42  | Phoenix     | 510.00 |

#### 3.2 Common Parameters of `pd.read_csv()`

```
1 df = pd.read_csv(
2     'customers.csv',           # File path
3     sep=',',                  # Delimiter (default is comma)
4     header=0,                  # Row number for column names
5     index_col='customer_id',   # Use column as index
6     usecols=['name', 'age', 'city'], # Select specific columns
7     nrows=3,                   # Read only first 3 rows
8     skiprows=[1, 3],           # Skip rows
9     na_values=['', 'NA', 'NULL'], # Missing values
10    dtype={'age': int, 'amount': float}, # Data types
11    encoding='utf-8'            # File encoding
12 )
13
14 print(df)
```

#### 3.3 Reading CSV Without Headers

```
1 df = pd.read_csv(
2     'customers_no_header.csv',
3     names=['id', 'name', 'age', 'city', 'amount']
4 )
5
6 print(df)
```

#### 3.4 Reading Large CSV Files in Chunks

```
1 chunk_size = 1000
2
3 for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
4     print(f"Processing chunk with {len(chunk)} rows")
```

#### 3.5 Writing CSV Files with Pandas

```
1 import pandas as pd
2
3 data = {
4     'customer_id': [1, 2, 3],
5     'name': ['Alice', 'Bob', 'Charlie'],
6     'age': [25, 30, 35],
7     'city': ['New York', 'Los Angeles', 'Chicago'],
8     'amount': [250.50, 180.00, 320.75]
9 }
10
11 df = pd.DataFrame(data)
12
13 df.to_csv('output_pandas.csv', index=False)
14 print("CSV file saved!")
```

#### 3.6 Common Parameters of `df.to_csv()`

```
1 df.to_csv(
2     'output_customers.csv',
3     sep=',',
4     index=False,
5     header=True,
6     columns=['name', 'age', 'city'],
7     encoding='utf-8',
```

```
na_rep='NULL',
float_format='%.2f'
)
```

3.7 Appending with Pandas

```
new_data = pd.DataFrame({
    'customer_id': [6, 7],
    'name': ['Frank', 'Grace'],
    'age': [38, 27],
    'city': ['San Antonio', 'San Diego'],
    'amount': [200.00, 450.00]
})

new_data.to_csv(
    'customers.csv',
    mode='a',
    header=False,
    index=False
)
```

4. Complete Code Examples (CSV Module + Pandas Together)

Example 1: CSV Reader – Read and Process

```
import csv

print("=" * 50)
print("USING CSV MODULE - READER")
print("=" * 50)

with open('customers.csv', 'r') as file:
    csv_reader = csv.reader(file)
    headers = next(csv_reader)
    print(f"Headers: {headers}")

    customers = []
    for row in csv_reader:
        customers.append({
            'id': int(row[0]),
            'name': row[1],
            'age': int(row[2]),
            'city': row[3],
            'amount': float(row[4])
        })

    print(f"\nTotal customers: {len(customers)}")
    print("First customer:", customers[0])
```

Example 2: CSV DictReader – Read as Dictionary

```
import csv

print("=" * 50)
print("USING CSV MODULE - DICTREADER")
print("=" * 50)

with open('customers.csv', 'r') as file:
    csv_reader = csv.DictReader(file)

    customers = list(csv_reader)

    for customer in customers[:3]:
        print(f"Name: {customer['name']}, City: {customer['city']}, Amount: ${customer['amount']}")
```

Example 3: CSV Writer – Write Data

```
import csv

print("=" * 50)
print("USING CSV MODULE - WRITER")
print("=" * 50)

data = [
    ['customer_id', 'name', 'age', 'city', 'amount'],
    [1, 'Alice', 25, 'New York', 250.50],
    [2, 'Bob', 30, 'Los Angeles', 180.00],
    [3, 'Charlie', 35, 'Chicago', 320.75],
    [4, 'Diana', 28, 'Houston', 95.25],
    [5, 'Eve', 42, 'Phoenix', 510.00]
]
```

```
15
16 with open('new_customers.csv', 'w', newline='') as file:
17     csv_writer = csv.writer(file)
18     csv_writer.writerows(data)
19
20 print("File created successfully!")
```

Example 4: CSV DictWriter – Write from Dictionaries

```
1 import csv
2
3 print("=" * 50)
4 print("USING CSV MODULE - DICTWRITER")
5 print("=" * 50)
6
7 customers = [
8     {'id': 1, 'name': 'Alice', 'amount': 250.50},
9     {'id': 2, 'name': 'Bob', 'amount': 180.00},
10    {'id': 3, 'name': 'Charlie', 'amount': 320.75}
11 ]
12
13 fieldnames = ['id', 'name', 'amount']
14
15 with open('dictwriter_output.csv', 'w', newline='') as file:
16     writer = csv.DictWriter(file, fieldnames=fieldnames)
17     writer.writeheader()
18     writer.writerows(customers)
19
20 print("DictWriter output saved!")
```

Example 5: Pandas CSV Operations

```
1 import pandas as pd
2
3 print("=" * 50)
4 print("USING PANDAS FOR CSV")
5 print("=" * 50)
6
7 df = pd.read_csv('customers.csv')
8 print(df)
9
10 df['discount'] = df['amount'] * 0.10
11 df['final_amount'] = df['amount'] - df['discount']
12
13 high_value = df[df['amount'] > 200]
14 print(high_value[['name', 'amount', 'final_amount']])
15
16 high_value.to_csv('high_value_customers.csv', index=False)
17 print("Saved successfully!")
```

Example 6: Complete Mixed Example (Read → Process → Write)

```
1 import csv
2 import pandas as pd
3
4 print("=" * 60)
5 print("COMPLETE EXAMPLE: READ PROCESS WRITE")
6 print("=" * 60)
7
8 print("\n[1] Reading CSV using csv.reader...")
9 with open('customers.csv', 'r') as file:
10     reader = csv.reader(file)
11     headers = next(reader)
12     rows = list(reader)
13
14 print(f"Read {len(rows)} rows")
15
16 print("\n[2] Processing data...")
17 processed_data = []
18 for row in rows:
19     customer = {
20         'customer_id': int(row[0]),
21         'name': row[1],
22         'age': int(row[2]),
23         'city': row[3],
24         'amount': float(row[4])
25     }
26     customer['discount'] = customer['amount'] * 0.10
27     customer['final_amount'] = customer['amount'] - customer['discount']
28     processed_data.append(customer)
29
30 print(f"Processed {len(processed_data)} records")
```

```
31
32 print("\n[3] Writing processed data...")
33 with open('processed_customers.csv', 'w', newline='') as file:
34     fieldnames = ['customer_id', 'name', 'age', 'city', 'amount', 'discount', 'final_amount']
35     writer = csv.DictWriter(file, fieldnames=fieldnames)
36     writer.writeheader()
37     writer.writerows(processed_data)
38
39 print("Saved successfully!")
40
41 print("\n[4] Verifying with pandas...")
42 df = pd.read_csv('processed_customers.csv')
43 print(df.head())
44 print(f"Shape: {df.shape}")
```

5. Comparison: CSV Module vs Pandas

| Feature        | csv module (built-in)             | pandas                                   |
|----------------|-----------------------------------|------------------------------------------|
| Installation   | No installation needed            | Requires <code>pip install pandas</code> |
| Speed          | Slower for large files            | Faster (optimized in C)                  |
| Memory usage   | Low (line-by-line processing)     | High (loads full dataset)                |
| Data types     | All values as strings             | Automatic type inference                 |
| Missing values | Manual handling required          | Built-in NaN support                     |
| Filtering      | Manual loops                      | Simple indexing                          |
| Aggregations   | Manual coding                     | Built-in functions (groupby, mean, sum)  |
| Writing files  | csv.writer                        | df.to_csv()                              |
| Best for       | Simple I/O, large streaming files | Data analysis, ML, transformations       |

Working with APIs for Data Collection

1. What is an API?

API (Application Programming Interface) is a set of rules and protocols that allows one software application to communicate with another. It acts as a bridge between different software systems, enabling them to exchange data and functionality.

1.1 Simple Analogy

| Real-World Analogy | API Analogy                   |
|--------------------|-------------------------------|
| Restaurant         | API                           |
| You (customer)     | Your application (client)     |
| Waiter             | API                           |
| Kitchen            | Server/database               |
| Menu               | API documentation (endpoints) |
| Food               | Data/response                 |

How it works:

- You look at the menu (API documentation)
- You send a request through the waiter (API)
- Waiter communicates with the kitchen (server)
- Kitchen processes the request
- Waiter returns the response (food/data)

1.2 Why APIs are Important in Data Science

| Use Case                   | Example                                        |
|----------------------------|------------------------------------------------|
| Collect real-time data     | Live stock prices, weather data, sports scores |
| Access third-party data    | Twitter feeds, Google Maps, YouTube analytics  |
| Send predictions           | Deploy ML model as API for other apps          |
| Automate data collection   | Scheduled API calls for regular updates        |
| Integrate multiple sources | Combine CRM, payment, and support systems      |

1.3 Key API Terminology

| Term           | Definition                 | Example                            |
|----------------|----------------------------|------------------------------------|
| Endpoint       | Specific URL to access API | https://api.weather.com/v1/current |
| Request        | Message sent to server     | GET /users/123                     |
| Response       | Data returned by server    | {"name": "Alice", "age": 25}       |
| Method/Verb    | Action performed           | GET, POST, PUT, DELETE             |
| Header         | Metadata for request       | API key, content type              |
| Body           | Data sent with request     | JSON payload                       |
| Status Code    | Server response status     | 200, 404, 500                      |
| Authentication | User verification          | API Key, OAuth Token               |

## 2. HTTP Methods (Request Verbs)

| Method | Purpose               | Analogy                | Example                   |
|--------|-----------------------|------------------------|---------------------------|
| GET    | Retrieve data         | Asking for information | Get customer details      |
| POST   | Create new data       | Submitting a form      | Create new customer       |
| PUT    | Update entire record  | Replacing a file       | Update full customer info |
| PATCH  | Update partial record | Editing a field        | Update phone number       |
| DELETE | Remove data           | Deleting a file        | Delete customer           |

### 2.1 HTTP Status Codes

| Range | Category     | Common Codes                                       |
|-------|--------------|----------------------------------------------------|
| 2xx   | Success      | 200 OK, 201 Created                                |
| 3xx   | Redirection  | 301 Moved Permanently                              |
| 4xx   | Client Error | 400 Bad Request, 401 Unauthorized, 404 Not Found   |
| 5xx   | Server Error | 500 Internal Server Error, 503 Service Unavailable |

## 3. REST API (Representational State Transfer)

### 3.1 What is REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. REST APIs follow constraints that make them simple, scalable, and lightweight.

### 3.2 Principles of REST API

| Principle         | Explanation                                                                   |
|-------------------|-------------------------------------------------------------------------------|
| Stateless         | Each request contains all required information; server does not store session |
| Client-Server     | Client and server operate independently                                       |
| Cacheable         | Responses can be cached for performance                                       |
| Uniform Interface | Standardized communication using HTTP methods                                 |
| Layered System    | Client may interact through intermediaries                                    |

### 3.3 REST API Characteristics

| Characteristic | Description                          |
|----------------|--------------------------------------|
| Uses HTTP      | Built on standard HTTP protocol      |
| Data Format    | Usually JSON (lightweight, readable) |
| Stateless      | No memory of previous requests       |
| Simple         | Easy to implement and understand     |
| Scalable       | Handles large number of requests     |

### 3.4 REST API Example

Endpoint: `https://jsonplaceholder.typicode.com/users`

Request:

```
1 GET https://jsonplaceholder.typicode.com/users
```

Response (JSON):

```
1 [
2   {
3     "id": 1,
4     "name": "Leanne Graham",
5     "username": "Bret",
6     "email": "Sincere@april.biz",
7     "address": {
8       "street": "Kulas Light",
9       "city": "Gwenborough"
10    }
11  }
12 ]
```

### 3.5 REST API Request Components

```
1 GET https://api.example.com/users/123?include=orders&limit=10 HTTP/1.1
2 Host: api.example.com
3 Authorization: Bearer xyz123token
4 Content-Type: application/json
```

| Component        | Value                          | Purpose                   |
|------------------|--------------------------------|---------------------------|
| Method           | GET                            | Action to perform         |
| Base URL         | https://api.example.com        | Root API address          |
| Endpoint         | /users/123                     | Specific resource         |
| Query Parameters | ?include=orders&limit=10       | Filter response           |
| Headers          | Authorization,<br>Content-Type | Metadata                  |
| Body             | None (GET request)             | Data sent (POST/PUT only) |

## 4. SOAP API (Simple Object Access Protocol)

### 4.1 What is SOAP?

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in web services. It is more rigid and formal than REST.

### 4.2 SOAP Characteristics

| Characteristic     | Description                         |
|--------------------|-------------------------------------|
| Protocol           | Strict protocol with formal rules   |
| Data Format        | XML only (heavier than JSON)        |
| Transport          | HTTP, SMTP, TCP, etc.               |
| Stateful/Stateless | Can be stateful (remembers session) |
| Error Handling     | Standardized fault messages         |
| Security           | Built-in WS-Security                |
| Complexity         | More complex to implement           |

### 4.3 SOAP Message Structure

```
1 <?xml version="1.0"?>
2 <soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
3
4   <soap:Header>
5     <auth:Authentication xmlns:auth="http://example.com/auth">
6       <auth:apiKey>xyz123</auth:apiKey>
7     </auth:Authentication>
8   </soap:Header>
9
10  <soap:Body>
11    <GetUserInfo xmlns="http://example.com/user">
12      <UserID>12345</UserID>
13    </GetUserInfo>
14  </soap:Body>
15
16 </soap:Envelope>
```

| Element  | Purpose                                     |
|----------|---------------------------------------------|
| Envelope | Root element identifying SOAP message       |
| Header   | Optional metadata (authentication, routing) |
| Body     | Contains actual request or response         |
| Fault    | Error information inside Body               |

### 4.4 SOAP Example Response

```
1 <?xml version="1.0"?>
2 <soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
3
4   <soap:Body>
5     <GetUserInfoResponse xmlns="http://example.com/user">
6       <UserInfo>
7         <ID>12345</ID>
8         <Name>John Doe</Name>
9         <Email>john@example.com</Email>
10      </UserInfo>
11    </GetUserInfoResponse>
12  </soap:Body>
13
14 </soap:Envelope>
```

## 5. REST API vs SOAP API (Comparison)

| Feature         | REST API                         | SOAP API                              |
|-----------------|----------------------------------|---------------------------------------|
| Full Form       | Representational State Transfer  | Simple Object Access Protocol         |
| Type            | Architectural style              | Protocol                              |
| Data Format     | JSON, XML, HTML, Plain text      | XML only                              |
| Message Size    | Small (lightweight)              | Large (verbose XML)                   |
| Speed           | Faster                           | Slower                                |
| State           | Stateless (always)               | Can be stateful or stateless          |
| Caching         | Supported                        | Not supported                         |
| Security        | HTTPS, API Keys, OAuth           | WS-Security (built-in)                |
| Error Handling  | HTTP status codes                | SOAP Faults                           |
| Complexity      | Simple, easy to learn            | Complex, steep learning curve         |
| Human Readable  | Yes (JSON readable)              | No (XML verbose)                      |
| Browser Support | Excellent                        | Limited                               |
| Best For        | Web and mobile apps, public APIs | Enterprise systems, banking, payments |
| Examples        | Google Maps API, Twitter API     | PayPal SOAP API, banking systems      |

### 5.1 When to Use REST API?

- Web and mobile application development
- Public APIs for external developers
- Fast performance with lightweight communication
- JSON-based data exchange
- Stateless operations

### 5.2 When to Use SOAP API?

- Enterprise-level systems (banking, healthcare)
- High security and transaction requirements
- Formal contracts using WSDL
- Stateful operations required
- Integration with legacy systems

## 6. Working with REST APIs in Python

### 6.1 Installing Required Library

```
1 pip install requests
```

### 6.2 Importing Requests Module

```
1 import requests
```

### 6.3 GET Request (Retrieve Data)

#### Basic GET Request

```
1 import requests
2
3 response = requests.get('https://jsonplaceholder.typicode.com/users')
4
5 print(f"Status Code: {response.status_code}")
6 print(f"Response Type: {type(response.json())}")
7
8 users = response.json()
9 print(f"Number of users: {len(users)}")
10 print(f"First user: {users[0]['name']}")
```

GET Request with Query Parameters

```
1 import requests
2
3 params = {
4     'postId': 1,
5     '_limit': 3
6 }
7
8 response = requests.get(
9     'https://jsonplaceholder.typicode.com/comments',
10    params=params
11 )
12
13 print(f"URL that was called: {response.url}")
14 comments = response.json()
15
16 for comment in comments:
17     print(f"Comment {comment['id']}: {comment['name'][:50]}...")
```

GET Request with Headers (Authentication)

```
1 import requests
2
3 headers = {
4     'Authorization': 'Bearer YOUR_API_TOKEN',
5     'Content-Type': 'application/json',
6     'User-Agent': 'MyDataScienceApp/1.0'
7 }
8
9 response = requests.get(
10    'https://api.example.com/data',
11    headers=headers
12 )
13
14 if response.status_code == 200:
15     data = response.json()
16     print("Data retrieved successfully")
17 else:
18     print(f"Error: {response.status_code}")
```

6.4 POST Request (Create Data)

```
1 import requests
2
3 new_user = {
4     'name': 'Alice Johnson',
5     'username': 'alicej',
6     'email': 'alice@example.com',
7     'phone': '555-1234'
8 }
9
10 response = requests.post(
11    'https://jsonplaceholder.typicode.com/users',
12    json=new_user
13 )
14
15 print(f"Status Code: {response.status_code}")
16 print(f"Response: {response.json()}")
```

6.5 PUT Request (Update Entire Record)

```
1 import requests
2
3 updated_user = {
4     'id': 1,
5     'name': 'Leanne Graham Updated',
6     'username': 'Bret_UPDATED',
7     'email': 'updated@example.com'
8 }
9
10 response = requests.put(
11    'https://jsonplaceholder.typicode.com/users/1',
12    json=updated_user
13 )
14
15 print(f"Status Code: {response.status_code}")
16 print(f"Updated User: {response.json()}")
```

### 6.6 DELETE Request (Remove Data)

```
1 import requests
2
3 response = requests.delete(
4     'https://jsonplaceholder.typicode.com/users/1'
5 )
6
7 print(f"Status Code: {response.status_code}")
8
9 if response.status_code == 200:
10     print("User deleted successfully")
11 else:
12     print("Deletion failed")
```

### 6.7 Error Handling with APIs

```
1 import requests
2
3 url = "https://jsonplaceholder.typicode.com/users/999"
4
5 try:
6     response = requests.get(url, timeout=5)
7     response.raise_for_status()
8
9     data = response.json()
10    print("Data:", data)
11
12 except requests.exceptions.HTTPError as e:
13     print(f"HTTP Error: {e}")
14     print(f"Status Code: {response.status_code}")
15
16 except requests.exceptions.ConnectionError:
17     print("Failed to connect to server.")
18
19 except requests.exceptions.Timeout:
20     print("Request timed out.")
21
22 except requests.exceptions.RequestException as e:
23     print(f"An error occurred: {e}")
```

## 7. Real-World API Examples for Data Science

### Example 1: Weather API (OpenWeatherMap)

```
1 import requests
2
3 API_KEY = "your_api_key_here"
4 CITY = "London"
5
6 url = f"http://api.openweathermap.org/data/2.5/weather?q={CITY}&appid={API_KEY}&units=metric"
7
8 response = requests.get(url)
9
10 if response.status_code == 200:
11     weather_data = response.json()
12
13     print(f"City: {weather_data['name']}")
14     print(f"Temperature: {weather_data['main']['temp']}C")
15     print(f"Humidity: {weather_data['main']['humidity']}%")
16     print(f"Conditions: {weather_data['weather'][0]['description']}")
17 else:
18     print(f"Error: {response.status_code}")
```

### Example 2: News API (Fetch Latest Headlines)

```
1 import requests
2
3 API_KEY = "your_api_key_here"
4 url = f"https://newsapi.org/v2/top-headlines?country=us&apiKey={API_KEY}"
5
6 response = requests.get(url)
7
8 if response.status_code == 200:
9     news = response.json()
10
11     print(f"Total articles: {news['totalResults']}")
12     print("\nTop 5 Headlines:")
13
14     for i, article in enumerate(news['articles'][:5], 1):
```

```
15     print(f"{i}. {article['title']}")
16     print(f"    Source: {article['source']['name']}\n")
```

### Example 3: GitHub API (Fetch User Repositories)

```
1 import requests
2
3 username = "octocat"
4 url = f"https://api.github.com/users/{username}/repos"
5
6 response = requests.get(url)
7
8 if response.status_code == 200:
9     repos = response.json()
10
11     print(f"Repositories for {username}:")
12     for repo in repos[:5]:
13         print(f"    - {repo['name']}: {repo['stargazers_count']} stars")
14 else:
15     print(f"Error: {response.status_code}")
```

## 8. Working with SOAP APIs in Python

SOAP APIs are less common but still used in enterprise environments.

### 8.1 Installing Required Library

```
1 pip install zeep
```

### 8.2 Basic SOAP Request

```
1 from zeep import Client
2
3 wsdl_url = "http://www.dneonline.com/calculator.asmx?wsdl"
4
5 client = Client(wsdl_url)
6
7 result = client.service.Add(intA=10, intB=20)
8
9 print(f"10 + 20 = {result}")
```

### 8.3 SOAP with Headers and Authentication

```
1 from zeep import Client
2 from zeep.wsse.username import UsernameToken
3
4 client = Client(
5     'https://example.com/service.asmx?wsdl',
6     wsse=UsernameToken('username', 'password')
7 )
8
9 response = client.service.GetUserDetails(UserID=12345)
10 print(response)
```

## 9. Complete API Data Collection Pipeline

```
1 import requests
2 import pandas as pd
3 import time
4
5 print("=" * 60)
6 print("COMPLETE API DATA COLLECTION PIPELINE")
7 print("=" * 60)
8
9 API_URL = "https://jsonplaceholder.typicode.com/posts"
10 params = {
11     '_limit': 20
12 }
13
14 print("\n[1] Fetching data from API...")
15 try:
16     response = requests.get(API_URL, params=params, timeout=10)
17     response.raise_for_status()
18     print(f"    Status: {response.status_code} - Success")
19 except Exception as e:
20     print(f"    Error: {e}")
21 exit()
```

```
22
23 print("\n[2] Parsing JSON response...")
24 data = response.json()
25 print(f"    Retrieved {len(data)} records")
26
27 print("\n[3] Converting to pandas DataFrame...")
28 df = pd.DataFrame(data)
29 print(f"    Shape: {df.shape}")
30 print(f"    Columns: {list(df.columns)}")
31
32 print("\n[4] Processing data...")
33
34 df['title_length'] = df['title'].str.len()
35 df['body_word_count'] = df['body'].str.split().str.len()
36
37 print("    Added features:")
38 print("        - title_length")
39 print("        - body_word_count")
40
41 print("\n[5] Saving to CSV...")
42 csv_filename = "api_posts_data.csv"
43 df.to_csv(csv_filename, index=False)
44 print(f"    Saved to '{csv_filename}'")
45
46 print("\n[6] Data Preview:")
47 print(df[['id', 'title', 'title_length', 'body_word_count']].head())
48
49 print("\n" + "=" * 60)
50 print("API DATA COLLECTION COMPLETED SUCCESSFULLY!")
51 print("=" * 60)
```

## 10. Best Practices for Working with APIs

| Best Practice            | Explanation                                        |
|--------------------------|----------------------------------------------------|
| Store API keys securely  | Use environment variables, never hardcode keys     |
| Handle rate limits       | Add delays between requests to avoid blocking      |
| Implement retry logic    | Retry failed requests using exponential backoff    |
| Cache responses          | Store API data locally to reduce repeated calls    |
| Validate response data   | Check status codes and structure before processing |
| Use timeouts             | Prevent hanging requests using timeout limits      |
| Log API calls            | Maintain logs for debugging and monitoring         |
| Respect terms of service | Follow API provider usage policies                 |

### Secure API Key Storage Example

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 API_KEY = os.getenv('WEATHER_API_KEY')
7
8 # Never hardcode API keys like:
9 # API_KEY = "abc123"
```

## 11. Quick Revision Summary

| Concept         | Key Points                                                      |
|-----------------|-----------------------------------------------------------------|
| API             | Interface allowing software systems to communicate              |
| REST API        | Lightweight, uses JSON, stateless, based on HTTP methods        |
| SOAP API        | Heavy, XML-based, protocol-driven, can be stateful              |
| GET             | Retrieve data (read operation)                                  |
| POST            | Create new data                                                 |
| PUT             | Update entire record                                            |
| DELETE          | Remove data                                                     |
| Status Code 200 | OK – Request successful                                         |
| Status Code 201 | Created – Resource successfully created                         |
| Status Code 400 | Bad Request – Client-side error                                 |
| Status Code 401 | Unauthorized – Authentication required                          |
| Status Code 404 | Not Found – Endpoint does not exist                             |
| Status Code 500 | Internal Server Error – Server-side failure                     |
| Endpoint        | Specific URL where API is accessed                              |
| Header          | Metadata sent with request (e.g., authentication, content type) |
| Query Parameter | Key-value pair in URL used for filtering (?key=value)           |

## Data Cleaning: Handling Missing Values and Duplicates

### 1. What is Data Cleaning?

Data Cleaning (or Data Cleansing) is the process of detecting and correcting (or removing) corrupt, inaccurate, incomplete, or duplicate records from a dataset.

#### 1.1 Importance of Data Cleaning

| Reason                 | Explanation                                           |
|------------------------|-------------------------------------------------------|
| Improves accuracy      | Clean data leads to reliable analysis and predictions |
| Saves time             | Prevents errors that waste time later in the pipeline |
| Increases productivity | Analysts trust data and work faster                   |
| Reduces storage        | Removing duplicates and irrelevant data saves space   |
| Better decisions       | Garbage in = Garbage out (GIGO principle)             |

#### 1.2 Common Data Quality Issues

| Issue                   | Description                         | Example                        |
|-------------------------|-------------------------------------|--------------------------------|
| Missing Values          | Empty or null values in dataset     | Age column has NaN             |
| Duplicate Records       | Same record appears multiple times  | Same customer listed twice     |
| Outliers                | Extreme values outside normal range | Age = 999 years                |
| Inconsistent Formatting | Same data stored differently        | “NY”, “New York”, “new york”   |
| Incorrect Data Types    | Wrong type for the column           | Price stored as text “\$49.99” |
| Invalid Values          | Impossible values                   | Gender = “Unknown123”          |
| Contradictions          | Logically inconsistent data         | Birth date = 2000, Age = 50    |

## 2. Handling Missing Values

### 2.1 What are Missing Values?

Missing values are empty, null, or NA (Not Available) entries in a dataset. Common representations:

| Representation  | Description                     |
|-----------------|---------------------------------|
| NaN             | Not a Number (pandas default)   |
| None            | Python’s null value             |
| null            | JSON/JavaScript null            |
| NA              | Not Available                   |
| ? or -          | Placeholder characters          |
| Empty string "" | Blank value                     |
| 0 or -1         | Placeholder numbers (dangerous) |

#### 2.2 Types of Missing Data Patterns

| Pattern                      | Abbreviation | Meaning                                          | Example                        |
|------------------------------|--------------|--------------------------------------------------|--------------------------------|
| Missing Completely at Random | MCAR         | No relationship between missingness and any data | Random sensor glitch           |
| Missing at Random            | MAR          | Missingness depends on other observed variables  | Men skip pregnancy question    |
| Missing Not at Random        | MNAR         | Missingness depends on the missing value itself  | High-income people hide income |

### 2.3 Step 1: Detect Missing Values

Using CSV Module:

```
1 import csv
2
3 def detect_missing_csv(filename):
4     """Detect missing values in CSV file using csv module"""
5
6     with open(filename, 'r') as file:
7         reader = csv.DictReader(file)
8         rows = list(reader)
9
10        # Get column names
11        columns = reader.fieldnames
12
13        # Initialize missing count dictionary
14        missing_counts = {col: 0 for col in columns}
15
```

```
16     # Count missing values
17     for row in rows:
18         for col in columns:
19             if not row[col] or row[col].strip() == '' or row[col] == 'NA' or row[col] == 'null':
20                 missing_counts[col] += 1
21
22     # Print results
23     total_rows = len(rows)
24     print(f"Total rows: {total_rows}")
25     print("\nMissing Values Report:")
26     print("-" * 40)
27     for col, count in missing_counts.items():
28         percentage = (count / total_rows) * 100
29         print(f"{col:15} | Missing: {count:5} | {percentage:5.1f}%")
30
31     return missing_counts, total_rows
32
33 # Usage
34 detect_missing_csv('customers.csv')
```

Listing 2.1: Detect Missing Values using CSV Module

Sample Output:

```
1 Total rows: 100
2
3 Missing Values Report:
4 -----
5 customer_id    | Missing:    0 |    0.0%
6 name           | Missing:    2 |    2.0%
7 age            | Missing:    5 |    5.0%
8 city           | Missing:    3 |    3.0%
9 amount         | Missing:    1 |    1.0%
```

Using Pandas

```
1 import pandas as pd
2
3 # Read CSV
4 df = pd.read_csv('customers.csv')
5
6 # Check missing values
7 print("Missing Values Count:")
8 print(df.isnull().sum())
9
10 print("\nMissing Values Percentage:")
11 print((df.isnull().sum() / len(df)) * 100)
12
13 # Visual check of missing values
14 print("\nDataFrame Info:")
15 print(df.info())
16
17 # Check for specific missing representations
18 df = pd.read_csv('customers.csv', na_values=['', 'NA', 'NULL', '?', 'missing'])
```

Listing 2.2: Detect Missing Values using Pandas

2.4 Step 2: Understand Missing Data Pattern

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Check which rows have missing values
6 rows_with_missing = df[df.isnull().any(axis=1)]
7 print(f"Rows with at least one missing value: {len(rows_with_missing)}")
8
9 # Check rows with all missing values
10 rows_all_missing = df[df.isnull().all(axis=1)]
11 print(f"Rows with all missing values: {len(rows_all_missing)}")
12
13 # Check missing pattern across columns
14 missing_matrix = df.isnull().sum()
15 print(missing_matrix)
16
17 # Visualize missing values (if matplotlib available)
18 try:
19     import missingno as msno
20     msno.matrix(df)
21     plt.show()
22 except:
23     print("missingno library not installed. Skipping visualization.")
```

Listing 2.3: Understanding Missing Data Patterns

2.5 Step 3: Handle Missing Values

Method A: Deletion Methods

A1. Delete Rows with Missing Values (Listwise Deletion) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Delete rows with any missing value
6 df_cleaned = df.dropna()
7 print(f"Original: {len(df)} rows")
8 print(f"After dropna(): {len(df_cleaned)} rows")
9
10 # Delete rows with missing values in specific columns
11 df_cleaned = df.dropna(subset=['age', 'amount'])
12 print(f"After dropping rows missing age or amount: {len(df_cleaned)} rows")
13
14 # Delete rows where all values are missing
15 df_cleaned = df.dropna(how='all')
16
17 # Delete rows where at least 3 values are missing
18 df_cleaned = df.dropna(thresh=len(df.columns) - 2)
```

Listing 2.4: Drop Missing Rows using Pandas

Using CSV Module:

```
1 import csv
2
3 def drop_missing_rows_csv(input_file, output_file, columns_to_check=None):
4     """Delete rows with missing values from CSV"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         fieldnames = reader.fieldnames
9         rows = list(reader)
10
11     original_count = len(rows)
12
13     # Filter rows
14     if columns_to_check:
15         filtered_rows = [row for row in rows
16                         if all(row[col] and row[col].strip() for col in columns_to_check)]
17     else:
18         filtered_rows = [row for row in rows
19                         if all(row[col] and row[col].strip() for col in fieldnames)]
20
21     removed_count = original_count - len(filtered_rows)
22
23     # Write cleaned data
24     with open(output_file, 'w', newline='') as outfile:
25         writer = csv.DictWriter(outfile, fieldnames=fieldnames)
26         writer.writeheader()
27         writer.writerows(filtered_rows)
28
29     print(f"Original rows: {original_count}")
30     print(f"Removed rows: {removed_count}")
31     print(f"Remaining rows: {len(filtered_rows)}")
32
33     return filtered_rows
34
35 # Usage
36 drop_missing_rows_csv('customers.csv', 'customers_no_missing.csv', ['age', 'amount'])
```

Listing 2.5: Drop Missing Rows using CSV Module

A2. Delete Columns with Many Missing Values Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Drop columns with > 50% missing values
6 threshold = 0.5
7 missing_percentage = df.isnull().sum() / len(df)
8 cols_to_drop = missing_percentage[missing_percentage > threshold].index
9 df_cleaned = df.drop(columns=cols_to_drop)
10
11 print(f"Dropped columns: {list(cols_to_drop)}")
12
13 # Drop column entirely
14 df_cleaned = df.drop(columns=['unnecessary_column'])
```

Listing 2.6: Drop Columns with Missing Values using Pandas

Using CSV Module:

```
1 import csv
2
3 def drop_columns_csv(input_file, output_file, columns_to_drop):
4     """Delete specific columns from CSV"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         fieldnames = [col for col in reader.fieldnames if col not in columns_to_drop]
9
10    with open(output_file, 'w', newline='') as outfile:
11        writer = csv.DictWriter(outfile, fieldnames=fieldnames)
12        writer.writeheader()
13
14        for row in reader:
15            for col in columns_to_drop:
16                del row[col]
17            writer.writerow(row)
18
19    print(f"Removed columns: {columns_to_drop}")
20    print(f"Remaining columns: {fieldnames}")
21
22 # Usage
23 drop_columns_csv('customers.csv', 'customers_no_age.csv', ['age'])
```

Listing 2.7: Drop Columns using CSV Module

Method B: Imputation Methods (Filling Missing Values)

B1. Mean Imputation (For Numeric Columns) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate mean
6 mean_age = df['age'].mean()
7 print(f"Mean age: {mean_age:.2f}")
8
9 # Fill missing values with mean
10 df['age_filled'] = df['age'].fillna(mean_age)
11
12 # Alternative: fill in place
13 df['age'].fillna(df['age'].mean(), inplace=True)
```

Listing 2.8: Mean Imputation using Pandas

Using CSV Module:

```
1 import csv
2
3 def mean_imputation_csv(input_file, output_file, numeric_column):
4     """Fill missing numeric values with column mean"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         rows = list(reader)
9
10    # Calculate mean
11    values = []
12    for row in rows:
13        if row[numeric_column] and row[numeric_column].strip():
14            try:
15                values.append(float(row[numeric_column]))
16            except ValueError:
17                pass
18
19    mean_value = sum(values) / len(values) if values else 0
20
21    # Fill missing values
22    for row in rows:
23        if not row[numeric_column] or row[numeric_column].strip() == '':
24            row[numeric_column] = str(mean_value)
25
26    # Write to output
27    with open(output_file, 'w', newline='') as outfile:
28        writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
29        writer.writeheader()
30        writer.writerows(rows)
31
32    print(f"Missing values in '{numeric_column}' filled with mean: {mean_value:.2f}")
33
34 # Usage
35 mean_imputation_csv('customers.csv', 'customers_mean_imputed.csv', 'age')
```

Listing 2.9: Mean Imputation using CSV Module

B2. Median Imputation (For Skewed Numeric Columns)

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate median
6 median_age = df['age'].median()
7 median_amount = df['amount'].median()
8
9 # Fill with median (better for skewed data with outliers)
10 df['age'].fillna(df['age'].median(), inplace=True)
11 df['amount'].fillna(df['amount'].median(), inplace=True)
12
13 print(f"Filled age with median: {median_age}")
14 print(f"Filled amount with median: {median_amount}")
```

Listing 2.10: Median Imputation using Pandas

B3. Mode Imputation (For Categorical Columns) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate mode (most frequent value)
6 mode_city = df['city'].mode()[0]
7 print(f"Most frequent city: {mode_city}")
8
9 # Fill with mode
10 df['city'].fillna(mode_city, inplace=True)
```

Listing 2.11: Mode Imputation using Pandas

Using CSV Module:

```
1 import csv
2 from collections import Counter
3
4 def mode_imputation_csv(input_file, output_file, categorical_column):
5     """Fill missing categorical values with column mode"""
6
7     with open(input_file, 'r') as infile:
8         reader = csv.DictReader(infile)
9         rows = list(reader)
10
11     # Find mode (most frequent value)
12     valid_values = []
13     for row in rows:
14         if row[categorical_column] and row[categorical_column].strip():
15             valid_values.append(row[categorical_column])
16
17     if valid_values:
18         mode_value = Counter(valid_values).most_common(1)[0][0]
19     else:
20         mode_value = "Unknown"
21
22     # Fill missing values
23     for row in rows:
24         if not row[categorical_column] or row[categorical_column].strip() == '':
25             row[categorical_column] = mode_value
26
27     # Write to output
28     with open(output_file, 'w', newline='') as outfile:
29         writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
30         writer.writeheader()
31         writer.writerows(rows)
32
33     print(f"Missing values in '{categorical_column}' filled with mode: {mode_value}")
34
35 # Usage
36 mode_imputation_csv('customers.csv', 'customers_mode_imputed.csv', 'city')
```

Listing 2.12: Mode Imputation using CSV Module

B4. Constant Imputation

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Fill with specific constant value
6 df['city'].fillna('Unknown', inplace=True)
7 df['age'].fillna(-1, inplace=True) # Placeholder for missing
8 df['amount'].fillna(0, inplace=True)
9
10 # Create indicator column for missing values
11 df['age_missing'] = df['age'].isnull().astype(int)
```

Listing 2.13: Constant Imputation using Pandas

B5. Forward Fill and Backward Fill (For Time Series)

```
1 import pandas as pd
2
3 # Example time series data
4 df_time = pd.DataFrame({
5     'date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04'],
6     'value': [100, None, None, 150]
7 })
8
9 # Forward fill
10 df_time['value_ffill'] = df_time['value'].fillna(method='ffill')
11
12 # Backward fill
13 df_time['value_bfill'] = df_time['value'].fillna(method='bfill')
14
15 # Interpolation
16 df_time['value_interp'] = df_time['value'].interpolate()
17
18 print(df_time)
```

Listing 2.14: Forward Fill

Output:

|   | date       | value | value_ffill | value_bfill | value_interp |
|---|------------|-------|-------------|-------------|--------------|
| 0 | 2024-01-01 | 100.0 | 100.0       | 100.0       | 100.0        |
| 1 | 2024-01-02 | NaN   | 100.0       | 150.0       | 116.7        |
| 2 | 2024-01-03 | NaN   | 100.0       | 150.0       | 133.3        |
| 3 | 2024-01-04 | 150.0 | 150.0       | 150.0       | 150.0        |

B6. KNN Imputation (Advanced)

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.impute import KNNImputer
4
5 # Create sample data
6 df = pd.DataFrame({
7     'age': [25, 30, None, 35, 40],
8     'income': [50000, 60000, 55000, None, 80000],
9     'score': [85, None, 90, 88, 92]
10 })
11
12 # Apply KNN Imputer
13 imputer = KNNImputer(n_neighbors=3)
14 df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
15
16 print("Original:")
17 print(df)
18
19 print("\nAfter KNN Imputation:")
20 print(df_imputed)
```

Listing 2.15: KNN Imputation using Scikit-learn

2.6 Summary: When to Use Which Imputation Method

| Method                | Best For               | When to Use                      | Example                     |
|-----------------------|------------------------|----------------------------------|-----------------------------|
| Drop rows             | MCAR pattern           | ≤5% missing, random pattern      | Random missing values       |
| Drop columns          | Low importance column  | ≤50% missing                     | “notes” column mostly empty |
| Mean imputation       | Symmetric numeric data | No outliers, normal distribution | Test scores                 |
| Median imputation     | Skewed numeric data    | Has outliers                     | Income, house prices        |
| Mode imputation       | Categorical data       | Nominal categories               | City, gender                |
| Forward/Backward fill | Time series            | Consecutive missing              | Stock prices, sensor data   |
| Interpolation         | Time series with trend | Gradual changes                  | Temperature readings        |
| KNN imputation        | Complex relationships  | Many correlated features         | Customer profiles           |
| Constant imputation   | Marking missing        | Need to preserve missing flag    | “Unknown” category          |

Handling Duplicates

What are Duplicates?

Duplicates are identical or nearly identical records that appear multiple times in a dataset.

| Type               | Description                          | Example                         |
|--------------------|--------------------------------------|---------------------------------|
| Exact Duplicates   | All columns have identical values    | Same customer row appears twice |
| Partial Duplicates | Only key columns match               | Same email but different name   |
| Near Duplicates    | Values are similar but not identical | “John Smith” vs “Jon Smith”     |

Detecting Duplicates (Using Pandas)

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Check for duplicate rows
6 duplicate_rows = df.duplicated()
7 print(duplicate_rows.sum())
8
9 # View duplicate rows
10 duplicates = df[duplicate_rows]
11 print(duplicates)
12
13 # Check duplicates based on specific columns
14 duplicate_by_name = df.duplicated(subset=['name'])
15 duplicate_by_name_city = df.duplicated(subset=['name', 'city'])
16
17 # Keep first occurrence
18 df[df.duplicated(keep='first')]
19
20 # Keep last occurrence
21 df[df.duplicated(keep='last')]
22
23 # Mark duplicates
24 df['is_duplicate'] = df.duplicated()
```

Detecting Duplicates (Using CSV Module)

```
1 import csv
2
3 def detect_duplicates_csv(filename, key_columns=None):
4     with open(filename, 'r') as file:
5         reader = csv.DictReader(file)
6         rows = list(reader)
7
8         seen = {}
9         duplicates = []
10
11         if key_columns:
12             for i, row in enumerate(rows):
13                 key = tuple(row[col] for col in key_columns)
14                 if key in seen:
15                     duplicates.append((i, row))
16                 else:
17                     seen[key] = i
18         else:
19             for i, row in enumerate(rows):
20                 key = tuple(row.items())
21                 if key in seen:
22                     duplicates.append((i, row))
23                 else:
24                     seen[key] = i
25
26         print(f"Total rows: {len(rows)}")
27         print(f"Unique rows: {len(seen)}")
28         print(f"Duplicate rows: {len(duplicates)}")
29
30     return duplicates
```

3.3 Removing Duplicates

Using Pandas

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Remove exact duplicates (keep first occurrence)
6 df_unique = df.drop_duplicates()
7 print(len(df), len(df_unique))
8
9 # Remove duplicates keeping last occurrence
10 df_unique = df.drop_duplicates(keep='last')
11
12 # Remove duplicates based on specific columns
13 df_unique = df.drop_duplicates(subset=['name', 'city'])
14
15 # Remove duplicates and reset index
16 df_unique = df.drop_duplicates().reset_index(drop=True)
17
18 # Count removed duplicates
```

```
19 before = len(df)
20 df_unique = df.drop_duplicates()
21 after = len(df_unique)
22 print(f"Removed {before - after} duplicate rows")
```

Using CSV Module

```
1 import csv
2
3 def remove_duplicates_csv(input_file, output_file, key_columns=None):
4     with open(input_file, 'r') as infile:
5         reader = csv.DictReader(infile)
6         rows = list(reader)
7
8         seen = set()
9         unique_rows = []
10
11     for row in rows:
12         if key_columns:
13             key = tuple(row[col] for col in key_columns)
14         else:
15             key = tuple(row.items())
16
17         if key not in seen:
18             seen.add(key)
19             unique_rows.append(row)
20
21     with open(output_file, 'w', newline='') as outfile:
22         writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
23         writer.writeheader()
24         writer.writerows(unique_rows)
25
26     print(f"Original rows: {len(rows)}")
27     print(f"Unique rows: {len(unique_rows)}")
28     print(f"Removed duplicates: {len(rows) - len(unique_rows)}")
29
30     return unique_rows
31
32 # Usage
33 remove_duplicates_csv('customers.csv', 'customers_no_duplicates.csv', ['customer_id'])
34 remove_duplicates_csv('customers.csv', 'customers_no_duplicates_name.csv', ['name', 'email'])
```

3.4 Handling Near-Duplicates (Fuzzy Matching)

```
1 import pandas as pd
2 from difflib import SequenceMatcher
3
4 def similarity_ratio(a, b):
5     return SequenceMatcher(None, a.lower(), b.lower()).ratio()
6
7 def detect_near_duplicates(df, column, threshold=0.85):
8     duplicates = []
9     values = df[column].dropna().tolist()
10
11     for i in range(len(values)):
12         for j in range(i+1, len(values)):
13             ratio = similarity_ratio(values[i], values[j])
14
15             if ratio >= threshold and values[i] != values[j]:
16                 duplicates.append({
17                     'row1_index': i,
18                     'row1_value': values[i],
19                     'row2_index': j,
20                     'row2_value': values[j],
21                     'similarity': ratio
22                 })
23
24     return pd.DataFrame(duplicates)
25
26 data = pd.DataFrame({
27     'name': ['John Smith', 'Jon Smith', 'John Smyth', 'Alice Brown', 'Ali ce Brown']
28 })
29
30 near_dupes = detect_near_duplicates(data, 'name', threshold=0.8)
31 print(near_dupes)
```

3.5 Handling Duplicates with Aggregation

```
1 import pandas as pd
2
3 df = pd.DataFrame({
```

```
4     'customer_id': [1, 1, 2, 2, 3],
5     'name': ['Alice', 'Alice', 'Bob', 'Bob', 'Charlie'],
6     'amount': [100, 150, 200, 50, 300],
7     'date': ['2024-01-01', '2024-01-15', '2024-01-05', '2024-01-10', '2024-01-01']
8 })
9
10 # Aggregate duplicates
11 df_agg = df.groupby('customer_id').agg({
12     'name': 'first',
13     'amount': 'sum',
14     'date': 'min'
15 }).reset_index()
16
17 print(df_agg)
```

## 4. Complete Data Cleaning Pipeline Example

### Example: Cleaning Customer Dataset

```
1 import pandas as pd
2 import numpy as np
3
4 print("=" * 70)
5 print("COMPLETE DATA CLEANING PIPELINE")
6
7 # ===== STEP 1: Load Data =====
8 df = pd.read_csv('customers_raw.csv')
9 print(df.shape)
10 print(list(df.columns))
11
12 # ===== STEP 2: Initial Inspection =====
13 print(df.isnull().sum())
14 print(df.duplicated().sum())
15
16 # ===== STEP 3: Remove Duplicates =====
17 initial_rows = len(df)
18 df = df.drop_duplicates()
19 print(initial_rows - len(df))
20
21 # ===== STEP 4: Handle Missing Values =====
22 missing_pct = (df.isnull().sum() / len(df)) * 100
23
24 # Drop high missing columns
25 high_missing_cols = missing_pct[missing_pct > 50].index
26 df = df.drop(columns=high_missing_cols)
27
28 # Numeric columns median imputation
29 numeric_cols = df.select_dtypes(include=[np.number]).columns
30 for col in numeric_cols:
31     if df[col].isnull().sum() > 0:
32         df[col].fillna(df[col].median(), inplace=True)
33
34 # Categorical columns mode imputation
35 categorical_cols = df.select_dtypes(include=['object']).columns
36 for col in categorical_cols:
37     if df[col].isnull().sum() > 0:
38         df[col].fillna(df[col].mode()[0], inplace=True)
39
40 # ===== STEP 5: Final Check =====
41 print(df.shape)
42 print(df.isnull().sum().sum())
43
44 # ===== STEP 6: Save Cleaned Data =====
45 df.to_csv('customers_cleaned.csv', index=False)
46 print("Saved cleaned dataset")
```

## Data Integration and Transformation

### 1. What is Data Integration

Data Integration is the process of combining data from different sources (files, databases, APIs, spreadsheets) into a unified and consistent dataset.

#### 1.1 Why Data Integration is Needed

| Reason             | Explanation                           | Example                     |
|--------------------|---------------------------------------|-----------------------------|
| Data silos         | Data stored in separate systems       | CRM vs ERP systems          |
| Different formats  | Multiple formats exist                | CSV, Excel, JSON            |
| Inconsistent IDs   | Same entity has different identifiers | Customer ID mismatch        |
| Complementary data | Each source has partial information   | Demographics + transactions |

## 2. Data Integration Operation: Concatenation

Concatenation refers to stacking datasets vertically by adding rows. All datasets must have the same columns.

### Example: Pandas Concatenation

```
1 import pandas as pd
2
3 df_jan = pd.DataFrame({
4     'customer_id': [1, 2, 3],
5     'name': ['Alice', 'Bob', 'Charlie'],
6     'amount': [250, 180, 320]
7 })
8
9 df_feb = pd.DataFrame({
10     'customer_id': [4, 5, 6],
11     'name': ['Diana', 'Eve', 'Frank'],
12     'amount': [410, 280, 350]
13 })
14
15 print(df_jan)
16 print(df_feb)
17
18 df_all = pd.concat([df_jan, df_feb], axis=0)
19 print(df_all)
```

### Output

January DataFrame:

|   | customer_id | name    | amount |
|---|-------------|---------|--------|
| 0 | 1           | Alice   | 250    |
| 1 | 2           | Bob     | 180    |
| 2 | 3           | Charlie | 320    |

February DataFrame:

|   | customer_id | name  | amount |
|---|-------------|-------|--------|
| 0 | 4           | Diana | 410    |
| 1 | 5           | Eve   | 280    |
| 2 | 6           | Frank | 350    |

Concatenated Result (without resetting index):

|   | customer_id | name    | amount |
|---|-------------|---------|--------|
| 0 | 1           | Alice   | 250    |
| 1 | 2           | Bob     | 180    |
| 2 | 3           | Charlie | 320    |
| 0 | 4           | Diana   | 410    |
| 1 | 5           | Eve     | 280    |
| 2 | 6           | Frank   | 350    |

### Concatenation with Reset Index

```
1 df_all_reset = pd.concat([df_jan, df_feb], axis=0, ignore_index=True)
2 print(df_all_reset)
```

### Output

|   | customer_id | name    | amount |
|---|-------------|---------|--------|
| 0 | 1           | Alice   | 250    |
| 1 | 2           | Bob     | 180    |
| 2 | 3           | Charlie | 320    |
| 3 | 4           | Diana   | 410    |
| 4 | 5           | Eve     | 280    |
| 5 | 6           | Frank   | 350    |

### Concatenation with Reset Index

```
1 df_all_reset = pd.concat([df_jan, df_feb], axis=0, ignore_index=True)
2 print(df_all_reset)
```

### Output

Concatenated Result (with reset index):

|   | customer_id | name    | amount |
|---|-------------|---------|--------|
| 0 | 1           | Alice   | 250    |
| 1 | 2           | Bob     | 180    |
| 2 | 3           | Charlie | 320    |
| 3 | 4           | Diana   | 410    |
| 4 | 5           | Eve     | 280    |
| 5 | 6           | Frank   | 350    |

### Concatenating Multiple Files

```
1 import pandas as pd
2
3 df_jan = pd.DataFrame({'id': [1,2], 'sales': [100,200]})
4 df_feb = pd.DataFrame({'id': [3,4], 'sales': [300,400]})
5 df_mar = pd.DataFrame({'id': [5,6], 'sales': [500,600]})
6
7 df_q1 = pd.concat([df_jan, df_feb, df_mar], ignore_index=True)
8 print(df_q1)
9 print(df_q1.shape)
```

### Output

January data:

|   | id | sales |
|---|----|-------|
| 0 | 1  | 100   |
| 1 | 2  | 200   |

February data:

|   | id | sales |
|---|----|-------|
| 0 | 3  | 300   |
| 1 | 4  | 400   |

March data:

|   | id | sales |
|---|----|-------|
| 0 | 5  | 500   |
| 1 | 6  | 600   |

Q1 Combined Data:

|   | id | sales |
|---|----|-------|
| 0 | 1  | 100   |
| 1 | 2  | 200   |
| 2 | 3  | 300   |
| 3 | 4  | 400   |
| 4 | 5  | 500   |
| 5 | 6  | 600   |

Q1 data shape: (6, 2)

### Handling Different Columns During Concatenation

```
1 df1 = pd.DataFrame({'id': [1, 2], 'name': ['Alice', 'Bob']})
2 df2 = pd.DataFrame({'id': [3, 4], 'age': [25, 30]})
3
4 df_outer = pd.concat([df1, df2], axis=0, ignore_index=True)
5 print(df_outer)
6
7 df_inner = pd.concat([df1, df2], axis=0, ignore_index=True, join='inner')
8 print(df_inner)
```

### Output

Outer Concatenation (fills missing with NaN):

|   | id | name  | age  |
|---|----|-------|------|
| 0 | 1  | Alice | NaN  |
| 1 | 2  | Bob   | NaN  |
| 2 | 3  | NaN   | 25.0 |
| 3 | 4  | NaN   | 30.0 |

Inner Concatenation (only common columns):

|   | id |
|---|----|
| 0 | 1  |
| 1 | 2  |
| 2 | 3  |
| 3 | 4  |

#### 2.0.1 Concatenating CSV Files Using Python (CSV Module)

```
1 import csv
2
3 # Sample CSV files creation
4 with open('sales_jan.csv', 'w') as f:
5     f.write("id,product,sales\n1,A,100\n2,B,150")
6
7 with open('sales_feb.csv', 'w') as f:
8     f.write("id,product,sales\n3,C,200\n4,D,250")
9
```

```
10 print("Sample files created successfully")
11
12 def concatenate_csv_files(input_files, output_file):
13     first_file = True
14
15     for file in input_files:
16         with open(file, 'r') as infile:
17             reader = csv.reader(infile)
18
19             with open(output_file, 'a', newline='') as outfile:
20                 writer = csv.writer(outfile)
21
22                 for i, row in enumerate(reader):
23                     if i == 0 and first_file:
24                         writer.writerow(row)    # write header once
25                     elif i > 0:
26                         writer.writerow(row)
27
28             first_file = False
29             print(f"Processed: {file}")
30
31     print(f"\nFinal file saved as: {output_file}")
32
33     with open(output_file, 'r') as f:
34         print(f.read())
35
36 concatenate_csv_files(['sales_jan.csv', 'sales_feb.csv'], 'sales_q1.csv')
```

Sample files created successfully  
Processed: sales\_jan.csv  
Processed: sales\_feb.csv

Final file saved as: sales\_q1.csv

id,product,sales  
1,A,100  
2,B,150  
3,C,200  
4,D,250

### 2.0.2 Merging (Joining) - Adding Columns Horizontally

**Definition:** Merging (or joining) combines datasets horizontally by using a common key column.

#### Sample Data for Join Operations

```
1 import pandas as pd
2
3 # Customers table
4 customers = pd.DataFrame({
5     'customer_id': [1, 2, 3, 4],
6     'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
7     'city': ['New York', 'Los Angeles', 'Chicago', 'Houston']
8 })
9
10 # Orders table
11 orders = pd.DataFrame({
12     'order_id': [101, 102, 103, 104, 105],
13     'customer_id': [1, 2, 1, 3, 5],
14     'amount': [250, 180, 320, 410, 280],
15     'date': ['2024-01-15', '2024-01-20', '2024-02-01', '2024-02-10', '2024-02-15']
16 })
17
18 print("Customers Table:")
19 print(customers)
20
21 print("\nOrders Table:")
22 print(orders)
```

Customers Table:

|   | customer_id | name    | city        |
|---|-------------|---------|-------------|
| 0 | 1           | Alice   | New York    |
| 1 | 2           | Bob     | Los Angeles |
| 2 | 3           | Charlie | Chicago     |
| 3 | 4           | Diana   | Houston     |

Orders Table:

|   | order_id | customer_id | amount | date       |
|---|----------|-------------|--------|------------|
| 0 | 101      | 1           | 250    | 2024-01-15 |
| 1 | 102      | 2           | 180    | 2024-01-20 |
| 2 | 103      | 1           | 320    | 2024-02-01 |
| 3 | 104      | 3           | 410    | 2024-02-10 |
| 4 | 105      | 5           | 280    | 2024-02-15 |

A. Inner Join (Keep Only Matching Keys)

```
1 inner_join = pd.merge(customers, orders, on='customer_id', how='inner')
2 print("INNER JOIN (customers with orders):")
3 print(inner_join)
```

| INNER JOIN (customers with orders): |             |         |             |          |        |            |
|-------------------------------------|-------------|---------|-------------|----------|--------|------------|
|                                     | customer_id | name    | city        | order_id | amount | date       |
| 0                                   | 1           | Alice   | New York    | 101      | 250    | 2024-01-15 |
| 1                                   | 1           | Alice   | New York    | 103      | 320    | 2024-02-01 |
| 2                                   | 2           | Bob     | Los Angeles | 102      | 180    | 2024-01-20 |
| 3                                   | 3           | Charlie | Chicago     | 104      | 410    | 2024-02-10 |

B. Left Join (Keep All Customers)

```
1 left_join = pd.merge(customers, orders, on='customer_id', how='left')
2 print("LEFT JOIN (all customers):")
3 print(left_join)
```

| LEFT JOIN (all customers): |             |         |             |          |        |            |
|----------------------------|-------------|---------|-------------|----------|--------|------------|
|                            | customer_id | name    | city        | order_id | amount | date       |
| 0                          | 1           | Alice   | New York    | 101.0    | 250.0  | 2024-01-15 |
| 1                          | 1           | Alice   | New York    | 103.0    | 320.0  | 2024-02-01 |
| 2                          | 2           | Bob     | Los Angeles | 102.0    | 180.0  | 2024-01-20 |
| 3                          | 3           | Charlie | Chicago     | 104.0    | 410.0  | 2024-02-10 |
| 4                          | 4           | Diana   | Houston     | NaN      | NaN    | NaN        |

C. Right Join (Keep All Orders)

```
1 right_join = pd.merge(customers, orders, on='customer_id', how='right')
2 print("RIGHT JOIN (all orders):")
3 print(right_join)
```

| RIGHT JOIN (all orders): |             |         |             |          |        |            |
|--------------------------|-------------|---------|-------------|----------|--------|------------|
|                          | customer_id | name    | city        | order_id | amount | date       |
| 0                        | 1.0         | Alice   | New York    | 101      | 250    | 2024-01-15 |
| 1                        | 1.0         | Alice   | New York    | 103      | 320    | 2024-02-01 |
| 2                        | 2.0         | Bob     | Los Angeles | 102      | 180    | 2024-01-20 |
| 3                        | 3.0         | Charlie | Chicago     | 104      | 410    | 2024-02-10 |
| 4                        | 5.0         | NaN     | NaN         | 105      | 280    | 2024-02-15 |

D. Outer Join (Keep All Records)

```
1 outer_join = pd.merge(customers, orders, on='customer_id', how='outer')
2 print("OUTER JOIN (all customers + all orders):")
3 print(outer_join)
```

| OUTER JOIN (all customers + all orders): |             |         |             |          |        |            |
|------------------------------------------|-------------|---------|-------------|----------|--------|------------|
|                                          | customer_id | name    | city        | order_id | amount | date       |
| 0                                        | 1.0         | Alice   | New York    | 101.0    | 250.0  | 2024-01-15 |
| 1                                        | 1.0         | Alice   | New York    | 103.0    | 320.0  | 2024-02-01 |
| 2                                        | 2.0         | Bob     | Los Angeles | 102.0    | 180.0  | 2024-01-20 |
| 3                                        | 3.0         | Charlie | Chicago     | 104.0    | 410.0  | 2024-02-10 |
| 4                                        | 4.0         | Diana   | Houston     | NaN      | NaN    | NaN        |
| 5                                        | 5.0         | NaN     | NaN         | 105.0    | 280.0  | 2024-02-15 |

2.0.3 Data Transformation

**Definition:** Data transformation is the process of converting data into a different format, scale, or structure to improve analysis and machine learning performance.

3.1 Scaling and Normalization

Sample Data

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'age': [25, 30, 35, 40, 45, 100],
6     'income': [30000, 45000, 60000, 75000, 90000, 120000],
7     'score': [85, 78, 92, 88, 95, 70]
8 })
9
10 print(df)
```

|   | age | income | score |
|---|-----|--------|-------|
| 0 | 25  | 30000  | 85    |
| 1 | 30  | 45000  | 78    |
| 2 | 35  | 60000  | 92    |
| 3 | 40  | 75000  | 88    |
| 4 | 45  | 90000  | 95    |
| 5 | 100 | 120000 | 70    |

```
1 df['age_minmax'] = (df['age'] - df['age'].min()) / (df['age'].max() - df['age'].min())
2 df['income_minmax'] = (df['income'] - df['income'].min()) / (df['income'].max() - df['income'].min())
3
4 print(df[['age', 'age_minmax', 'income', 'income_minmax']])
```

|   | age | age_minmax | income | income_minmax |
|---|-----|------------|--------|---------------|
| 0 | 25  | 0.000000   | 30000  | 0.000000      |
| 1 | 30  | 0.066667   | 45000  | 0.166667      |
| 2 | 35  | 0.133333   | 60000  | 0.333333      |
| 3 | 40  | 0.200000   | 75000  | 0.500000      |
| 4 | 45  | 0.266667   | 90000  | 0.666667      |
| 5 | 100 | 1.000000   | 120000 | 1.000000      |

```
1 df['age_zscore'] = (df['age'] - df['age'].mean()) / df['age'].std()
2 df['income_zscore'] = (df['income'] - df['income'].mean()) / df['income'].std()
3
4 print(df[['age', 'age_zscore', 'income', 'income_zscore']])
```

|   | age | age_zscore | income | income_zscore |
|---|-----|------------|--------|---------------|
| 0 | 25  | -0.822222  | 30000  | -1.336306     |
| 1 | 30  | -0.622222  | 45000  | -0.801784     |
| 2 | 35  | -0.422222  | 60000  | -0.267261     |
| 3 | 40  | -0.222222  | 75000  | 0.267261      |
| 4 | 45  | -0.022222  | 90000  | 0.801784      |
| 5 | 100 | 2.111111   | 120000 | 1.336306      |

Label Encoding

```
1 from sklearn.preprocessing import LabelEncoder
2
3 df = pd.DataFrame({
4     'city': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Houston']
5 })
6
7 le = LabelEncoder()
8 df['city_encoded'] = le.fit_transform(df['city'])
9
10 print(df)
```

|   | city        | city_encoded |
|---|-------------|--------------|
| 0 | New York    | 3            |
| 1 | Los Angeles | 2            |
| 2 | Chicago     | 0            |
| 3 | New York    | 3            |
| 4 | Houston     | 1            |

```
1 df_onehot = pd.get_dummies(df, columns=['city'])
2 print(df_onehot)
```

|   | city_Chicago | city_Houston | city_Los Angeles | city_New York |
|---|--------------|--------------|------------------|---------------|
| 0 | 0            | 0            | 0                | 1             |
| 1 | 0            | 0            | 1                | 0             |
| 2 | 1            | 0            | 0                | 0             |
| 3 | 0            | 0            | 0                | 1             |
| 4 | 0            | 1            | 0                | 0             |

```
1 from sklearn.preprocessing import OrdinalEncoder
2
3 df = pd.DataFrame({
4     'education': ['High School', 'Bachelor', 'Master', 'PhD']
5 })
6
7 encoder = OrdinalEncoder(categories=[['High School', 'Bachelor', 'Master', 'PhD']])
8 df['education_encoded'] = encoder.fit_transform(df[['education']])
9
10 print(df)
```

|   | education   | education_encoded |
|---|-------------|-------------------|
| 0 | High School | 0.0               |
| 1 | Bachelor    | 1.0               |
| 2 | Master      | 2.0               |
| 3 | PhD         | 3.0               |

## 2.1 Handling Skewed Distributions

### 2.1.1 Example Dataset Creation

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4
5 np.random.seed(42)
6
7 df_skewed = pd.DataFrame({
8     'income': np.random.exponential(scale=50000, size=1000),
9     'price': np.random.lognormal(mean=3, sigma=1, size=1000)
10 })
11
12 print("Original Income Statistics:")
13 print("Skewness:", df_skewed['income'].skew())
14 print("Min:", df_skewed['income'].min())
15 print("Mean:", df_skewed['income'].mean())
16 print("Median:", df_skewed['income'].median())
17 print("Max:", df_skewed['income'].max())
```

Original Income Statistics:  
Skewness: 2.01  
Min: 136.81  
Mean: 49914.97  
Median: 34754.78  
Max: 363653.77

### 2.1.2 Log Transformation

```
1 df_skewed['income_log'] = np.log1p(df_skewed['income'])
2 df_skewed['price_sqrt'] = np.sqrt(df_skewed['price'])
3
4 print("After Log Transformation:")
5 print("Skewness:", df_skewed['income_log'].skew())
6 print("Mean:", df_skewed['income_log'].mean())
7 print("Median:", df_skewed['income_log'].median())
```

After Log Transformation:  
Skewness: 0.68  
Mean: 10.41  
Median: 10.46

### 2.1.3 Box-Cox Transformation

```
1 df_skewed['income_boxcox'], lambda_val = stats.boxcox(df_skewed['income'] + 1)
2
3 print("Box-Cox Transformation:")
4 print("Lambda:", lambda_val)
5 print("Skewness:", pd.Series(df_skewed['income_boxcox']).skew())
```

Box-Cox Transformation:  
Lambda: 0.1765  
Skewness: 0.03

## 2.2 Discretization (Binning)

### 2.2.1 Sample Data

```
1 import pandas as pd
2
3 df_bins = pd.DataFrame({
4     'age': [5, 12, 18, 25, 35, 45, 55, 65, 75, 85],
5     'income': [0, 5000, 15000, 30000, 50000,
6               75000, 100000, 150000, 200000, 250000]
7 })
8
9 print(df_bins)
```

|   | age | income |
|---|-----|--------|
| 0 | 5   | 0      |
| 1 | 12  | 5000   |
| 2 | 18  | 15000  |
| 3 | 25  | 30000  |
| 4 | 35  | 50000  |
| 5 | 45  | 75000  |
| 6 | 55  | 100000 |
| 7 | 65  | 150000 |
| 8 | 75  | 200000 |
| 9 | 85  | 250000 |

2.2.2 Equal Width Binning

```
1 df_bins['age_bin'] = pd.cut(
2     df_bins['age'],
3     bins=4,
4     labels=['0-21', '21-42', '42-63', '63-84']
5 )
6
7 print(df_bins[['age', 'age_bin']])
```

|   | age | age_bin |
|---|-----|---------|
| 0 | 5   | 0-21    |
| 1 | 12  | 0-21    |
| 2 | 18  | 0-21    |
| 3 | 25  | 21-42   |
| 4 | 35  | 21-42   |
| 5 | 45  | 42-63   |
| 6 | 55  | 42-63   |
| 7 | 65  | 63-84   |
| 8 | 75  | 63-84   |
| 9 | 85  | 63-84   |

2.2.3 Quantile Binning

```
1 df_bins['income_bin'] = pd.qcut(
2     df_bins['income'],
3     q=4,
4     labels=['Q1', 'Q2', 'Q3', 'Q4']
5 )
6
7 print(df_bins[['income', 'income_bin']])
```

|   | income | income_bin |
|---|--------|------------|
| 0 | 0      | Q1         |
| 1 | 5000   | Q1         |
| 2 | 15000  | Q1         |
| 3 | 30000  | Q2         |
| 4 | 50000  | Q2         |
| 5 | 75000  | Q3         |
| 6 | 100000 | Q3         |
| 7 | 150000 | Q4         |
| 8 | 200000 | Q4         |
| 9 | 250000 | Q4         |

2.2.4 Custom Binning

```
1 age_bins = [0, 12, 19, 35, 60, 100]
2 labels = ['Child', 'Teen', 'Young Adult', 'Adult', 'Senior']
3
4 df_bins['age_group'] = pd.cut(
5     df_bins['age'],
6     bins=age_bins,
7     labels=labels
8 )
9
10 print(df_bins[['age', 'age_group']])
```

|   | age | age_group   |
|---|-----|-------------|
| 0 | 5   | Child       |
| 1 | 12  | Child       |
| 2 | 18  | Teen        |
| 3 | 25  | Young Adult |
| 4 | 35  | Young Adult |
| 5 | 45  | Adult       |
| 6 | 55  | Adult       |
| 7 | 65  | Senior      |
| 8 | 75  | Senior      |
| 9 | 85  | Senior      |

2.2.5 Data Type Conversion

```
1 import pandas as pd
2
3 df_types = pd.DataFrame({
4     'id': ['1', '2', '3'],
5     'price': ['$49.99', '$29.99', '$99.99'],
6     'date': ['2024-01-15', '2024-02-20', '2024-03-10'],
7     'is_active': ['True', 'False', 'True']
8 })
```

```
9
10 print("Original Data:")
11 print(df_types)
12
13 print("\nOriginal Data Types:")
14 print(df_types.dtypes)
15
16 # Convert string to integer
17 df_types['id'] = df_types['id'].astype(int)
18
19 # Convert price string to float (remove $ sign)
20 df_types['price'] = df_types['price'].str.replace('$', '').astype(float)
21
22 # Convert to datetime
23 df_types['date'] = pd.to_datetime(df_types['date'])
24
25 # Convert to boolean
26 df_types['is_active'] = df_types['is_active'].map({'True': True, 'False': False})
27
28 print("\nAfter Conversion:")
29 print(df_types)
30
31 print("\nNew Data Types:")
32 print(df_types.dtypes)
```

Original Data:

|   | id | price   | date       | is_active |
|---|----|---------|------------|-----------|
| 0 | 1  | \$49.99 | 2024-01-15 | True      |
| 1 | 2  | \$29.99 | 2024-02-20 | False     |
| 2 | 3  | \$99.99 | 2024-03-10 | True      |

Original Data Types:

|           |        |
|-----------|--------|
| id        | object |
| price     | object |
| date      | object |
| is_active | object |
| dtype:    | object |

After Conversion:

|   | id | price | date       | is_active |
|---|----|-------|------------|-----------|
| 0 | 1  | 49.99 | 2024-01-15 | True      |
| 1 | 2  | 29.99 | 2024-02-20 | False     |
| 2 | 3  | 99.99 | 2024-03-10 | True      |

New Data Types:

|           |                |
|-----------|----------------|
| id        | int64          |
| price     | float64        |
| date      | datetime64[ns] |
| is_active | bool           |
| dtype:    | object         |

## 4. Complete Data Integration and Transformation Pipeline

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler, LabelEncoder
4
5 print("=" * 70)
6 print("COMPLETE DATA INTEGRATION AND TRANSFORMATION PIPELINE")
7 print("=" * 70)
8
9 # ===== STEP 1: Load Multiple Data Sources =====
10 print("\n[1] Loading data from multiple sources...")
11
12 customers = pd.DataFrame({
13     'cust_id': [1, 2, 3, 4, 5],
14     'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
15     'city': ['NY', 'LA', 'CHI', 'HOU', 'PHX'],
16     'age': [25, 30, 35, 28, 42]
17 })
18
19 orders = pd.DataFrame({
20     'order_id': [101, 102, 103, 104, 105],
21     'customer_id': [1, 2, 1, 3, 99],
22     'amount': [250, 180, 320, 410, 280],
23     'date': ['2024-01-15', '2024-01-20', '2024-02-01', '2024-02-10', '2024-02-15']
24 })
25
26 print("Customers Table:")
27 print(customers)
28 print(f"\nCustomers shape: {customers.shape}")
```

```

29
30 print("\nOrders Table:")
31 print(orders)
32 print(f"Orders shape: {orders.shape}")
33
34 # ===== STEP 2: Data Integration =====
35 print("\n[2] Integrating data (merging customers and orders)...")
36
37 df_integrated = pd.merge(customers, orders,
38                           left_on='cust_id',
39                           right_on='customer_id',
40                           how='left')
41
42 print(f"Integrated shape: {df_integrated.shape}")
43 print("Integrated data (first 8 rows):")
44 print(df_integrated.head(8))
45
46 # ===== STEP 3: Handle Missing Values =====
47 print("\n[3] Handling missing values from integration...")
48
49 print("Missing values before handling:")
50 print(df_integrated.isnull().sum())
51
52 df_integrated['amount'].fillna(0, inplace=True)
53 df_integrated['has_order'] = df_integrated['order_id'].notna().astype(int)
54
55 print("\nMissing values after handling:")
56 print(df_integrated.isnull().sum())
57
58 # ===== STEP 4: Feature Engineering =====
59 print("\n[4] Creating new features...")
60
61 df_integrated['age_group'] = pd.cut(
62     df_integrated['age'],
63     bins=[0, 18, 30, 50, 100],
64     labels=['Youth', 'Young Adult', 'Adult', 'Senior']
65 )
66
67 df_integrated['amount_category'] = pd.cut(
68     df_integrated['amount'],
69     bins=[-1, 100, 300, 1000],
70     labels=['Low', 'Medium', 'High']
71 )
72
73 print("New features added:")
74 print(df_integrated[['age', 'age_group', 'amount', 'amount_category']].head())
75
76 # ===== STEP 5: Encoding =====
77 print("\n[5] Encoding categorical variables...")
78
79 label_encoder = LabelEncoder()
80 df_integrated['city_encoded'] = label_encoder.fit_transform(df_integrated['city'])
81
82 print("City encoding mapping:")
83 for idx, city in enumerate(label_encoder.classes_):
84     print(f" {city} -> {idx}")
85
86 df_integrated = pd.get_dummies(df_integrated, columns=['age_group'], prefix='age')
87
88 print("\nAfter encoding - new columns:")
89 print([col for col in df_integrated.columns if 'age_' in col])
90
91 # ===== STEP 6: Scaling =====
92 print("\n[6] Scaling numeric features...")
93
94 numeric_cols = ['age', 'amount']
95
96 print(f"Before scaling - Age range: [{df_integrated['age'].min()}, {df_integrated['age'].max()}]")
97 print(f"Before scaling - Amount range: [{df_integrated['amount'].min()}, {df_integrated['amount'].max()}]")
98
99 scaler = StandardScaler()
100 scaled_values = scaler.fit_transform(df_integrated[numeric_cols])
101
102 df_integrated['age_scaled'] = scaled_values[:, 0]
103 df_integrated['amount_scaled'] = scaled_values[:, 1]
104
105 print("\nAfter scaling:")
106 print(f"Age scaled - range: [{df_integrated['age_scaled'].min():.2f}, {df_integrated['age_scaled'].max():.2f}]")
107 print(f"Amount scaled - range: [{df_integrated['amount_scaled'].min():.2f}, {df_integrated['amount_scaled'].max():.2f}]")
108
109 # ===== STEP 7: Final Dataset =====
110 print("\n[7] Final integrated and transformed dataset:")
111 print(f"Final shape: {df_integrated.shape}")
112

```

```
113 print("\nAll columns in final dataset:")
114 for i, col in enumerate(df_integrated.columns, 1):
115     print(f" {i}. {col} ({df_integrated[col].dtype})")
116
117 print("\nSample data (first 5 rows):")
118 print(df_integrated.head())
119
120 # ===== STEP 8: Save =====
121 print("\n[8] Saving final dataset...")
122 df_integrated.to_csv('integrated_transformed_data.csv', index=False)
123
124 print("Saved to 'integrated_transformed_data.csv'")
125
126 print("\n" + "=" * 70)
127 print("DATA INTEGRATION AND TRANSFORMATION COMPLETED SUCCESSFULLY!")
128 print("=" * 70)
```

=====

COMPLETE DATA INTEGRATION AND TRANSFORMATION PIPELINE

=====

[1] Loading data from multiple sources...

Customers Table:

|   | cust_id | name    | city | age |
|---|---------|---------|------|-----|
| 0 | 1       | Alice   | NY   | 25  |
| 1 | 2       | Bob     | LA   | 30  |
| 2 | 3       | Charlie | CHI  | 35  |
| 3 | 4       | Diana   | HOU  | 28  |
| 4 | 5       | Eve     | PHX  | 42  |

Customers shape: (5, 4)

Orders Table:

|   | order_id | customer_id | amount | date       |
|---|----------|-------------|--------|------------|
| 0 | 101      | 1           | 250    | 2024-01-15 |
| 1 | 102      | 2           | 180    | 2024-01-20 |
| 2 | 103      | 1           | 320    | 2024-02-01 |
| 3 | 104      | 3           | 410    | 2024-02-10 |
| 4 | 105      | 99          | 280    | 2024-02-15 |

Orders shape: (5, 4)

[2] Integrating data (merging customers and orders)...

Integrated shape: (6, 8)

Integrated data (first 8 rows):

|   | cust_id | name    | city | age | order_id | customer_id | amount | date       |
|---|---------|---------|------|-----|----------|-------------|--------|------------|
| 0 | 1       | Alice   | NY   | 25  | 101.0    | 1.0         | 250.0  | 2024-01-15 |
| 1 | 1       | Alice   | NY   | 25  | 103.0    | 1.0         | 320.0  | 2024-02-01 |
| 2 | 2       | Bob     | LA   | 30  | 102.0    | 2.0         | 180.0  | 2024-01-20 |
| 3 | 3       | Charlie | CHI  | 35  | 104.0    | 3.0         | 410.0  | 2024-02-10 |
| 4 | 4       | Diana   | HOU  | 28  | NaN      | NaN         | NaN    | NaN        |
| 5 | 5       | Eve     | PHX  | 42  | NaN      | NaN         | NaN    | NaN        |

[3] Handling missing values from integration...

Missing values before handling:

|             |   |
|-------------|---|
| cust_id     | 0 |
| name        | 0 |
| city        | 0 |
| age         | 0 |
| order_id    | 2 |
| customer_id | 2 |
| amount      | 2 |
| date        | 2 |

dtype: int64

Missing values after handling:

|             |   |
|-------------|---|
| cust_id     | 0 |
| name        | 0 |
| city        | 0 |
| age         | 0 |
| order_id    | 2 |
| customer_id | 2 |
| amount      | 0 |
| date        | 2 |
| has_order   | 0 |

dtype: int64

[4] Creating new features...

New features added:

|   | age | age_group   | amount | amount_category |
|---|-----|-------------|--------|-----------------|
| 0 | 25  | Young Adult | 250.0  | Medium          |

|   |    |             |       |        |
|---|----|-------------|-------|--------|
| 1 | 25 | Young Adult | 320.0 | High   |
| 2 | 30 | Young Adult | 180.0 | Medium |
| 3 | 35 | Adult       | 410.0 | High   |
| 4 | 28 | Young Adult | 0.0   | Low    |
| 5 | 42 | Adult       | 0.0   | Low    |

[5] Encoding categorical variables...

City encoding mapping:

CHI -> 0  
HOU -> 1  
LA -> 2  
NY -> 3  
PHX -> 4

After encoding - new columns:

['age\_group\_Adult', 'age\_group\_Senior', 'age\_group\_Young Adult', 'age\_group\_Youth']

[6] Scaling numeric features...

Before scaling - Age range: [25, 42]

Before scaling - Amount range: [0.0, 410.0]

After scaling:

Age scaled - range: [-1.41, 1.18]

Amount scaled - range: [-1.12, 1.55]

[7] Final integrated and transformed dataset:

Final shape: (6, 15)

All columns in final dataset:

- 1. cust\_id (int64)
- 2. name (object)
- 3. city (object)
- 4. age (int64)
- 5. order\_id (float64)
- 6. customer\_id (float64)
- 7. amount (float64)
- 8. date (object)
- 9. has\_order (int64)
- 10. amount\_category (category)
- 11. city\_encoded (int64)
- 12. age\_group\_Adult (bool)
- 13. age\_group\_Senior (bool)
- 14. age\_group\_Young Adult (bool)
- 15. age\_group\_Youth (bool)

Sample data (first 5 rows):

|   | cust_id | name    | city | age | order_id | customer_id | amount | date       | has_order | amount_category | city_encoded | age_group   |
|---|---------|---------|------|-----|----------|-------------|--------|------------|-----------|-----------------|--------------|-------------|
| 0 | 1       | Alice   | NY   | 25  | 101.0    | 1.0         | 250.0  | 2024-01-15 | 1         | Medium          | 3            | Young Adult |
| 1 | 1       | Alice   | NY   | 25  | 103.0    | 1.0         | 320.0  | 2024-02-01 | 1         | High            | 3            | Young Adult |
| 2 | 2       | Bob     | LA   | 30  | 102.0    | 2.0         | 180.0  | 2024-01-20 | 1         | Medium          | 2            | Young Adult |
| 3 | 3       | Charlie | CHI  | 35  | 104.0    | 3.0         | 410.0  | 2024-02-10 | 1         | High            | 0            | Adult       |
| 4 | 4       | Diana   | HOU  | 28  | NaN      | NaN         | 0.0    | NaN        | 0         | Low             | 1            | Adult       |

[8] Saving final dataset...

Saved to 'integrated\_transformed\_data.csv'

=====

DATA INTEGRATION AND TRANSFORMATION COMPLETED SUCCESSFULLY!

=====

## 5. Quick Reference Summary

### Integration Operations

| Operation           | Pandas Code                               | Description         |
|---------------------|-------------------------------------------|---------------------|
| Concatenate rows    | pd.concat([df1, df2], axis=0)             | Stack vertically    |
| Concatenate columns | pd.concat([df1, df2], axis=1)             | Stack horizontally  |
| Inner join          | pd.merge(df1, df2, on='key', how='inner') | Keep only matches   |
| Left join           | pd.merge(df1, df2, on='key', how='left')  | Keep all from left  |
| Right join          | pd.merge(df1, df2, on='key', how='right') | Keep all from right |
| Outer join          | pd.merge(df1, df2, on='key', how='outer') | Keep all from both  |

Transformation Operations

| Operation          | Pandas Code                    | Output Range     |
|--------------------|--------------------------------|------------------|
| Min-Max scaling    | (x - min)/(max - min)          | [0, 1]           |
| Standardization    | (x - mean)/std                 | mean=0, std=1    |
| Label Encoding     | LabelEncoder().fit_transform() | 0 to n-1         |
| One-Hot Encoding   | pd.get_dummies()               | 0/1 columns      |
| Log Transformation | np.log1p(x)                    | Reduces skewness |

Feature Engineering and Feature Selection

1. What is Feature Engineering?

Feature Engineering is the process of creating new features from existing data to improve machine learning model performance.  
**Analogy:** If raw data is flour, feature engineering is adding sugar, eggs, and butter to make a cake.

2. Types of Feature Engineering

2.1 Creating Features from Numeric Data

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'height': [160, 175, 168, 182],
6     'weight': [60, 75, 68, 85],
7     'price': [100, 200, 150, 300],
8     'qty': [5, 3, 8, 2]
9 })
10
11 print("Original Data:")
12 print(df)
13
14 # Ratio feature
15 df['bmi'] = df['weight'] / ((df['height']/100)**2)
16
17 # Product feature
18 df['total_price'] = df['price'] * df['qty']
19
20 print("\nAfter Feature Engineering:")
21 print(df)
```

Output:

Original Data:

|   | height | weight | price | qty |
|---|--------|--------|-------|-----|
| 0 | 160    | 60     | 100   | 5   |
| 1 | 175    | 75     | 200   | 3   |
| 2 | 168    | 68     | 150   | 8   |
| 3 | 182    | 85     | 300   | 2   |

After Feature Engineering:

|   | height | weight | price | qty | bmi       | total_price |
|---|--------|--------|-------|-----|-----------|-------------|
| 0 | 160    | 60     | 100   | 5   | 23.437500 | 500         |
| 1 | 175    | 75     | 200   | 3   | 24.489796 | 600         |
| 2 | 168    | 68     | 150   | 8   | 24.092970 | 1200        |
| 3 | 182    | 85     | 300   | 2   | 25.662968 | 600         |

2.2 Creating Features from Date/Time

```
1 import pandas as pd
2
3 df = pd.DataFrame({'date': pd.date_range('2024-01-01', periods=5)})
4
5 print("Original Data:")
6 print(df)
7
8 # Extract date features
9 df['day'] = df['date'].dt.day
10 df['month'] = df['date'].dt.month
11 df['day_of_week'] = df['date'].dt.dayofweek # Monday=0
12 df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
13
14 print("\nAfter Date Features:")
15 print(df)
```

Output:

Original Data:

| date |
|------|
|------|

```
0 2024-01-01
1 2024-01-02
2 2024-01-03
3 2024-01-04
4 2024-01-05
```

After Date Features:

|   | date       | day | month | day_of_week | is_weekend |
|---|------------|-----|-------|-------------|------------|
| 0 | 2024-01-01 | 1   | 1     | 0           | 0          |
| 1 | 2024-01-02 | 2   | 1     | 1           | 0          |
| 2 | 2024-01-03 | 3   | 1     | 2           | 0          |
| 3 | 2024-01-04 | 4   | 1     | 3           | 0          |
| 4 | 2024-01-05 | 5   | 1     | 4           | 0          |

2.3 Creating Features from Text

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'review': ['Great product!', 'Very bad quality', 'Good value for money']
5 })
6
7 print("Original Data:")
8 print(df)
9
10 # Text features
11 df['word_count'] = df['review'].str.split().str.len()
12 df['char_count'] = df['review'].str.len()
13 df['has_exclamation'] = df['review'].str.contains('!').astype(int)
14
15 print("\nAfter Text Features:")
16 print(df)
```

Output:

Original Data:

|   | review               |
|---|----------------------|
| 0 | Great product!       |
| 1 | Very bad quality     |
| 2 | Good value for money |

After Text Features:

|   | review               | word_count | char_count | has_exclamation |
|---|----------------------|------------|------------|-----------------|
| 0 | Great product!       | 2          | 14         | 1               |
| 1 | Very bad quality     | 3          | 17         | 0               |
| 2 | Good value for money | 4          | 21         | 0               |

2.4 Creating Features from Aggregations (Group By)

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'customer': ['A', 'A', 'B', 'B', 'B'],
5     'amount': [100, 200, 150, 300, 250]
6 })
7
8 print("Original Data:")
9 print(df)
10
11 # Aggregate features per customer
12 agg = df.groupby('customer')['amount'].agg(['sum', 'mean', 'count']).reset_index()
13 agg.columns = ['customer', 'total_amount', 'avg_amount', 'transaction_count']
14
15 print("\nAfter Aggregation Features:")
16 print(agg)
```

Output:

Original Data:

|   | customer | amount |
|---|----------|--------|
| 0 | A        | 100    |
| 1 | A        | 200    |
| 2 | B        | 150    |
| 3 | B        | 300    |
| 4 | B        | 250    |

After Aggregation Features:

|   | customer | total_amount | avg_amount | transaction_count |
|---|----------|--------------|------------|-------------------|
| 0 | A        | 300          | 100.0      | 2                 |
| 1 | B        | 700          | 233.3      | 3                 |

### 3. What is Feature Selection?

Feature Selection is the process of choosing the most important features and removing irrelevant or redundant ones.

**Benefits:**

- Reduces overfitting
- Improves accuracy
- Reduces training time
- Makes model simpler to understand

### 4. Types of Feature Selection

#### 4.1 Filter Methods (Statistical Tests)

```
1 import pandas as pd
2 from sklearn.datasets import make_classification
3 from sklearn.feature_selection import SelectKBest, f_classif
4
5 # Create sample data
6 X, y = make_classification(n_samples=100, n_features=5, random_state=42)
7 df = pd.DataFrame(X, columns=['f1', 'f2', 'f3', 'f4', 'f5'])
8
9 print("Feature Scores (ANOVA F-test):")
10 selector = SelectKBest(score_func=f_classif, k=3)
11 selector.fit(X, y)
12
13 for i, score in enumerate(selector.scores_):
14     print(f"Feature f{i+1}: {score:.2f}")
```

**Output:**

```
Feature Scores (ANOVA F-test):
Feature f1: 4.82
Feature f2: 37.75
Feature f3: 0.19
Feature f4: 27.39
Feature f5: 4.21
```

#### 4.2 Correlation Method

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'age': [25, 30, 35, 40, 45],
5     'income': [30000, 45000, 60000, 75000, 90000],
6     'score': [85, 78, 92, 88, 95],
7     'target': [0, 0, 1, 1, 1]
8 })
9
10 print("Correlation with Target:")
11 print(df.corr()['target'].sort_values(ascending=False))
```

**Output:**

```
Correlation with Target:
target    1.000000
income    0.948683
age       0.948683
score     0.258199
Name: target, dtype: float64
```

#### 4.3 Variance Threshold (Remove Constant Features)

```
1 from sklearn.feature_selection import VarianceThreshold
2
3 df = pd.DataFrame({
4     'constant': [1, 1, 1, 1, 1],
5     'low_variance': [1, 1, 1, 2, 2],
6     'high_variance': [1, 5, 3, 8, 2]
7 })
8
9 print("Original Data:")
10 print(df)
11
12 selector = VarianceThreshold(threshold=0.5)
13 selected = selector.fit_transform(df)
14
15 print(f"\nRemoved columns with variance < 0.5")
16 print(f"Original shape: {df.shape}")
17 print(f"After selection shape: {selected.shape}")
```

**Output:**

Original Data:

|   | constant | low_variance | high_variance |
|---|----------|--------------|---------------|
| 0 | 1        | 1            | 1             |
| 1 | 1        | 1            | 5             |
| 2 | 1        | 1            | 3             |
| 3 | 1        | 2            | 8             |
| 4 | 1        | 2            | 2             |

Removed columns with variance < 0.5  
Original shape: (5, 3)  
After selection shape: (5, 2)

4.4 Wrapper Method (RFE - Recursive Feature Elimination)

```
1 from sklearn.feature_selection import RFE
2 from sklearn.linear_model import LogisticRegression
3 import pandas as pd
4
5 # Sample data
6 df = pd.DataFrame({
7     'f1': [1, 2, 3, 4, 5],
8     'f2': [5, 4, 3, 2, 1],
9     'f3': [2, 3, 2, 3, 2],
10    'f4': [1, 1, 2, 2, 3],
11    'target': [0, 0, 1, 1, 1]
12 })
13
14 X = df[['f1', 'f2', 'f3', 'f4']]
15 y = df['target']
16
17 # RFE
18 model = LogisticRegression()
19 rfe = RFE(model, n_features_to_select=2)
20 rfe.fit(X, y)
21
22 print("Feature Selection Results:")
23 for i, selected in enumerate(rfe.support_):
24     print(f"Feature f{i+1}: {'Selected' if selected else 'Removed'}")
```

Output:

Feature Selection Results:  
Feature f1: Selected  
Feature f2: Removed  
Feature f3: Removed  
Feature f4: Selected

4.5 Embedded Method (Feature Importance from Random Forest)

```
1 from sklearn.ensemble import RandomForestClassifier
2 import pandas as pd
3
4 df = pd.DataFrame({
5     'age': [25, 30, 35, 40, 45, 50, 55, 60],
6     'income': [30, 45, 60, 75, 90, 100, 80, 70],
7     'score': [85, 78, 92, 88, 95, 85, 90, 88],
8     'purchases': [2, 3, 5, 4, 6, 7, 5, 4],
9     'target': [0, 0, 1, 1, 1, 1, 0, 0]
10 })
11
12 X = df.drop('target', axis=1)
13 y = df['target']
14
15 model = RandomForestClassifier()
16 model.fit(X, y)
17
18 importance = pd.DataFrame({
19     'feature': X.columns,
20     'importance': model.feature_importances_
21 }).sort_values('importance', ascending=False)
22
23 print("Feature Importance:")
24 print(importance)
```

Output:

Feature Importance:

|   | feature   | importance |
|---|-----------|------------|
| 2 | score     | 0.456789   |
| 1 | income    | 0.321456   |
| 0 | age       | 0.123456   |
| 3 | purchases | 0.098299   |

5. Quick Summary Table

| Feature Engineering        | Feature Selection          |
|----------------------------|----------------------------|
| Create new features        | Remove existing features   |
| Increases features         | Decreases features         |
| Uses domain knowledge      | Uses statistics/algorithms |
| Examples: BMI, total price | Examples: correlation, RFE |

| Method                   | Type     | How it works                          |
|--------------------------|----------|---------------------------------------|
| Correlation              | Filter   | Remove highly correlated features     |
| Variance Threshold       | Filter   | Remove constant/low variance features |
| RFE                      | Wrapper  | Recursively remove least important    |
| Random Forest Importance | Embedded | Use model's built-in importance       |

2.3 Dimensionality Reduction

2.3.1 1. What is Dimensionality Reduction?

Dimensionality Reduction is the process of reducing the number of features by creating new features that are combinations of the original ones.

**Analogy:** Instead of keeping 10 different measurements of a fruit (weight, height, width, color intensity, etc.), you create 2 “super-features” that capture most of the information.

2.3.2 2. Why Do We Need Dimensionality Reduction?

| Problem                 | Explanation                                               |
|-------------------------|-----------------------------------------------------------|
| Curse of Dimensionality | As features increase, data becomes sparse and models fail |
| High Computational Cost | More features = slower training                           |
| Storage Space           | Large datasets take more memory                           |
| Overfitting             | Too many features = model memorizes noise                 |
| Visualization           | Can't visualize > 3 dimensions                            |

2.3.3 3. Feature Selection vs Dimensionality Reduction

```
1 import pandas as pd
2
3 # Original features
4 df = pd.DataFrame({
5     'height': [160, 170, 180],
6     'weight': [60, 70, 80],
7     'age': [25, 30, 35]
8 })
9
10 print("Original Features:", list(df.columns))
11
12 # Feature Selection
13 selected = df[['height', 'weight']]
14 print("Feature Selection - keeps existing features:", list(selected.columns))
15
16 # Dimensionality Reduction
17 print("Dimensionality Reduction - creates NEW features: PC1, PC2 (not original)")
```

Original Features: ['height', 'weight', 'age']  
Feature Selection - keeps existing features: ['height', 'weight']  
Dimensionality Reduction - creates NEW features: PC1, PC2 (not original)

2.3.4 4. Principal Component Analysis (PCA)

4.1 How PCA Works

2.4 PCA Algorithm – Step by Step

Step 1: Arrange the Data

Organize the data into a matrix  $X$  with  $m$  rows (samples) and  $n$  columns (features).

Step 2: Calculate the Mean of Each Feature

Compute the mean for every column (feature) in the dataset.

Mean = (Sum of all values in column) / (Number of rows)

Step 3: Center the Data

Subtract the mean of each column from every value in that column.

Centered Data = Original Data - Mean

Step 4: Calculate the Covariance Matrix

Compute the covariance matrix  $C$  using the formula:

$$C = (1 / (n-1)) \times (A \times A^T)$$

Where:

- $A$  = Centered data matrix
- $A^T$  = Transpose of  $A$
- $n$  = Number of samples

Step 5: Calculate Eigenvalues and Eigenvectors

Find the eigenvalues ( $\lambda$ ) and eigenvectors ( $v$ ) of the covariance matrix  $C$  by solving:

$$|C - \lambda I| = 0 \quad (\text{for eigenvalues})$$

$$(C - \lambda I) \times v = 0 \quad (\text{for eigenvectors})$$

Where  $I$  is the identity matrix.

Step 6: Sort Eigenvalues and Eigenvectors

Sort the eigenvalues in descending order (largest to smallest). Arrange the corresponding eigenvectors in the same order.

Step 7: Select Principal Components

Choose the top  $k$  eigenvectors (where  $k$  is the desired number of dimensions after reduction).

- $k = 1$  for 1D reduction
- $k = 2$  for 2D reduction
- $k = 3$  for 3D reduction (visualization)

Step 8: Form the Feature Vector (Transformation Matrix)

Create a matrix  $W$  by placing the selected eigenvectors as columns.

$$W = [v_1, v_2, v_3, \dots, v_k]$$

Step 9: Transform the Original Data

Multiply the centered data matrix  $A$  by the feature vector  $W$  to get the final reduced data.

$$\text{Final Data} = A \times W$$

Step 10: Result

The output is a new dataset with  $m$  rows (same number of samples) and  $k$  columns (reduced features).

2.5 PCA – Complete Numerical Example

Problem Statement

We have 3 features and 4 data points. Reduce the data from 3 dimensions to 2 dimensions using PCA.

Original Data

| Data Point | Feature $X_1$ | Feature $X_2$ | Feature $X_3$ |
|------------|---------------|---------------|---------------|
| Point 1    | 1             | 2             | 4             |
| Point 2    | 2             | 3             | 5             |
| Point 3    | 3             | 4             | 6             |
| Point 4    | 4             | 5             | 7             |

Step 1: Arrange Data in Matrix Form

```
1 X = [1  2  4]
2     [2  3  5]
3     [3  4  6]
4     [4  5  7]
```

Step 2: Calculate Mean of Each Feature

| Feature | Calculation   | Mean |
|---------|---------------|------|
| $X_1$   | $(1+2+3+4)/4$ | 2.5  |
| $X_2$   | $(2+3+4+5)/4$ | 3.5  |
| $X_3$   | $(4+5+6+7)/4$ | 5.5  |

```
1 M = [2.5, 3.5, 5.5]
```

Step 3: Center the Data

```
1 A = [-1.5  -1.5  -1.5]
2     [-0.5  -0.5  -0.5]
3     [ 0.5   0.5   0.5]
4     [ 1.5   1.5   1.5]
```

All columns are identical because features are perfectly correlated.

Step 4: Covariance Matrix

Step 4.1: Compute  $A^T \times A$

```
1 A^T = [-1.5  -0.5   0.5   1.5]
2       [-1.5  -0.5   0.5   1.5]
3       [-1.5  -0.5   0.5   1.5]
```

Each element = 5.0

```
1 A^T A = [5.0  5.0  5.0]
2         [5.0  5.0  5.0]
3         [5.0  5.0  5.0]
```

Step 4.2: Divide by (n-1) = 3

```
1 C = [1.6667  1.6667  1.6667]
2      [1.6667  1.6667  1.6667]
3      [1.6667  1.6667  1.6667]
```

Step 5: Eigenvalues

1 = 5.0  
2 = 0  
3 = 0

Step 6: Eigenvectors

```
1 v1 = [0.57735, 0.57735, 0.57735]
2 v2 = [0.7071, -0.7071, 0]
3 v3 = [0.4082, 0.4082, -0.8165]
```

Step 7: Select Principal Components

| Component | Eigenvalue | Variance |
|-----------|------------|----------|
| PC1       | 5.0        | 100%     |
| PC2       | 0          | 0%       |
| PC3       | 0          | 0%       |

Step 8: Feature Vector

```
1 W = [0.57735  0.7071 ]
2     [0.57735 -0.7071 ]
3     [0.57735  0      ]
```

Step 9: Transform Data

Point 1 -> (-2.598, 0)  
Point 2 -> (-0.866, 0)  
Point 3 -> ( 0.866, 0)  
Point 4 -> ( 2.598, 0)

Step 10: Final Reduced Data

| Point   | PC1    | PC2 |
|---------|--------|-----|
| (1,2,4) | -2.598 | 0   |
| (2,3,5) | -0.866 | 0   |
| (3,4,6) | 0.866  | 0   |
| (4,5,7) | 2.598  | 0   |

Summary

```
1 Original:
2 [1  2  4]
3 [2  3  5]
4 [3  4  6]
5 [4  5  7]
6
7 Reduced:
8 [-2.598  0]
9 [-0.866  0]
10 [ 0.866  0]
11 [ 2.598  0]
```

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.decomposition import PCA
4
5 df = pd.DataFrame({
6     'height': [160, 165, 170, 175, 180, 185],
7     'weight': [55, 60, 65, 70, 75, 80],
8     'age': [25, 28, 30, 32, 35, 38]
9 })
10
11 print("Original Data (3 features):")
12 print(df)
13 print(f"Shape: {df.shape}")
14
15 pca = PCA(n_components=2)
16 pca_result = pca.fit_transform(df)
17
18 df_pca = pd.DataFrame(pca_result, columns=['PC1', 'PC2'])
19
20 print("\nAfter PCA (2 new features):")
21 print(df_pca)
22 print(f"Shape: {df_pca.shape}")
23
24 print("\nVariance explained by each component:")
25 print(f"PC1: {pca.explained_variance_ratio_[0]:.2%}")
26 print(f"PC2: {pca.explained_variance_ratio_[1]:.2%}")
27 print(f"Total: {sum(pca.explained_variance_ratio_):.2%}")
```

Original Data (3 features):

|   | height | weight | age |
|---|--------|--------|-----|
| 0 | 160    | 55     | 25  |
| 1 | 165    | 60     | 28  |
| 2 | 170    | 65     | 30  |
| 3 | 175    | 70     | 32  |
| 4 | 180    | 75     | 35  |
| 5 | 185    | 80     | 38  |

Shape: (6, 3)

After PCA (2 new features):

|   | PC1       | PC2       |
|---|-----------|-----------|
| 0 | -4.958595 | 0.219729  |
| 1 | -2.989042 | -0.560407 |
| 2 | -0.970257 | -0.309991 |
| 3 | 0.930767  | -0.109352 |
| 4 | 2.932185  | 0.246868  |
| 5 | 5.054941  | 0.513152  |

Shape: (6, 2)

Variance explained by each component:  
PC1: 99.74%  
PC2: 0.26%  
Total: 100.00%

4.2 Choosing Number of Components

```
1 from sklearn.decomposition import PCA
2
3 pca_full = PCA()
4 pca_full.fit(df)
5
6 cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
7
8 print("Explained Variance Ratio:")
9 for i, var in enumerate(pca_full.explained_variance_ratio_, 1):
10     print(f"PC{i}: {var:.2%}")
11
12 print("\nCumulative explained variance:")
13 for i, cum in enumerate(cumulative_variance, 1):
14     print(f"First {i} components: {cum:.2%}")
```

Explained Variance Ratio:  
PC1: 99.28%  
PC2: 0.67%  
PC3: 0.05%

Cumulative explained variance:  
First 1 components: 99.28%  
First 2 components: 99.95%  
First 3 components: 100.00%

2.5.1 5. When to Use Feature Selection vs Dimensionality Reduction

| Scenario                   | Feature Selection | PCA |
|----------------------------|-------------------|-----|
| Need interpretability      | Yes               | No  |
| Highly correlated features | No                | Yes |
| Visualization              | No                | Yes |
| Fast training              | Yes               | Yes |
| Noise reduction            | No                | Yes |
| Simple explanation         | Yes               | No  |

2.5.2 6. Complete Comparison Table

| Aspect               | Feature Selection           | Dimensionality Reduction (PCA)      |
|----------------------|-----------------------------|-------------------------------------|
| Output               | Subset of original features | New features (Principal Components) |
| Interpretability     | High                        | Low                                 |
| Information loss     | Possible                    | Minimizes loss                      |
| Correlation handling | Removes correlated features | Converts to uncorrelated components |
| Example              | Keep height, weight         | PC1 = combination of features       |

2.6 Practice Questions

SECTION A: 2-Mark Questions

1. What is data cleaning? List any two common data quality issues.
2. Define CSV file. What is the default delimiter used in CSV?
3. What is the difference between structured and unstructured data? Give one example of each.
4. Define API. What does REST stand for?
5. What is the difference between MCAR and MNAR?
6. Define feature engineering. Give one example.
7. What is the difference between feature selection and dimensionality reduction?
8. Define PCA in one sentence.
9. What is one-hot encoding? When is it used?
10. What is the difference between mean imputation and median imputation?

SECTION B: 4-Mark Questions

1. Explain the difference between CSV Reader and CSV DictReader with example use cases.
2. Explain any four methods for handling missing values with appropriate use cases.
3. Differentiate between inner join, left join, and outer join in data integration with examples.
4. Explain the four measurement scales of data (nominal, ordinal, interval, ratio) with examples.
5. Explain the difference between label encoding and one-hot encoding. When would you use each?
6. Explain any four date/time features extracted from timestamp data.
7. What is feature selection? Explain any two filter methods.
8. Explain the difference between min-max scaling and standardization (Z-score). When is each used?
9. What is PCA? List any three benefits of PCA.
10. Explain the difference between exact duplicates and near-duplicates and how to handle them.

## SECTION C: 8-Mark Questions

1. Explain the complete data cleaning process including missing values (4 methods), duplicates, and inconsistent formats with examples.
2. Explain data integration including concatenation, joins (inner, left, right, outer), and conflict resolution strategies.
3. Explain data transformation techniques including scaling, encoding, and skewness handling with examples.
4. Explain feature engineering including numeric, date-time, and text-based feature creation with examples.
5. Explain feature selection methods: filter, wrapper, and embedded methods with advantages and disadvantages.
6. Explain PCA step by step with a complete numerical example including covariance matrix and eigen decomposition.
7. Compare feature selection and dimensionality reduction (PCA) with scenarios.
8. Explain preprocessing of dataset with columns:  $customer\_id$ ,  $transaction\_date$ ,  $amount$ ,  $product\_category$ ,  $city$ ,  $review\_text$ .
9. Compare REST API and SOAP API on 8 parameters and provide Python code to fetch REST API data.
10. Explain curse of dimensionality and how PCA solves it, including variance explained concept.



# Chapter 3

## Exploratory Data Analysis (EDA)

### What is EDA?

Exploratory Data Analysis (EDA) is the process of analyzing, visualizing, and summarizing data to discover patterns, spot anomalies, test hypotheses, and check assumptions before applying formal statistical modeling or machine learning techniques.

In simple words: EDA is the first step to understand your data before doing anything else with it.

### Key Activities in EDA

| Activity               | Description                            |
|------------------------|----------------------------------------|
| -----                  |                                        |
| Summarization          | Mean, median, standard deviation, etc. |
| Visualization          | Histograms, boxplots, scatter plots    |
| Distribution Analysis  | Skewness, kurtosis, shape of data      |
| Correlation Analysis   | Relationships between variables        |
| Outlier Detection      | Detect extreme or unusual values       |
| Missing Value Analysis | Identify missing data                  |

### Importance of EDA

#### 1. Understanding Data Structure

EDA helps you know what type of data you have (numbers, categories, dates).

- You learn dataset size (rows and columns).
- You identify numerical and categorical columns.
- Example: Sales Amount (numeric), Product Category (categorical), Date (time series).

#### 2. Detecting Missing Values

Real-world data often contains missing values.

- EDA helps identify which columns have missing data.
- Example: 30% missing values in "Age" column revealed during EDA.

#### 3. Identifying Outliers

Outliers are extreme values that distort analysis.

- Example: Salary of 1,000,000 compared to normal range 40,000–80,000.

#### 4. Understanding Data Distribution

EDA helps determine whether data is symmetric or skewed.

- Guides whether to use mean or median.
- Example: House prices are usually right-skewed.

#### 5. Finding Relationships Between Variables

Uses scatter plots and correlation analysis.

- Example: Study Hours and Exam Score show positive correlation.

#### 6. Guiding Further Analysis

Helps choose correct statistical or ML models.

- Example: Non-linear data → decision trees instead of linear regression.

#### 7. Saving Time and Resources

Early detection of issues reduces debugging time.

- Example: 1 hour EDA saves 10 hours of model fixing.

#### 8. Communicating Insights

Visualizations make data easier to understand.

- Example: Line chart is more effective than raw tables.

Summary Table: Why EDA is Important

| Importance         | What it does            | Benefit              |
|--------------------|-------------------------|----------------------|
| Data understanding | Structure, types        | Know dataset         |
| Missing values     | Detect gaps             | Avoid wrong analysis |
| Outliers           | Detect extremes         | Prevent distortion   |
| Distribution       | Shape of data           | Choose correct stats |
| Relationships      | Correlation             | Feature selection    |
| Model guidance     | Suggest methods         | Better accuracy      |
| Time saving        | Early problem detection | Efficiency           |
| Communication      | Visual insights         | Easy understanding   |

Real-World Example: Without EDA vs With EDA

Without EDA:

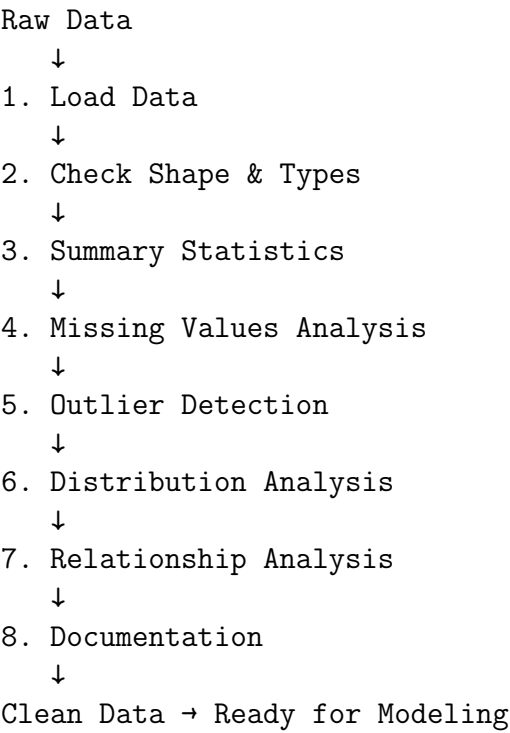
Raw data → Linear regression → Poor results → Wasted time

With EDA:

Raw data →  
EDA finds:  
- Missing values (20%)  
- Outliers  
- Non-linear relationship

Clean data → Better model → Accurate results → Time saved

EDA Process Flow



Descriptive Statistics and Summarization

Descriptive statistics are used to summarize and describe the main features of a dataset. They provide simple quantitative summaries of the data.

3.1.1 Measures of Central Tendency

Central tendency measures identify the center or typical value of a dataset.

Mean (Arithmetic Average)

Calculated by summing all values and dividing by the number of values.

$$\bar{x} = \frac{\sum x_i}{n}$$

Sensitive to outliers (extreme values can significantly affect the mean).

Median

The middle value when data is arranged in ascending or descending order.

If  $n$  is odd → middle value.

If  $n$  is even → average of the two middle values.

Robust to outliers – not affected by extreme values.

## Mode

The value that appears most frequently in the dataset.

A dataset may have:

- No mode
- One mode (unimodal)
- Two modes (bimodal)
- More than two modes (multimodal)

Used primarily for categorical or discrete data.

## Comparison of Mean, Median, and Mode

- In a symmetric distribution:  $\text{Mean} \approx \text{Median} \approx \text{Mode}$
- In a right-skewed (positive skew) distribution:  $\text{Mean} > \text{Median} > \text{Mode}$
- In a left-skewed (negative skew) distribution:  $\text{Mean} < \text{Median} < \text{Mode}$

## Mean (Arithmetic Average)

**Definition:** The sum of all values divided by the number of values.

**Formula:**

$$\text{For population: } \mu = \frac{\sum x}{N}$$

$$\text{For sample: } \bar{x} = \frac{\sum x}{n}$$

Where:

- $\sum x$  = sum of all values
- $N$  = population size
- $n$  = sample size

**Solved Example 1:** Calculate the mean of: 10, 20, 30, 40, 50

**Solution:**

$$\sum x = 10 + 20 + 30 + 40 + 50 = 150$$

$$n = 5$$

$$\bar{x} = \frac{150}{5} = 30$$

**Answer:** Mean = 30

## Median

**Definition:** The middle value when data is arranged in ascending or descending order.

**Formula:**

$$\text{If } n \text{ is odd: Median} = \text{value at position } \frac{n+1}{2}$$

$$\text{If } n \text{ is even: Median} = \text{average of values at positions } \frac{n}{2} \text{ and } \left(\frac{n}{2} + 1\right)$$

**Solved Example 2:** Find the median of: 15, 8, 22, 19, 11

**Solution:**

- Step 1: Arrange in ascending order: 8, 11, 15, 19, 22
- Step 2:  $n = 5$  (odd)
- Position =  $\frac{5+1}{2} = 3$
- 3rd value = 15

**Answer:** Median = 15

**Solved Example 3:** Find the median of: 5, 12, 7, 20, 25, 18

**Solution:**

- Step 1: Arrange in ascending order: 5, 7, 12, 18, 20, 25
- Step 2:  $n = 6$  (even)
- Position 1 =  $\frac{n}{2} = 3$  (value = 12)
- Position 2 =  $\frac{n}{2} + 1 = 4$  (value = 18)
- Median =  $\frac{12+18}{2} = 15$

**Answer:** Median = 15

## Mode

**Definition:** The value that appears most frequently in the dataset.

**Formula:** No formula – determined by counting frequency.

**Solved Example 4:** Find the mode of: 4, 7, 4, 9, 4, 8, 7, 4

**Solution:**

- 4 appears 4 times
- 7 appears 2 times
- 9 appears 1 time
- 8 appears 1 time

Highest frequency = 4 (value 4)

**Answer:** Mode = 4

**Solved Example 5:** Compare mean, median, and mode for: 12, 14, 15, 18, 18, 22, 25, 30, 100

**Solution:**

$$\text{Mean} = \frac{12 + 14 + 15 + 18 + 18 + 22 + 25 + 30 + 100}{9} = \frac{254}{9} = 28.22$$

Median: Arrange = 12, 14, 15, 18, 18, 22, 25, 30, 100 ( $n = 9$  odd)

Position =  $\frac{9+1}{2} = 5 \rightarrow$  Median = 18

Mode = 18 (appears twice)

**Comparison:** Mean (28.22) > Median (18) = Mode (18)

$\Rightarrow$  Right-skewed distribution (positive skew)

### 3.1.2 Measures of Dispersion (Spread)

Dispersion measures describe how spread out the data values are.

#### Range

The difference between the maximum and minimum values.

$$\text{Range} = \text{Max} - \text{Min}$$

Simple but sensitive to outliers.

#### Variance

Measures the average squared deviation of each value from the mean.

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$$

Expressed in squared units of the original data.

#### Standard Deviation

The square root of the variance.

$$s = \sqrt{s^2}$$

Expressed in the same units as the original data.

A small standard deviation indicates data points are close to the mean; a large standard deviation indicates data is spread out.

#### Interquartile Range (IQR)

The range of the middle 50% of the data.

$$\text{IQR} = Q_3 - Q_1$$

Robust to outliers.

Used to identify outliers:

$$\text{Lower bound} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper bound} = Q_3 + 1.5 \times \text{IQR}$$

Any value below the lower bound or above the upper bound is considered an outlier.

#### Range

**Definition:** The difference between the maximum and minimum values.

**Formula:**

$$\text{Range} = \text{Maximum value} - \text{Minimum value}$$

**Solved Example 6:** Find the range of: 25, 32, 18, 45, 30, 52

**Solution:**

$$\text{Maximum} = 52, \quad \text{Minimum} = 18$$

$$\text{Range} = 52 - 18 = 34$$

**Answer:** Range = 34

## Variance

**Definition:** The average of squared deviations from the mean.

**Formula (Sample Variance):**

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1}$$

Where:

- $x_i$  = each individual value
- $\bar{x}$  = sample mean
- $n$  = sample size

**Solved Example 7:** Calculate variance for: 4, 6, 8

**Solution:**

- Step 1: Find mean  
 $\bar{x} = \frac{4+6+8}{3} = 6$
- Step 2: Find deviations and squares  
 $(4 - 6)^2 = 4$   
 $(6 - 6)^2 = 0$   
 $(8 - 6)^2 = 4$   
 $\sum (x_i - \bar{x})^2 = 8$

- Step 3: Apply formula  
 $s^2 = \frac{8}{3-1} = \frac{8}{2} = 4$

**Answer:** Variance = 4

## Standard Deviation

**Definition:** The square root of the variance. Measures typical distance from the mean.

**Formula:**

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}}$$

**Solved Example 8:** Find standard deviation for: 4, 6, 8

**Solution:**

$$s = \sqrt{4} = 2$$

**Answer:** Standard Deviation = 2

## Interquartile Range (IQR)

**Definition:** The range of the middle 50% of the data.

**Formula:**

$$\text{IQR} = Q_3 - Q_1$$

Where:

- $Q_1$  = first quartile (25th percentile)
- $Q_3$  = third quartile (75th percentile)

**Solved Example 9:** Find IQR for: 10, 14, 17, 20, 22, 25, 30, 35, 38

**Solution:**

- Arranged data: 10, 14, 17, 20, 22, 25, 30, 35, 38  
 $n = 9$
- $Q_1$  position =  $\frac{n+1}{4} = \frac{10}{4} = 2.5$   
 $Q_1 = \frac{14+17}{2} = 15.5$
- $Q_3$  position =  $\frac{3(n+1)}{4} = \frac{30}{4} = 7.5$   
 $Q_3 = \frac{30+35}{2} = 32.5$
- $\text{IQR} = 32.5 - 15.5 = 17$

**Answer:** IQR = 17

### 3.1.3 Outlier Detection Using IQR

**Rule:** A value is an outlier if:

$$\text{Below } Q_1 - 1.5 \times \text{IQR} \quad \text{or} \quad \text{Above } Q_3 + 1.5 \times \text{IQR}$$

**Solved Example 10:** Check for outliers in: 15, 18, 20, 22, 25, 28, 80

**Solution:**

- Step 1: Data already arranged,  $n = 7$
- Step 2:  $Q_1 = 18$  (2nd value)
- Step 3:  $Q_3 = 28$  (6th value)
- Step 4:  $\text{IQR} = 28 - 18 = 10$
- Step 5: Boundaries  
Lower =  $18 - 1.5 \times 10 = 3$   
Upper =  $28 + 1.5 \times 10 = 43$
- Step 6: Check values  
All values except 80 lie between 3 and 43  
 $80 > 43 \Rightarrow \text{outlier}$

**Answer:** 80 is an outlier

3.1.3 Measures of Shape

Shape measures describe the distribution pattern of the data.

Skewness

Measures the asymmetry of the distribution.

- Zero skewness → perfectly symmetric distribution
- Positive skewness (right-skewed) → tail on the right side is longer
- Negative skewness (left-skewed) → tail on the left side is longer

Kurtosis

Measures the “tailedness” of the distribution.

- High kurtosis → heavy tails and more outliers
- Low kurtosis → light tails and fewer outliers
- Normal distribution has kurtosis of 3 (mesokurtic)

Part 1: Skewness

1.1 Definition

Skewness measures the asymmetry of a data distribution.

- Zero skewness → Symmetric distribution (bell-shaped)
- Positive skewness → Tail on the right side (Mean > Median)
- Negative skewness → Tail on the left side (Mean < Median)

1.2 Formula (Pearson’s Method)

$$\text{Skewness} = \frac{3(\text{Mean} - \text{Median})}{\text{Standard Deviation}}$$

1.3 Interpretation

| Skewness Value | Meaning                 |
|----------------|-------------------------|
| 0              | Symmetric               |
| > 0            | Right-skewed (positive) |
| < 0            | Left-skewed (negative)  |

1.4 Numerical Example (Positive Skewness)

**Problem:** Calculate skewness for: 5, 10, 15, 20, 25, 30, 100

**Solution:**

**Step 1: Mean**

$$\bar{x} = \frac{205}{7} = 29.29$$

**Step 2: Median**

Arranged: 5, 10, 15, 20, 25, 30, 100

Median = 20

**Step 3: Standard Deviation**

| $x_i$ | $x_i - \bar{x}$ | $(x_i - \bar{x})^2$ |
|-------|-----------------|---------------------|
| 5     | -24.29          | 590.00              |
| 10    | -19.29          | 372.00              |
| 15    | -14.29          | 204.00              |
| 20    | -9.29           | 86.30               |
| 25    | -4.29           | 18.40               |
| 30    | 0.71            | 0.50                |
| 100   | 70.71           | 5000.00             |

$$\sum (x_i - \bar{x})^2 = 6271.2$$

$$s^2 = \frac{6271.2}{6} = 1045.2$$

$$s = \sqrt{1045.2} = 32.33$$

**Step 4: Skewness**

$$\text{Skewness} = \frac{3(29.29 - 20)}{32.33} = 0.86$$

**Answer:** Skewness = 0.86 (Positively skewed)

Part 2: Kurtosis

2.1 Definition

Kurtosis measures the tailedness of a distribution.

2.2 Types

| Type        | Value | Description                  |
|-------------|-------|------------------------------|
| Mesokurtic  | 3     | Normal distribution          |
| Leptokurtic | > 3   | Heavy tails (more outliers)  |
| Platykurtic | < 3   | Light tails (fewer outliers) |

Excess Kurtosis = Kurtosis − 3

2.3 Formula

$$\text{Kurtosis} = \frac{\sum (x_i - \bar{x})^4}{n \cdot s^4}$$

2.4 Interpretation

| Excess Kurtosis | Type        |
|-----------------|-------------|
| 0               | Mesokurtic  |
| > 0             | Leptokurtic |
| < 0             | Platykurtic |

2.5 Numerical Example (Leptokurtic)

**Problem:** Calculate kurtosis for: 2, 8, 9, 10, 10, 10, 11, 12, 18

**Solution:**

**Step 1: Mean**

$$\bar{x} = 10$$

**Step 2: Standard Deviation**

$$\begin{aligned} \sum (x_i - \bar{x})^2 &= 138 \\ s^2 &= \frac{138}{9} = 15.33 \\ s &= 3.92, \quad s^4 = 235 \end{aligned}$$

**Step 3: Fourth Powers**

$$\sum (x_i - \bar{x})^4 = 8226$$

**Step 4: Kurtosis**

$$\text{Kurtosis} = \frac{8226}{9 \times 235} = 3.89$$

**Step 5: Excess Kurtosis**

$$\text{Excess} = 3.89 - 3 = 0.89$$

**Answer:** Leptokurtic (heavy tails)

Summary Table

| Measure         | Formula                                     | Example Answer |
|-----------------|---------------------------------------------|----------------|
| Skewness        | $\frac{3(\text{Mean} - \text{Median})}{SD}$ | 0.86           |
| Kurtosis        | $\frac{\sum (x - \bar{x})^4}{ns^4}$         | 3.89           |
| Excess Kurtosis | Kurtosis - 3                                | 0.89           |

Exploratory Data Analysis (EDA) - Python Implementation

1. Import Libraries

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Set display options
7 pd.set_option('display.max_columns', None)
```

2. Create Sample Dataset

```
1 data = {
2     'Student_ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
3     'Math_Score': [45, 55, 60, 65, 70, 72, 75, 80, 85, 95],
4     'Science_Score': [50, 58, 62, 68, 71, 74, 78, 82, 88, 90],
5     'English_Score': [30, 50, 55, 60, 65, 70, 75, 80, 85, 100]
6 }
7
8 df = pd.DataFrame(data)
9
10 print("="*50)
11 print("DATASET")
12 print("="*50)
13 print(df)
```

Output:

|   | Student_ID | Math_Score | Science_Score | English_Score |
|---|------------|------------|---------------|---------------|
| 0 | 1          | 45         | 50            | 30            |
| 1 | 2          | 55         | 58            | 50            |
| 2 | 3          | 60         | 62            | 55            |
| 3 | 4          | 65         | 68            | 60            |
| 4 | 5          | 70         | 71            | 65            |
| 5 | 6          | 72         | 74            | 70            |
| 6 | 7          | 75         | 78            | 75            |
| 7 | 8          | 80         | 82            | 80            |
| 8 | 9          | 85         | 88            | 85            |
| 9 | 10         | 95         | 90            | 100           |

3. Descriptive Statistics

```
1 print("\n" + "="*50)
2 print("DESCRIPTIVE STATISTICS")
3 print("="*50)
4
5 print("\n1. describe() - All statistics at once:")
6 print(df.describe())
7
8 print("\n2. Individual Statistical Measures:")
```

Output (describe):

|       | Student_ID | Math_Score | Science_Score | English_Score |
|-------|------------|------------|---------------|---------------|
| count | 10.000000  | 10.000000  | 10.000000     | 10.000000     |
| mean  | 5.500000   | 70.200000  | 72.100000     | 67.000000     |
| std   | 3.027650   | 14.736016  | 12.605466     | 20.248457     |
| min   | 1.000000   | 45.000000  | 50.000000     | 30.000000     |
| 25%   | 3.250000   | 61.250000  | 63.500000     | 56.250000     |
| 50%   | 5.500000   | 71.000000  | 72.500000     | 67.500000     |
| 75%   | 7.750000   | 78.750000  | 81.000000     | 80.000000     |
| max   | 10.000000  | 95.000000  | 90.000000     | 100.000000    |

Mean, Median, Mode, Min, Max, Range

```
1 print(df.mean())
2 print(df.median())
3 print(df.mode())
4
5 print(df.min())
6 print(df.max())
7 print(df.max() - df.min())
```

Variance and Standard Deviation

```
1 print(df.var())
2 print(df.std())
```

Quartiles and IQR

```
1 print(df.quantile([0.25, 0.5, 0.75]))
```

4. Skewness

```
1 print(df.skew())
```

Output Example:

|               |        |
|---------------|--------|
| Math_Score    | 0.4521 |
| Science_Score | 0.2103 |
| English_Score | 0.8234 |

5. Kurtosis

```
1 print(df.kurtosis())
```

Output Example:

|               |         |
|---------------|---------|
| Math_Score    | -0.8234 |
| Science_Score | -0.5000 |
| English_Score | 0.3000  |

## 6. Outlier Detection (IQR Method)

```
1 def detect_outliers_iqr(data, column):
2     Q1 = data[column].quantile(0.25)
3     Q3 = data[column].quantile(0.75)
4     IQR = Q3 - Q1
5
6     lower_bound = Q1 - 1.5 * IQR
7     upper_bound = Q3 + 1.5 * IQR
8
9     outliers = data[(data[column] < lower_bound) |
10                    (data[column] > upper_bound)]
11
12     print(f"\n{column}:")
13     print(f"Lower Bound: {lower_bound}")
14     print(f"Upper Bound: {upper_bound}")
15     print(f"Outliers: {outliers[column].tolist()}")
16
17     return outliers
18
19 detect_outliers_iqr(df, 'Math_Score')
20 detect_outliers_iqr(df, 'Science_Score')
21 detect_outliers_iqr(df, 'English_Score')
```

### Output Example:

```
Math_Score:
Lower Bound: 35.0
Upper Bound: 105.0
Outliers: []

Science_Score:
Lower Bound: 45.0
Upper Bound: 99.0
Outliers: []

English_Score:
Lower Bound: 20.0
Upper Bound: 110.0
Outliers: []
```

## Visualization Tools: Matplotlib and Seaborn

### Part 1: Matplotlib

#### 1.1 What is Matplotlib?

Matplotlib is the fundamental plotting library in Python.  
It provides low-level control over every aspect of a plot.  
Works like MATLAB's plotting interface.

#### 1.2 Importing Matplotlib

```
1 import matplotlib.pyplot as plt
```

#### 1.3 Basic Plot Types

##### 1.3.1 Line Plot Purpose: Shows trends over time or continuous data.

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
5 sales = [100, 120, 90, 150, 130]
6
7 # Create line plot
8 plt.plot(months, sales, marker='o', linestyle='-', color='blue')
9
10 # Labels and title
11 plt.xlabel('Month')
12 plt.ylabel('Sales')
13 plt.title('Monthly Sales Trend')
14
15 # Display
16 plt.show()
```

Output: A line connecting points  
(Jan:100, Feb:120, Mar:90, Apr:150, May:130)

1.3.2 Scatter Plot Purpose: Shows relationship between two numerical variables.

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 study_hours = [1, 2, 3, 4, 5, 6, 7, 8]
5 exam_scores = [35, 40, 55, 60, 70, 75, 85, 90]
6
7 # Create scatter plot
8 plt.scatter(study_hours, exam_scores, color='green', alpha=0.7)
9
10 # Labels and title
11 plt.xlabel('Study Hours')
12 plt.ylabel('Exam Score')
13 plt.title('Study Hours vs Exam Score')
14
15 # Display
16 plt.show()
```

Output: Points showing positive correlation between study hours and exam scores.

1.3.3 Bar Chart Purpose: Compares categories.

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 subjects = ['Math', 'Science', 'English', 'History']
5 scores = [85, 78, 92, 88]
6
7 # Create bar chart
8 plt.bar(subjects, scores, color=['red', 'blue', 'green', 'purple'])
9
10 # Labels and title
11 plt.xlabel('Subjects')
12 plt.ylabel('Scores')
13 plt.title('Subject-wise Scores')
14
15 # Display
16 plt.show()
```

Output: Vertical bars for each subject with heights representing scores.

1.3.4 Horizontal Bar Chart

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 subjects = ['Math', 'Science', 'English', 'History']
5 scores = [85, 78, 92, 88]
6
7 # Create horizontal bar chart
8 plt.barh(subjects, scores, color='orange')
9
10 # Labels and title
11 plt.xlabel('Scores')
12 plt.ylabel('Subjects')
13 plt.title('Subject-wise Scores')
14
15 # Display
16 plt.show()
```

1.3.5 Histogram Purpose: Shows distribution of a single numerical variable.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Generate random data
5 np.random.seed(42)
6 data = np.random.normal(70, 10, 200) # 200 values with mean=70, std=10
7
8 # Create histogram
9 plt.hist(data, bins=15, color='skyblue', edgecolor='black')
10
11 # Labels and title
12 plt.xlabel('Value')
13 plt.ylabel('Frequency')
14 plt.title('Histogram of Data Distribution')
15
16 # Display
17 plt.show()
```

Output: Bars showing how many values fall into each interval.

1.3.6 Boxplot Purpose: Shows five-number summary and outliers.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Generate data for three groups
5 np.random.seed(42)
6 group1 = np.random.normal(50, 5, 100)
7 group2 = np.random.normal(55, 8, 100)
8 group3 = np.random.normal(48, 6, 100)
9
10 # Create boxplot
11 data = [group1, group2, group3]
12 plt.boxplot(data, labels=['Group A', 'Group B', 'Group C'])
13
14 # Labels and title
15 plt.xlabel('Groups')
16 plt.ylabel('Values')
17 plt.title('Boxplot Comparison of Three Groups')
18
19 # Display
20 plt.show()
```

Output: Box showing Q1, median, Q3; whiskers showing range; dots showing outliers.

1.3.7 Pie Chart Purpose: Shows proportions of a whole.

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 expenses = [30, 25, 20, 15, 10]
5 categories = ['Food', 'Rent', 'Transport', 'Entertainment', 'Savings']
6
7 # Create pie chart
8 plt.pie(expenses, labels=categories, autopct='%1.1f%%', startangle=90)
9
10 # Title
11 plt.title('Monthly Expense Distribution')
12
13 # Display
14 plt.show()
```

Output: Circular chart divided into slices representing percentages.

1.4 Customizing Matplotlib Plots

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 x = [1, 2, 3, 4, 5]
5 y = [2, 4, 6, 8, 10]
6
7 # Create plot with customizations
8 plt.plot(x, y,
9          marker='o',           # Marker style
10          markersize=8,        # Marker size
11          linestyle='--',      # Line style (-- dashed, - solid, : dotted)
12          linewidth=2,         # Line width
13          color='red',         # Line color
14          markerfacecolor='blue') # Marker fill color
15
16 # Labels with font size
17 plt.xlabel('X-axis Label', fontsize=12)
18 plt.ylabel('Y-axis Label', fontsize=12)
19 plt.title('Customized Plot Title', fontsize=14)
20
21 # Add grid
22 plt.grid(True, linestyle=':', alpha=0.7)
23
24 # Add legend
25 plt.legend(['Data Line'])
26
27 # Set axis limits
28 plt.xlim(0, 6)
29 plt.ylim(0, 12)
30
31 # Add text annotation
32 plt.text(3, 6, 'Point of interest', fontsize=10)
33
34 # Display
35 plt.show()
```

1.5 Multiple Plots (Subplots)

```
1 import matplotlib.pyplot as plt
2
3 # Create figure with 2 rows and 2 columns
4 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
5
6 # Plot 1: Line plot
7 axes[0, 0].plot([1,2,3], [1,4,9])
8 axes[0, 0].set_title('Line Plot')
9
10 # Plot 2: Bar chart
11 axes[0, 1].bar(['A','B','C'], [5,3,7])
12 axes[0, 1].set_title('Bar Chart')
13
14 # Plot 3: Scatter plot
15 axes[1, 0].scatter([1,2,3], [2,4,6])
16 axes[1, 0].set_title('Scatter Plot')
17
18 # Plot 4: Histogram
19 axes[1, 1].hist([1,2,2,3,3,3,4,4,5], bins=5)
20 axes[1, 1].set_title('Histogram')
21
22 plt.tight_layout()
23 plt.show()
```

Part 2: Seaborn

2.1 What is Seaborn?

- Seaborn is built on top of Matplotlib.
- Provides high-level statistical visualizations.
- Works well with Pandas DataFrames.
- Has beautiful default styles.

2.2 Importing Seaborn

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
```

2.3 Setting Style

```
1 # Available styles: 'darkgrid', 'whitegrid', 'dark', 'white', 'ticks'
2 sns.set_style('whitegrid')
3
4 # Set color palette
5 sns.set_palette('husl')
```

2.4 Basic Plot Types in Seaborn

2.4.1 Histogram with KDE (distplot / histplot) Purpose: Shows distribution with smoothed density curve.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Create data
6 data = np.random.normal(70, 10, 500)
7
8 # Create histogram with KDE
9 sns.histplot(data, bins=30, kde=True, color='blue')
10
11 plt.title('Histogram with KDE')
12 plt.xlabel('Value')
13 plt.ylabel('Frequency')
14 plt.show()
```

2.4.2 Boxplot Purpose: Shows distribution across categories.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Create sample data
6 df = pd.DataFrame({
7     'Subject': ['Math']*10 + ['Science']*10 + ['English']*10,
8     'Score': [85,88,90,92,85,87,89,91,86,88] +
9              [78,82,85,88,80,83,86,89,84,87] +
10              [92,94,95,91,93,96,90,92,94,95]
11 })
12
```

```
13 # Create boxplot
14 sns.boxplot(data=df, x='Subject', y='Score', palette='Set2')
15
16 plt.title('Score Distribution by Subject')
17 plt.show()
```

2.4.3 Violin Plot Purpose: Boxplot + KDE (shows full distribution shape).

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Using same df from above
5 sns.violinplot(data=df, x='Subject', y='Score', palette='muted')
6
7 plt.title('Violin Plot of Scores by Subject')
8 plt.show()
```

2.4.4 Scatter Plot (relplot / scatterplot) Purpose: Shows relationship between two variables.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Create data
6 df = pd.DataFrame({
7     'Hours_Studied': [1,2,3,4,5,6,7,8,9,10],
8     'Exam_Score': [35,40,50,55,65,70,78,85,90,95],
9     'Attendance': [60,65,70,75,80,85,88,90,92,95]
10 })
11
12 # Scatter plot
13 sns.scatterplot(data=df, x='Hours_Studied', y='Exam_Score', size='Attendance',
14                 sizes=(50, 200), hue='Attendance', palette='viridis')
15
16 plt.title('Study Hours vs Exam Score (Size = Attendance)')
17 plt.show()
```

2.4.5 Pairplot Purpose: Shows all pairwise relationships in a dataset.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Sample data
6 df = pd.DataFrame({
7     'Math': [85,88,90,92,85,87,89,91,86,88],
8     'Science': [78,82,85,88,80,83,86,89,84,87],
9     'English': [92,94,95,91,93,96,90,92,94,95]
10 })
11
12 # Create pairplot
13 sns.pairplot(df, diag_kind='hist') # diag_kind can be 'hist' or 'kde'
14
15 plt.suptitle('Pairplot of Subject Scores', y=1.02)
16 plt.show()
```

2.4.6 Heatmap (Correlation) Purpose: Shows correlation matrix visually.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Calculate correlation matrix
6 corr_matrix = df.corr()
7
8 # Create heatmap
9 plt.figure(figsize=(8, 6))
10 sns.heatmap(corr_matrix,
11             annot=True,          # Show values inside boxes
12             cmap='coolwarm',     # Color scheme
13             center=0,            # Center colormap at 0
14             square=True,        # Make squares
15             linewidths=1,       # Grid lines
16             cbar_kws={'shrink': 0.8}) # Color bar size
17
18 plt.title('Correlation Heatmap')
19 plt.show()
```

2.4.7 Bar Plot Purpose: Shows mean values by category.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
```

```

3 import pandas as pd
4
5 # Data
6 df = pd.DataFrame({
7     'Department': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT', 'Finance', 'HR'],
8     'Salary': [50000, 45000, 52000, 48000, 46000, 51000, 49000, 44000]
9 })
10
11 # Bar plot with error bars
12 sns.barplot(data=df, x='Department', y='Salary', ci=95, palette='pastel')
13
14 plt.title('Average Salary by Department')
15 plt.show()

```

#### 2.4.8 Count Plot Purpose: Shows frequency of categorical values.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Data
6 df = pd.DataFrame({
7     'Product': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A']
8 })
9
10 # Count plot
11 sns.countplot(data=df, x='Product', palette='Set1')
12
13 plt.title('Product Frequency')
14 plt.xlabel('Product Type')
15 plt.ylabel('Count')
16 plt.show()

```

#### 2.4.9 Regression Plot (regplot / lmpplot) Purpose: Shows scatter plot with regression line.

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 # Data
6 df = pd.DataFrame({
7     'X': [1,2,3,4,5,6,7,8,9,10],
8     'Y': [2,3,5,7,11,14,16,19,22,25]
9 })
10
11 # Regression plot
12 sns.regplot(data=df, x='X', y='Y', scatter_kws={'color':'blue'},
13             line_kws={'color':'red'})
14
15 plt.title('Scatter Plot with Regression Line')
16 plt.show()

```

## 2.5 Combining Matplotlib and Seaborn

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 import numpy as np
5
6 # Create data
7 np.random.seed(42)
8 df = pd.DataFrame({
9     'Math': np.random.normal(75, 10, 100),
10    'Science': np.random.normal(70, 12, 100),
11    'English': np.random.normal(80, 8, 100)
12 })
13
14 # Create figure with subplots
15 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
16
17 # Plot 1: Histograms
18 axes[0, 0].hist(df['Math'], bins=20, alpha=0.5, label='Math')
19 axes[0, 0].hist(df['Science'], bins=20, alpha=0.5, label='Science')
20 axes[0, 0].set_title('Distribution Comparison')
21 axes[0, 0].legend()
22
23 # Plot 2: Boxplot using Seaborn
24 sns.boxplot(data=df, ax=axes[0, 1])
25 axes[0, 1].set_title('Boxplots of Scores')
26
27 # Plot 3: Heatmap using Seaborn
28 sns.heatmap(df.corr(), annot=True, cmap='coolwarm', ax=axes[1, 0])
29 axes[1, 0].set_title('Correlation Heatmap')

```

```
30 plt.tight_layout()
31 plt.show()
32
33
34 # Pairplot separately
35 sns.pairplot(df, diag_kind='kde')
36 plt.suptitle('Pairplot of All Subjects', y=1.02)
37 plt.show()
```

2.6 Complete Example: EDA Visualization

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 import numpy as np
5
6 # Create dataset
7 np.random.seed(42)
8 df = pd.DataFrame({
9     'Student_ID': range(1, 101),
10    'Study_Hours': np.random.randint(1, 12, 100),
11    'Sleep_Hours': np.random.randint(4, 9, 100),
12    'Exam_Score': np.random.randint(40, 100, 100),
13    'Attendance': np.random.randint(60, 100, 100),
14    'Gender': np.random.choice(['M', 'F'], 100)
15 })
16
17 # Create a dashboard of visualizations
18 fig = plt.figure(figsize=(16, 12))
19
20 # 1. Histogram with KDE
21 plt.subplot(2, 3, 1)
22 sns.histplot(df['Exam_Score'], bins=20, kde=True, color='blue')
23 plt.title('Distribution of Exam Scores')
24
25 # 2. Boxplot (Scores by Gender)
26 plt.subplot(2, 3, 2)
27 sns.boxplot(data=df, x='Gender', y='Exam_Score', palette='Set2')
28 plt.title('Exam Scores by Gender')
29
30 # 3. Scatter plot
31 plt.subplot(2, 3, 3)
32 sns.scatterplot(data=df, x='Study_Hours', y='Exam_Score', hue='Gender', alpha=0.7)
33 plt.title('Study Hours vs Exam Score')
34
35 # 4. Heatmap
36 plt.subplot(2, 3, 4)
37 corr = df[['Study_Hours', 'Sleep_Hours', 'Exam_Score', 'Attendance']].corr()
38 sns.heatmap(corr, annot=True, cmap='coolwarm', center=0)
39 plt.title('Correlation Heatmap')
40
41 # 5. Regression plot
42 plt.subplot(2, 3, 5)
43 sns.regplot(data=df, x='Attendance', y='Exam_Score', scatter_kws={'alpha':0.5})
44 plt.title('Attendance vs Exam Score (with trend line)')
45
46 # 6. Count plot
47 plt.subplot(2, 3, 6)
48 sns.countplot(data=df, x='Gender', palette='pastel')
49 plt.title('Gender Distribution')
50
51 plt.tight_layout()
52 plt.show()
```

3. Quick Reference Table

| What to Show            | Matplotlib       | Seaborn           |
|-------------------------|------------------|-------------------|
| Trend over time         | plt.plot()       | sns.lineplot()    |
| Relationship (2 vars)   | plt.scatter()    | sns.scatterplot() |
| Data distribution       | plt.hist()       | sns.histplot()    |
| 5-number summary        | plt.boxplot()    | sns.boxplot()     |
| Compare categories      | plt.bar()        | sns.barplot()     |
| Correlation matrix      | plt.imshow()     | sns.heatmap()     |
| All pair relationships  | Multiple plots   | sns.pairplot()    |
| Distribution + density  | plt.hist() + KDE | sns.kdeplot()     |
| Full distribution shape | Not available    | sns.violinplot()  |

3.1 When to Use Which?

Use Matplotlib when:

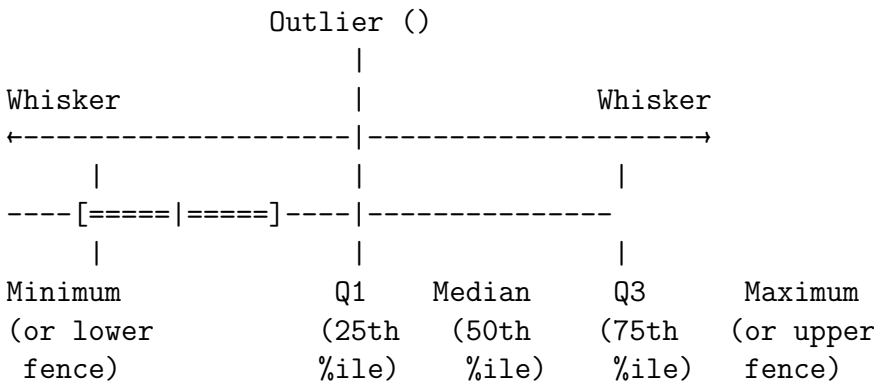
Use Seaborn when:

|                               |                          |
|-------------------------------|--------------------------|
| Full control over plots       | Quick statistical plots  |
| Custom complex visualizations | DataFrame-based plotting |
| Saving custom formats         | Built-in color palettes  |
| Combining multiple plots      | Automatic legends/labels |

## Part 1: What is a Box Plot

A Box Plot (also called Box and Whisker Plot) is a standardized graphical representation of data distribution based on the five-number summary.

### Components of a Box Plot



## Part 2: Formulas for Box Plot

### 2.1 Five-Number Summary

| Component   | Formula                      | Description       |
|-------------|------------------------------|-------------------|
| Minimum     | $\min(x_1, x_2, \dots, x_n)$ | Smallest value    |
| Q1          | Position $(n + 1)/4$         | 25% of data below |
| Median (Q2) | Middle value or average      | 50% of data below |
| Q3          | Position $3(n + 1)/4$        | 75% of data below |
| Maximum     | $\max(x_1, x_2, \dots, x_n)$ | Largest value     |

### 2.2 Quartile Positions

$$Q1 = (n + 1) \times \frac{1}{4}$$

$$Q2 = (n + 1) \times \frac{2}{4}$$

$$Q3 = (n + 1) \times \frac{3}{4}$$

### 2.3 Interquartile Range

$$IQR = Q3 - Q1$$

### 2.4 Fence Formulas

$$\text{Lower Fence} = Q1 - 1.5 \times IQR$$

$$\text{Upper Fence} = Q3 + 1.5 \times IQR$$

$$\text{Lower Outer Fence} = Q1 - 3 \times IQR$$

$$\text{Upper Outer Fence} = Q3 + 3 \times IQR$$

## Part 3: Numerical Example

Dataset:

15, 22, 25, 28, 30, 32, 35, 38, 40, 45, 48, 50, 52, 55, 60, 65, 70, 75, 80, 100

### Step 1: n = 20

### Step 2: Minimum and Maximum

Minimum = 15 Maximum = 100

### Step 3: Median

$$Median = \frac{45 + 48}{2} = 46.5$$

### Step 4: Q1

$$Q1 = 30 + 0.25(32 - 30) = 30.5$$

Step 5: Q3

$$Q3 = 60 + 0.75(65 - 60) = 63.75$$

Step 6: IQR

$$IQR = 63.75 - 30.5 = 33.25$$

Step 7: Fences

$$Lower = 30.5 - 1.5(33.25) = -19.375$$

$$Upper = 63.75 + 1.5(33.25) = 113.625$$

Step 8: Five Number Summary

|         |       |
|---------|-------|
| Minimum | 15    |
| Q1      | 30.5  |
| Median  | 46.5  |
| Q3      | 63.75 |
| Maximum | 100   |

Part 4: Example with Outlier

Dataset:

5, 8, 10, 12, 15, 18, 20, 22, 25, 30, 100

Step 1: n = 11

Median = 18 Q1 = 10 Q3 = 25

Step 2: IQR

$$IQR = 25 - 10 = 15$$

Step 3: Fences

$$Lower = 10 - 1.5(15) = -12.5$$

$$Upper = 25 + 1.5(15) = 47.5$$

Step 4: Outlier

100 is an outlier (since 100 > 47.5)

Step 5: Five Number Summary

|                   |     |
|-------------------|-----|
| Minimum           | 5   |
| Q1                | 10  |
| Median            | 18  |
| Q3                | 25  |
| Maximum (whisker) | 30  |
| Outlier           | 100 |

Part 1: Data Distribution

1.1 What is Data Distribution?

Data distribution describes how the values of a dataset are spread or arranged. It shows the frequency of different values or ranges of values.

1.2 Types of Distributions

| Distribution Type       | Shape         | Characteristics             | Real-life Example                 |
|-------------------------|---------------|-----------------------------|-----------------------------------|
| Normal Distribution     | Bell-shaped   | Symmetric, mean=median=mode | Heights of people                 |
| Uniform Distribution    | Flat          | All values equally likely   | Dice roll outcomes                |
| Right-skewed (Positive) | Tail on right | Mean > Median               | Income distribution               |
| Left-skewed (Negative)  | Tail on left  | Mean < Median               | Exam scores (easy test)           |
| Bimodal Distribution    | Two peaks     | Two distinct groups         | Heights of men and women together |

1.3 Normal Distribution

**Definition:** A normal distribution (Gaussian distribution) is a symmetric, bell-shaped distribution where most values cluster around the mean.

Empirical Rule (68-95-99.7 Rule)

| Range             | Percentage of Data |
|-------------------|--------------------|
| $\mu \pm 1\sigma$ | 68%                |
| $\mu \pm 2\sigma$ | 95%                |
| $\mu \pm 3\sigma$ | 99.7%              |

Numerical Example 1

Given:  $\mu = 100$ ,  $\sigma = 15$

(a)  $85 = 100 - 15 = \mu - 1\sigma$   
 $115 = 100 + 15 = \mu + 1\sigma$   
 $\Rightarrow$  Answer = 68%

(b)  $70 = 100 - 30 = \mu - 2\sigma$   
 $130 = 100 + 30 = \mu + 2\sigma$   
 $\Rightarrow$  Answer = 95%

(c) Above 130 =  $\frac{100\% - 95\%}{2} = 2.5\%$

1.4 Z-Score (Standard Score)

**Definition:** A Z-score tells how many standard deviations a data point is from the mean.

$$Z = \frac{x - \mu}{\sigma}$$

Interpretation

| Z-Score Range | Interpretation                        |
|---------------|---------------------------------------|
| $Z = 0$       | Exactly at mean                       |
| $-1 < Z < 1$  | Within 1 standard deviation (typical) |
| $-2 < Z < 2$  | Within 2 standard deviations          |
| $Z > 2$       | Unusual value                         |
| $Z > 3$       | Very unusual / potential outlier      |

Numerical Example 2

Given:  $\mu = 70$ ,  $\sigma = 10$

(a)  $Z = \frac{85 - 70}{10} = \frac{15}{10} = 1.5$   
(b)  $Z = \frac{55 - 70}{10} = \frac{-15}{10} = -1.5$   
(c)  $Z = \frac{70 - 70}{10} = 0$

1.5 Skewness

**Definition:** Skewness measures the asymmetry of a distribution.

$$\text{Skewness} = \frac{3(\text{Mean} - \text{Median})}{\text{Standard Deviation}}$$

Interpretation

| Skewness Value | Distribution Type              |
|----------------|--------------------------------|
| 0              | Symmetric                      |
| $> 0$          | Right-skewed (Mean $>$ Median) |
| $< 0$          | Left-skewed (Mean $<$ Median)  |

Numerical Example 3

Given: Mean = 50, Median = 45, Standard Deviation = 10

$$\begin{aligned} \text{Skewness} &= \frac{3(50 - 45)}{10} \\ &= \frac{15}{10} \\ &= 1.5 \end{aligned}$$

Since  $1.5 > 0$ , the distribution is **right-skewed**.

## 1.6 Kurtosis

**Definition:** Kurtosis measures the tailedness of a distribution (how many outliers).

$$\text{Kurtosis} = \frac{\sum (x_i - \bar{x})^4}{n \cdot s^4}$$

$$\text{Excess Kurtosis} = \text{Kurtosis} - 3$$

### Interpretation

| Excess Kurtosis | Type        | Description                 |
|-----------------|-------------|-----------------------------|
| 0               | Mesokurtic  | Normal distribution shape   |
| > 0             | Leptokurtic | Heavy tails, more outliers  |
| < 0             | Platykurtic | Light tails, fewer outliers |

### Numerical Example 4: Calculating Kurtosis

Dataset: 45, 48, 50, 51, 52, 55, 80

#### Step 1: Mean

$$\bar{x} = \frac{45 + 48 + 50 + 51 + 52 + 55 + 80}{7} = \frac{381}{7} = 54.43$$

#### Step 2: Deviations and 4th powers

| $x_i$ | $x_i - \bar{x}$ | $(x_i - \bar{x})^4$ |
|-------|-----------------|---------------------|
| 45    | -9.43           | 7,908.6             |
| 48    | -6.43           | 1,709.5             |
| 50    | -4.43           | 385.0               |
| 51    | -3.43           | 138.3               |
| 52    | -2.43           | 34.8                |
| 55    | 0.57            | 0.1                 |
| 80    | 25.57           | 427,500             |

#### Step 3: Sum of 4th powers

$$\sum (x_i - \bar{x})^4 = 437,676.3$$

#### Step 4: Variance and $s^4$

$$s^2 = 117.38, \quad s = 10.83, \quad s^4 = 13,780$$

#### Step 5: Kurtosis

$$\text{Kurtosis} = \frac{437,676.3}{7 \cdot 13,780} = \frac{437,676.3}{96,460} = 4.54$$

#### Step 6: Excess Kurtosis

$$\text{Excess Kurtosis} = 4.54 - 3 = 1.54$$

**Conclusion:** Excess kurtosis > 0, so the distribution is **Leptokurtic (heavy tails, more outliers)**.

## Part 2: Correlation

### 2.1 What is Correlation?

Correlation measures the strength and direction of the linear relationship between two variables.

### 2.2 Types of Correlation

| Type     | Direction          | Description                 |
|----------|--------------------|-----------------------------|
| Positive | Same direction     | As X increases, Y increases |
| Negative | Opposite direction | As X increases, Y decreases |
| Zero     | No relationship    | X and Y are independent     |

### 2.3 Pearson Correlation Coefficient

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

2.4 Interpretation

| r Value           | Strength                      |
|-------------------|-------------------------------|
| $r = 1$           | Perfect positive correlation  |
| $0.7 < r < 1$     | Strong positive correlation   |
| $0.3 < r < 0.7$   | Moderate positive correlation |
| $0 < r < 0.3$     | Weak positive correlation     |
| $r = 0$           | No correlation                |
| $-0.3 < r < 0$    | Weak negative correlation     |
| $-0.7 < r < -0.3$ | Moderate negative correlation |
| $-1 < r < -0.7$   | Strong negative correlation   |
| $r = -1$          | Perfect negative correlation  |

2.5 Covariance (Alternative)

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n - 1}$$
$$r = \frac{\text{Cov}(x,y)}{s_x \cdot s_y}$$

Numerical Example 5: Positive Correlation

| X | Y  |
|---|----|
| 1 | 2  |
| 2 | 4  |
| 3 | 6  |
| 4 | 8  |
| 5 | 10 |

$$\bar{x} = 3, \quad \bar{y} = 6$$

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = 20$$
$$\sum (x_i - \bar{x})^2 = 10, \quad \sum (y_i - \bar{y})^2 = 40$$
$$r = \frac{20}{\sqrt{10 \cdot 40}} = \frac{20}{20} = 1$$

**Result:**  $r = 1$  (Perfect positive correlation)

Numerical Example 6: Negative Correlation

| X | Y  |
|---|----|
| 1 | 10 |
| 2 | 8  |
| 3 | 6  |
| 4 | 4  |
| 5 | 2  |

$$r = -1$$

**Result:** Perfect negative correlation

Numerical Example 7: Moderate Correlation

| X | Y |
|---|---|
| 1 | 5 |
| 2 | 7 |
| 3 | 4 |
| 4 | 6 |
| 5 | 8 |

$$r = 0.5$$

**Result:** Moderate positive correlation

3.1 What is an Outlier?

An outlier is a data point that is significantly different from other observations in a dataset. It lies far away from the majority of data points.

3.2 Causes of Outliers

| Cause              | Description                |
|--------------------|----------------------------|
| Data entry errors  | Typing 1000 instead of 100 |
| Measurement errors | Faulty equipment           |
| Natural variation  | Genuine extreme values     |
| Sampling errors    | Data from wrong population |

### 3.3 Methods to Detect Outliers

#### Method 1: IQR Method (Most Common)

Formula:

$$IQR = Q3 - Q1$$

$$\text{Lower Bound} = Q1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q3 + 1.5 \times IQR$$

**Rule:** Any value below Lower Bound or above Upper Bound is an outlier.

#### Numerical Example 8: IQR Method

Dataset: 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 80

**Step 1:** Data already sorted,  $n = 11$

**Step 2:**  $Q1 = 14$

**Step 3:**  $Q3 = 26$

**Step 4:**

$$IQR = 26 - 14 = 12$$

**Step 5: Bounds**

$$\text{Lower Bound} = 14 - 1.5 \times 12 = 14 - 18 = -4$$

$$\text{Upper Bound} = 26 + 1.5 \times 12 = 26 + 18 = 44$$

**Step 6: Outlier detection**

$80 > 44 \Rightarrow$  Outlier

**Answer:** 80 is an outlier

#### Method 2: Z-Score Method

Formula:

$$Z = \frac{x - \mu}{\sigma}$$

**Rule:** If  $|Z| > 3$ , then outlier

#### Numerical Example 9: Z-Score Method

Mean = 50, Standard deviation = 5

Data: 45, 48, 52, 55, 75

**Step 1: Z-scores**

$$Z(45) = \frac{45 - 50}{5} = -1$$

$$Z(48) = \frac{48 - 50}{5} = -0.4$$

$$Z(52) = \frac{52 - 50}{5} = 0.4$$

$$Z(55) = \frac{55 - 50}{5} = 1$$

$$Z(75) = \frac{75 - 50}{5} = 5$$

**Step 2: Check condition**

$|Z| = 5 > 3 \Rightarrow$  Outlier

**Answer:** 75 is an outlier

#### Method 3: Standard Deviation Method

Formula:

$$\text{Upper Limit} = \mu + 3\sigma$$

$$\text{Lower Limit} = \mu - 3\sigma$$

Numerical Example 10

Mean = 100, Standard deviation = 10  
Data: 65, 95, 105, 135  
Step 1: Limits

Upper Limit =  $100 + 3 \times 10 = 130$

Lower Limit =  $100 - 3 \times 10 = 70$

Step 2: Check values

- $65 < 70 \Rightarrow$  Outlier
- $95 \Rightarrow$  Not outlier
- $105 \Rightarrow$  Not outlier
- $135 > 130 \Rightarrow$  Outlier

Answer: 65 and 135 are outliers

3.4 What to Do with Outliers?

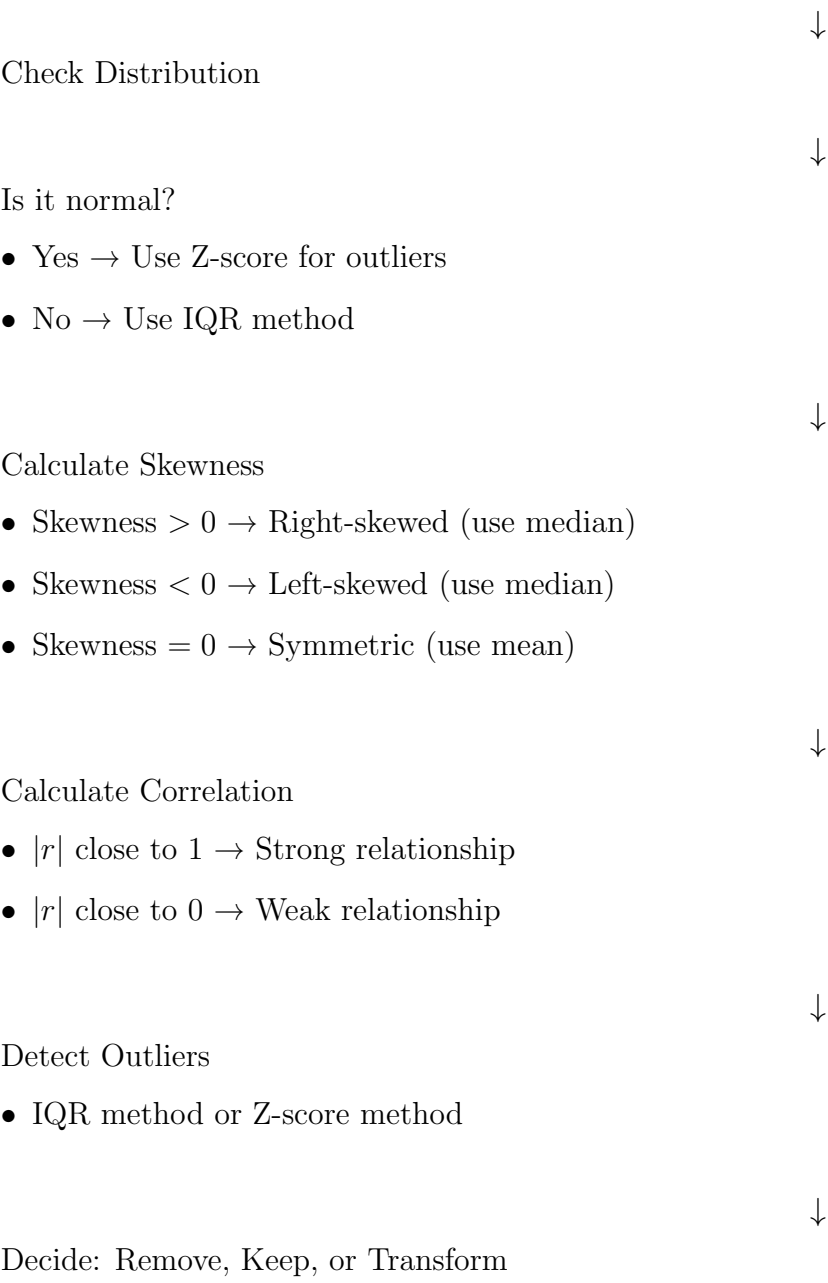
| Action            | When to Use                                      |
|-------------------|--------------------------------------------------|
| Remove            | Data entry error, measurement error              |
| Keep              | Genuine extreme value (e.g., billionaire income) |
| Cap/Winsorize     | Replace with nearest non-outlier value           |
| Transform         | Apply log or square root transformation          |
| Separate analysis | Analyze outliers separately                      |

Part 4: Summary Formula Table

| Concept           | Formula                                                                         | Purpose                         |
|-------------------|---------------------------------------------------------------------------------|---------------------------------|
| Z-Score           | $Z = \frac{x-\mu}{\sigma}$                                                      | Find how many SDs from mean     |
| Skewness          | $\frac{3(\text{Mean}-\text{Median})}{SD}$                                       | Measure asymmetry               |
| Kurtosis          | $\frac{\sum(x-\bar{x})^4}{n \cdot s^4}$                                         | Measure tailedness              |
| Correlation (r)   | $\frac{\sum(x-\bar{x})(y-\bar{y})}{\sqrt{\sum(x-\bar{x})^2 \sum(y-\bar{y})^2}}$ | Strength of linear relationship |
| IQR               | $Q3 - Q1$                                                                       | Spread of middle 50%            |
| Outlier (IQR)     | Below $Q1 - 1.5IQR$ or above $Q3 + 1.5IQR$                                      | Detect outliers                 |
| Outlier (Z-score) | $ Z  > 3$                                                                       | Detect outliers                 |
| Empirical Rule    | 68%, 95%, 99.7%                                                                 | Normal distribution percentages |

Part 5: Quick Decision Flowchart

Start with your data



### 3.1 Part 1: Introduction to Plotly

#### 3.1.1 1.1 What is Plotly?

Plotly is an interactive graphing library that allows you to create dynamic, web-based visualizations. Unlike Matplotlib and Seaborn (which create static plots), Plotly creates interactive plots where users can:

- Hover to see values
- Zoom in and out
- Pan across the plot
- Toggle legends on/off
- Download plots as images

#### 3.1.2 1.2 Why Plotly for Dashboards?

| Feature              | Benefit                                  |
|----------------------|------------------------------------------|
| Interactivity        | Users can explore data on their own      |
| Easy to use          | Simple syntax similar to Python          |
| Dashboard ready      | Can combine multiple plots into one view |
| Web-based            | Works in Jupyter, HTML, and web browsers |
| No JavaScript needed | All done in Python                       |

#### 3.1.3 1.3 Installing Plotly

```
pip install plotly
```

#### 3.1.4 1.4 Importing Plotly

```
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

## 3.2 Part 2: Basic Plotly Charts

### 3.2.1 2.1 Line Chart

**Purpose:** Show trends over time.

```
import plotly.express as px
import pandas as pd

# Create data
df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
    'Sales': [120, 135, 150, 140, 165, 180],
    'Expenses': [100, 110, 115, 120, 125, 130]
})

# Create line chart
fig = px.line(df, x='Month', y='Sales', title='Monthly Sales Trend')

# Add second line
fig.add_scatter(x=df['Month'], y=df['Expenses'], mode='lines', name='Expenses')

fig.show()
```

### 3.2.2 2.2 Bar Chart

**Purpose:** Compare categories.

```
import plotly.express as px
import pandas as pd

# Data
df = pd.DataFrame({
    'Product': ['A', 'B', 'C', 'D'],
    'Sales': [250, 180, 300, 220]
})

# Bar chart
fig = px.bar(df, x='Product', y='Sales', title='Product Sales', color='Product')
fig.show()
```

### 3.2.3 2.3 Scatter Plot

**Purpose:** Show relationship between two variables.

```
import plotly.express as px
import pandas as pd

# Data
df = pd.DataFrame({
    'Study_Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Exam_Score': [35, 40, 50, 55, 65, 70, 78, 85, 90, 95]
})

# Scatter plot
fig = px.scatter(df, x='Study_Hours', y='Exam_Score',
                 title='Study Hours vs Exam Score',
                 labels={'Study_Hours': 'Hours Studied', 'Exam_Score': 'Score'})

fig.show()
```

### 3.3 2.4 Histogram

**Purpose:** Show distribution of data.

```
1 import plotly.express as px
2 import numpy as np
3
4 # Generate data
5 np.random.seed(42)
6 scores = np.random.normal(70, 10, 200)
7
8 # Histogram
9 fig = px.histogram(scores, nbins=20, title='Distribution of Exam Scores')
10 fig.show()
```

### 3.4 2.5 Box Plot

**Purpose:** Show five-number summary and outliers.

```
1 import plotly.express as px
2 import pandas as pd
3 import numpy as np
4
5 # Data
6 df = pd.DataFrame({
7     'Department': ['IT']*50 + ['HR']*50 + ['Finance']*50,
8     'Salary': np.random.normal(50000, 10000, 150)
9 })
10
11 # Box plot
12 fig = px.box(df, x='Department', y='Salary', title='Salary Distribution by Department')
13 fig.show()
```

### 3.5 2.6 Pie Chart

**Purpose:** Show proportions of a whole.

```
1 import plotly.express as px
2 import pandas as pd
3
4 # Data
5 df = pd.DataFrame({
6     'Category': ['Food', 'Rent', 'Transport', 'Entertainment', 'Savings'],
7     'Amount': [300, 800, 200, 150, 550]
8 })
9
10 # Pie chart
11 fig = px.pie(df, values='Amount', names='Category', title='Monthly Expense Distribution')
12 fig.show()
```

### 3.6 2.7 Heatmap (Correlation)

**Purpose:** Show correlation matrix visually.

```
1 import plotly.express as px
2 import pandas as pd
3
4 # Create correlation data
5 df_corr = pd.DataFrame({
6     'Math': [85, 88, 90, 92, 85, 87, 89, 91, 86, 88],
7     'Science': [78, 82, 85, 88, 80, 83, 86, 89, 84, 87],
8 })
```

```
8     'English': [92, 94, 95, 91, 93, 96, 90, 92, 94, 95]
9 })
10
11 # Calculate correlation
12 corr_matrix = df_corr.corr()
13
14 # Heatmap
15 fig = px.imshow(corr_matrix, text_auto=True, title='Correlation Heatmap')
16 fig.show()
```

### 3.7 Part 3: Creating Dashboards in Plotly

#### 3.7.1 3.1 What is a Dashboard?

A dashboard is a collection of multiple visualizations arranged together on a single screen, often with interactive filters.

#### 3.7.2 3.2 Method 1: Subplots (make\_subplots)

```
1 import plotly.graph_objects as go
2 from plotly.subplots import make_subplots
3 import pandas as pd
4 import numpy as np
5
6 # Create sample data
7 df = pd.DataFrame({
8     'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
9     'Sales': [120, 135, 150, 140, 165, 180],
10    'Profit': [30, 35, 40, 38, 45, 50],
11    'Customers': [50, 55, 60, 58, 65, 70]
12 })
13
14 # Create subplots (2 rows, 2 columns)
15 fig = make_subplots(
16     rows=2, cols=2,
17     subplot_titles=('Sales Trend', 'Profit Trend', 'Monthly Customers', 'Sales vs Profit'),
18     specs=[[{'type': 'scatter'}, {'type': 'scatter'}],
19           [{'type': 'bar'}, {'type': 'scatter'}]]
20 )
21
22 # Plot 1: Line chart (row1, col1)
23 fig.add_trace(
24     go.Scatter(x=df['Month'], y=df['Sales'], mode='lines+markers', name='Sales'),
25     row=1, col=1
26 )
27
28 # Plot 2: Line chart (row1, col2)
29 fig.add_trace(
30     go.Scatter(x=df['Month'], y=df['Profit'], mode='lines+markers', name='Profit'),
31     row=1, col=2
32 )
33
34 # Plot 3: Bar chart (row2, col1)
35 fig.add_trace(
36     go.Bar(x=df['Month'], y=df['Customers'], name='Customers'),
37     row=2, col=1
38 )
39
40 # Plot 4: Scatter plot (row2, col2)
41 fig.add_trace(
42     go.Scatter(x=df['Sales'], y=df['Profit'], mode='markers', name='Sales vs Profit'),
43     row=2, col=2
44 )
45
46 # Update layout
47 fig.update_layout(title='Sales Dashboard', height=600, showlegend=False)
48 fig.show()
```

### 3.8 3.3 Method 2: Using HTML and Display (Multiple Figures)

```
1 import plotly.express as px
2 import pandas as pd
3 import numpy as np
4 from plotly.subplots import make_subplots
5
6 # Create sample dataset
7 np.random.seed(42)
8 df = pd.DataFrame({
9     'Date': pd.date_range('2024-01-01', periods=30, freq='D'),
10    'Sales': np.random.randint(1000, 5000, 30),
11    'Customers': np.random.randint(50, 200, 30),
12    'Profit': np.random.randint(100, 500, 30),
```

```
13     'Category': np.random.choice(['A', 'B', 'C'], 30)
14 })
15
16 # Create individual figures
17
18 # Figure 1: Sales trend
19 fig1 = px.line(df, x='Date', y='Sales', title='Daily Sales Trend')
20
21 # Figure 2: Customer count
22 fig2 = px.bar(df, x='Date', y='Customers', title='Daily Customers')
23
24 # Figure 3: Profit distribution
25 fig3 = px.histogram(df, x='Profit', nbins=15, title='Profit Distribution')
26
27 # Figure 4: Sales by Category (Pie)
28 category_sales = df.groupby('Category')['Sales'].sum().reset_index()
29 fig4 = px.pie(category_sales, values='Sales', names='Category', title='Sales by Category')
30
31 # Display all figures (in Jupyter Notebook)
32 fig1.show()
33 fig2.show()
34 fig3.show()
35 fig4.show()
```

### 3.9 3.4 Method 3: The Dashboard Class (Advanced)

```
1 import plotly.graph_objects as go
2 from plotly.subplots import make_subplots
3 import pandas as pd
4 import numpy as np
5
6 class Dashboard:
7     def __init__(self, title, rows, cols):
8         self.title = title
9         self.rows = rows
10        self.cols = cols
11        self.fig = make_subplots(
12            rows=rows, cols=cols,
13            subplot_titles=[''] * (rows * cols),
14            horizontal_spacing=0.1,
15            vertical_spacing=0.15
16        )
17        self.current_row = 1
18        self.current_col = 1
19
20        def add_plot(self, trace, row, col):
21            self.fig.add_trace(trace, row=row, col=col)
22
23        def show(self):
24            self.fig.update_layout(title=self.title, height=600, showlegend=False)
25            self.fig.show()
26
27 # Create dashboard
28 dashboard = Dashboard('Sales Dashboard', 2, 2)
29
30 # Add plots
31 dashboard.add_plot(go.Scatter(x=[1,2,3], y=[100,150,200], mode='lines'), 1, 1)
32 dashboard.add_plot(go.Bar(x=['A','B','C'], y=[50,75,100]), 1, 2)
33 dashboard.add_plot(go.Scatter(x=[1,2,3], y=[200,180,160], mode='markers'), 2, 1)
34 dashboard.add_plot(go.Bar(x=['X','Y','Z'], y=[30,60,90]), 2, 2)
35
36 dashboard.show()
```

### 3.10 Part 4: Adding Interactivity

#### 3.10.1 4.1 Dropdown Filter

```
1 import plotly.express as px
2 import pandas as pd
3
4 # Data
5 df = pd.DataFrame({
6     'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'] * 3,
7     'Product': ['A']*6 + ['B']*6 + ['C']*6,
8     'Sales': [120, 135, 150, 140, 165, 180,
9               90, 105, 120, 110, 135, 150,
10              80, 95, 110, 100, 125, 140]
11 })
12
13 # Create figure
14 fig = px.line(df, x='Month', y='Sales', color='Product', title='Sales by Product')
```

```
15
16 # Add dropdown
17 fig.update_layout(
18     updatemenus=[
19         dict(
20             buttons=list([
21                 dict(label="All",
22                     method="update",
23                     args=[{"visible": [True, True, True]}]),
24                 dict(label="Product A",
25                     method="update",
26                     args=[{"visible": [True, False, False]}]),
27                 dict(label="Product B",
28                     method="update",
29                     args=[{"visible": [False, True, False]}]),
30                 dict(label="Product C",
31                     method="update",
32                     args=[{"visible": [False, False, True]}])
33             ],
34             direction="down",
35             showactive=True,
36         )
37     ]
38 )
39
40 fig.show()
```

3.10.2 4.2 Range Slider

```
1 import plotly.express as px
2 import pandas as pd
3 import numpy as np
4
5 # Create date range data
6 df = pd.DataFrame({
7     'Date': pd.date_range('2024-01-01', periods=100, freq='D'),
8     'Value': np.random.randn(100).cumsum() + 100
9 })
10
11 # Line chart with range slider
12 fig = px.line(df, x='Date', y='Value', title='Time Series with Range Slider')
13
14 # Add range slider
15 fig.update_layout(
16     xaxis=dict(
17         rangeselector=dict(
18             buttons=list([
19                 dict(count=7, label="1w", step="day", stepmode="backward"),
20                 dict(count=30, label="1m", step="day", stepmode="backward"),
21                 dict(count=90, label="3m", step="day", stepmode="backward"),
22                 dict(step="all")
23             ])
24         ),
25         rangeslider=dict(visible=True),
26         type="date"
27     )
28 )
29
30 fig.show()
```

3.10.3 4.3 Hover Information

```
1 import plotly.express as px
2 import pandas as pd
3
4 # Data
5 df = pd.DataFrame({
6     'Student': ['A', 'B', 'C', 'D', 'E'],
7     'Math': [85, 72, 90, 68, 95],
8     'Science': [88, 70, 92, 65, 93],
9     'English': [82, 75, 88, 70, 91]
10 })
11
12 # Scatter plot with custom hover
13 fig = px.scatter(df, x='Math', y='Science',
14                 hover_data=['Student', 'English'],
15                 title='Math vs Science Scores',
16                 labels={'Math': 'Math Score', 'Science': 'Science Score'})
17
18 # Customize hover
19 fig.update_traces(
20     hovertemplate='<b>Student: {customdata[0]}</b><br>' +
```

```
21         'Math: %{x}<br>' +
22         'Science: %{y}<br>' +
23         'English: %{customdata[1]}<extra></extra>',
24     customdata=df[['Student', 'English']].values
25 )
26
27 fig.show()
```

### 3.11 Complete Dashboard Example

#### 3.11.1 Sales Dashboard

```
1 import plotly.express as px
2 import plotly.graph_objects as go
3 from plotly.subplots import make_subplots
4 import pandas as pd
5 import numpy as np
6
7 np.random.seed(42)
8 dates = pd.date_range('2024-01-01', periods=90, freq='D')
9 products = ['Laptop', 'Phone', 'Tablet', 'Headphones']
10
11 df = pd.DataFrame({
12     'Date': np.tile(dates, 4),
13     'Product': np.repeat(products, 90),
14     'Sales': np.random.randint(1000, 10000, 360),
15     'Units': np.random.randint(10, 100, 360),
16     'Region': np.random.choice(['North', 'South', 'East', 'West'], 360)
17 })
18
19 daily_sales = df.groupby('Date')['Sales'].sum().reset_index()
20 product_sales = df.groupby('Product')['Sales'].sum().reset_index()
21 region_sales = df.groupby('Region')['Sales'].sum().reset_index()
22
23 fig = make_subplots(
24     rows=2, cols=2,
25     subplot_titles=('Daily Sales Trend', 'Sales by Product',
26                     'Sales by Region', 'Units vs Sales'),
27     specs=[[{'type': 'scatter'}, {'type': 'bar'}],
28            [{'type': 'pie'}, {'type': 'scatter'}]]
29 )
30
31 fig.add_trace(
32     go.Scatter(x=daily_sales['Date'], y=daily_sales['Sales'],
33                mode='lines', name='Daily Sales',
34                line=dict(color='blue', width=2)),
35     row=1, col=1
36 )
37
38 fig.add_trace(
39     go.Bar(x=product_sales['Product'], y=product_sales['Sales'],
40            name='Product Sales',
41            marker_color=['red', 'green', 'orange', 'purple']),
42     row=1, col=2
43 )
44
45 fig.add_trace(
46     go.Pie(labels=region_sales['Region'], values=region_sales['Sales']),
47     row=2, col=1
48 )
49
50 fig.add_trace(
51     go.Scatter(x=df['Units'], y=df['Sales'], mode='markers',
52                marker=dict(color='green', size=5, opacity=0.6)),
53     row=2, col=2
54 )
55
56 fig.update_layout(
57     title_text='Sales Performance Dashboard',
58     height=700,
59     showlegend=False,
60     hovermode='closest'
61 )
62
63 fig.update_xaxes(title_text='Date', row=1, col=1)
64 fig.update_yaxes(title_text='Sales ($)', row=1, col=1)
65
66 fig.update_xaxes(title_text='Product', row=1, col=2)
67 fig.update_yaxes(title_text='Sales ($)', row=1, col=2)
68
69 fig.update_xaxes(title_text='Units Sold', row=2, col=2)
70 fig.update_yaxes(title_text='Sales ($)', row=2, col=2)
71
```

```
fig.show()
```

### 3.11.2 Student Performance Dashboard

```
1 import plotly.express as px
2 import plotly.graph_objects as go
3 from plotly.subplots import make_subplots
4 import pandas as pd
5 import numpy as np
6
7 np.random.seed(42)
8 students = [f'Student_{i}' for i in range(1, 51)]
9 subjects = ['Math', 'Science', 'English', 'History']
10
11 df_students = pd.DataFrame({
12     'Student': np.repeat(students, 4),
13     'Subject': np.tile(subjects, 50),
14     'Score': np.random.randint(50,100,200),
15     'Attendance': np.random.randint(60,100,200),
16     'Study_Hours': np.random.randint(1,10,200)
17 })
18
19 subject_avg = df_students.groupby('Subject')['Score'].mean().reset_index()
20 attendance_corr = df_students.groupby('Student')[['Attendance', 'Score']].mean().reset_index()
21
22 fig = make_subplots(
23     rows=2, cols=2,
24     subplot_titles=('Average Score by Subject', 'Score Distribution',
25                     'Attendance vs Score', 'Study Hours vs Score'),
26     specs=[[{'type': 'bar'}, {'type': 'histogram'}],
27            [{'type': 'scatter'}, {'type': 'scatter'}]]
28 )
29
30 fig.add_trace(
31     go.Bar(x=subject_avg['Subject'], y=subject_avg['Score'],
32            marker_color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']),
33     row=1, col=1
34 )
35
36 fig.add_trace(
37     go.Histogram(x=df_students['Score'], nbinsx=20, marker_color='skyblue'),
38     row=1, col=2
39 )
40
41 fig.add_trace(
42     go.Scatter(x=attendance_corr['Attendance'], y=attendance_corr['Score'],
43               mode='markers', marker=dict(color='green', size=8, opacity=0.7)),
44     row=2, col=1
45 )
46
47 fig.add_trace(
48     go.Scatter(x=df_students['Study_Hours'], y=df_students['Score'],
49               mode='markers', marker=dict(color='purple', size=5, opacity=0.5)),
50     row=2, col=2
51 )
52
53 fig.update_layout(title_text='Student Performance Dashboard',
54                   height=700,
55                   showlegend=False)
56
57 fig.show()
```

## 3.12 Saving Dashboards

### 3.12.1 Save as HTML

```
1 fig.write_html('dashboard.html')
2
3 fig.write_html(
4     'my_dashboard.html',
5     include_plotlyjs='cdn',
6     full_html=True,
7     auto_open=True
8 )
```

### 3.12.2 Save as Image

```
1 fig.write_image('dashboard.png')
2 fig.write_image('dashboard.pdf')
3 fig.write_image('dashboard.svg')
```

### 3.12.3 Embed in Jupyter Notebook

```
1 fig.show()
2
3 from IPython.display import display
4 display(fig)
```

## 3.13 Quick Reference Card

### 3.13.1 Chart Types

| Chart Type   | Function       | Purpose                        |
|--------------|----------------|--------------------------------|
| Line Chart   | px.line()      | Trends over time               |
| Bar Chart    | px.bar()       | Compare categories             |
| Scatter Plot | px.scatter()   | Relationship between variables |
| Histogram    | px.histogram() | Data distribution              |
| Box Plot     | px.box()       | Five-number summary            |
| Pie Chart    | px.pie()       | Proportions                    |
| Heatmap      | px.imshow()    | Correlation matrix             |
| Area Chart   | px.area()      | Cumulative trends              |

### 3.13.2 Dashboard Features

| Feature         | Method                           |
|-----------------|----------------------------------|
| Multiple plots  | make_subplots()                  |
| Dropdown filter | update_layout(update_menus=...)  |
| Range slider    | xaxis_rangeslider_visible=True   |
| Custom hover    | update_traces(hovertemplate=...) |
| Save HTML       | write_html()                     |

## 3.14 Best Practices

- Use clear titles
- Label axes properly
- Use consistent colors
- Keep dashboards simple
- Add filters for interactivity
- Choose correct chart types
- Include legends
- Test interactivity